UNIVERSIDAD FRANCISCO GAVIDIA

Powered by Arizona State University®

Proyecto:

Nervio

Docente:

Silvia Janeth Rico De Serrano

Asignatura:

Aplicación De Técnica De Ingeniería De Software - Grupo 01

Estudiantes:

Cornejo Raymundo, Grecia Michelle - CR100723

Cruz García, Melanie Adriana - CG100223

Coreas Hernández, Fátima Sarahi - CH100723

Esquivel Ramírez, Jesús Gerardo - ER100523

Mejía Reinosa, Ronald Eduardo - MR100823

Orellana Morán, Oscar Alejandro - OM100123

Fecha: 24 de agosto de 2026

Contenido

Marco metodológico 6

Entorno de trabajo y herramientas 6

Stack tecnológico actualizado 6

Modelado conceptual de análisis de requerimientos 7

Diagramas UML 8

Arquitectura Lógica de Módulos 8

Diagrama de Clases del Dominio 8

Reglas y cardinalidades del modelo 9

Diagrama de casos de uso 10

Diagramas de procesos de negocio 24

Flujo conceptual de autenticación 24

Diagrama de actividades: jornada del usuario 25

Diagrama de secuencia: entrevista interactiva 26

Diagrama de canales: ejecución de una entrevista 27

Matriz de riesgos y mitigación 28

Estrategia de prevención y gestión de bloqueos de desarrollo 30

Sprint 1: Acceso seguro y arranque de entrevista 31

Sprint Goal 31

Producto backlog: 31

Historias de usuario 33

NER-6: Registro de cuenta del candidato 33

Subtareas asociadas.....35

NER-7: Inicio de sesión seguro del candidato.....41

Subtareas asociadas.....43

NER-9: Configuración del contexto de entrevista 45

Subtareas asociadas.....47

NER-11: Creación de sesión de entrevista 49

Subtareas asociadas.....51

NER-8: Recuperación de contraseña 54

Subtareas asociadas.....56

Partición del Modelo en Subsistemas y Componentes 60

Arquitectura del sistema 60

Descomposición en Módulos y Subsistemas 62

Subsistema de Presentación y Experiencia de Usuario (UI/UX) 62

Subsistema de Autenticación y Control de Acceso 62

Subsistema de Gestión del Ciclo de Vida de Entrevistas 63

Subsistema de Conversación y Voz en Tiempo Real 63

Subsistema de Persistencia y Acceso a Datos 64

Subsistema de Automatización, Evaluaciones y Procesos Asíncronos 64

Mecanismos de Control y Recursos Globales 68

Gestión de Seguridad (Autenticación y Autorización) 68

Manejo Global de Errores y Tolerancia a Fallos 68

Gestión de Conexiones a la Base de Datos 69

Matriz de Consistencia y Trazabilidad con la Fase 1 70

Diseño de la Base de Datos (Estrategia de Administración de Datos).....71

Modelo Relacional (MR) / Esquema Físico: 71

Diccionario de datos 74

Diseño de la interfaz de usuario (UI/UX).....85

Prototipos de alta fidelidad / wireframes 85

Guia de estilos y navegación. 88

Mapa de pantallas.....94

Mapeo de clases y responsabilidades técnicas 96

Arquitectura de Clases y Principios de Diseño 96

Diagrama de Clases de Diseño (DCD).....97

Definición de Controladores y APIs (Capa de Control).....97

Definición de Servicios de Negocio (Capa de Dominio).....99

Definición de Repositorios de Acceso a Datos (Capa de Persistencia) 102

Matriz de Responsabilidades y Patrones de Asignación (GRASP / SOLID).....106

Bibliografías 108

Anexos 109

Urls de jira: backlog 109

Marco metodológico

Product Owner (PO):

- Ronald Eduardo Mejía Reinosa

Scrum Master (SM):

- Jesús Gerardo Esquivel Ramírez

Equipo de Desarrollo (Developers):

- Grecia Michelle Cornejo Raymundo – **Backend**
- Melanie Adriana Cruz García – **Frontend/QA**
- Fátima Sarahi Coreas Hernández – **Frontend**
- Oscar Alejandro Orellana Morán – **DBA/QA**

Cadencia del proyecto: sprints de dos semanas con pausas de 2 dias entre sprint.

Entorno de trabajo y herramientas

Repositorio Github publico

Jira Space - Aclaración, para ver este tablero debe crear una cuenta en Jira, se le envió una invitación a **sserrano@ufg.edu.sv,** actualmente tiene **Role** de **Guest - Collaborator**

Stack tecnológico actualizado

La arquitectura actual de la plataforma se basa en tecnologías modernas impulsadas por el ecosistema de frontend y automatizaciones serverless. Se ha prescindido de un backend monolítico tradicional en favor de una arquitectura ágil, centralizando la lógica en el framework de React:

- **Frontend / Orquestador principal:**
  - **Framework:** Next.js (App Router), manejando tanto la interfaz de usuario como las rutas de API internas (/api/*).

- **Librerías y estilos:** React y Tailwind CSS.
- **Manejo de estado:** estado reactivo mediante Hooks (useState, useConversation) y persistencia local (localStorage/sessionStorage).

- **Autenticación:**
  - Better Auth integrado en Next.js, parcialmente implementado y pendiente de completarse.
- **Base de datos:**
  - PostgreSQL gestionado a través de Prisma ORM.
- **Backend lógico:**
  - Las rutas de la API de Next.js (/api/interview/*) actúan como controladores de negocio, procesan el contexto inicial y gestionan el ciclo de vida de la sesión.
- **Integración de IA (voz y conversación):**
  - ElevenLabs (ConvAI): conexión directa vía WebSockets desde el frontend mediante el SDK @elevenlabs/react para lograr latencia baja en streaming de voz bidireccional.
- **Almacenamiento de archivos (próximos sprints):**
  - AWS S3 o Azure blob storage para almacenar de forma segura los archivos físicos de los CV en formatos PDF o DOCX.

Modelado conceptual de análisis de requerimientos

Nota. Los siguientes diagramas modelan el sistema objetivo del MVP, incluyendo características que actualmente se encuentran en fase de desarrollo en el backlog, como login completo y carga de CV.

Diagramas UML

Arquitectura Lógica de Módulos

Representa los bloques funcionales primarios del sistema y cómo se comunican a alto nivel, abstrayendo las herramientas técnicas (Frameworks, ORMs, librerías).

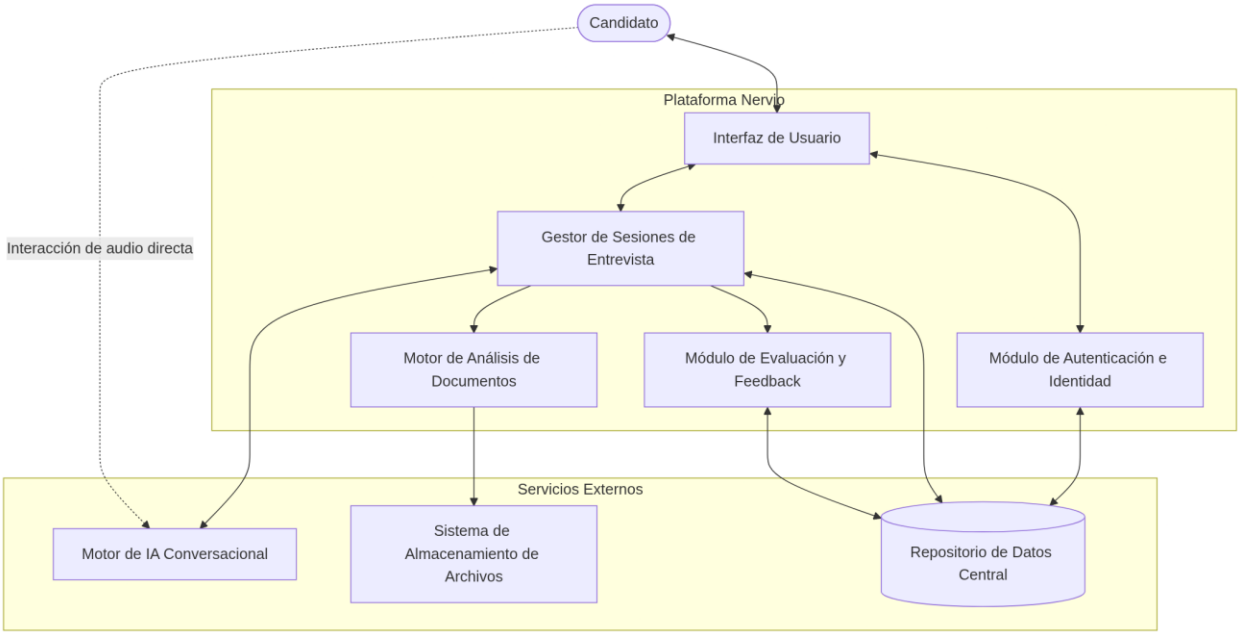


Diagrama Arquitectura Lógica de Módulos en svg

Diagrama de Clases del Dominio

Este diagrama representa la estructura estática del sistema: clases, atributos principales, operaciones relevantes y relaciones entre objetos. A diferencia del modelo conceptual, este diagrama se alinea con las entidades persistentes definidas en schema.prisma, incluyendo las clases de autenticación de Better Auth y las entidades del dominio de entrevistas. Los nombres de las tablas se indican entre comillas cuando difieren del nombre de la clase.

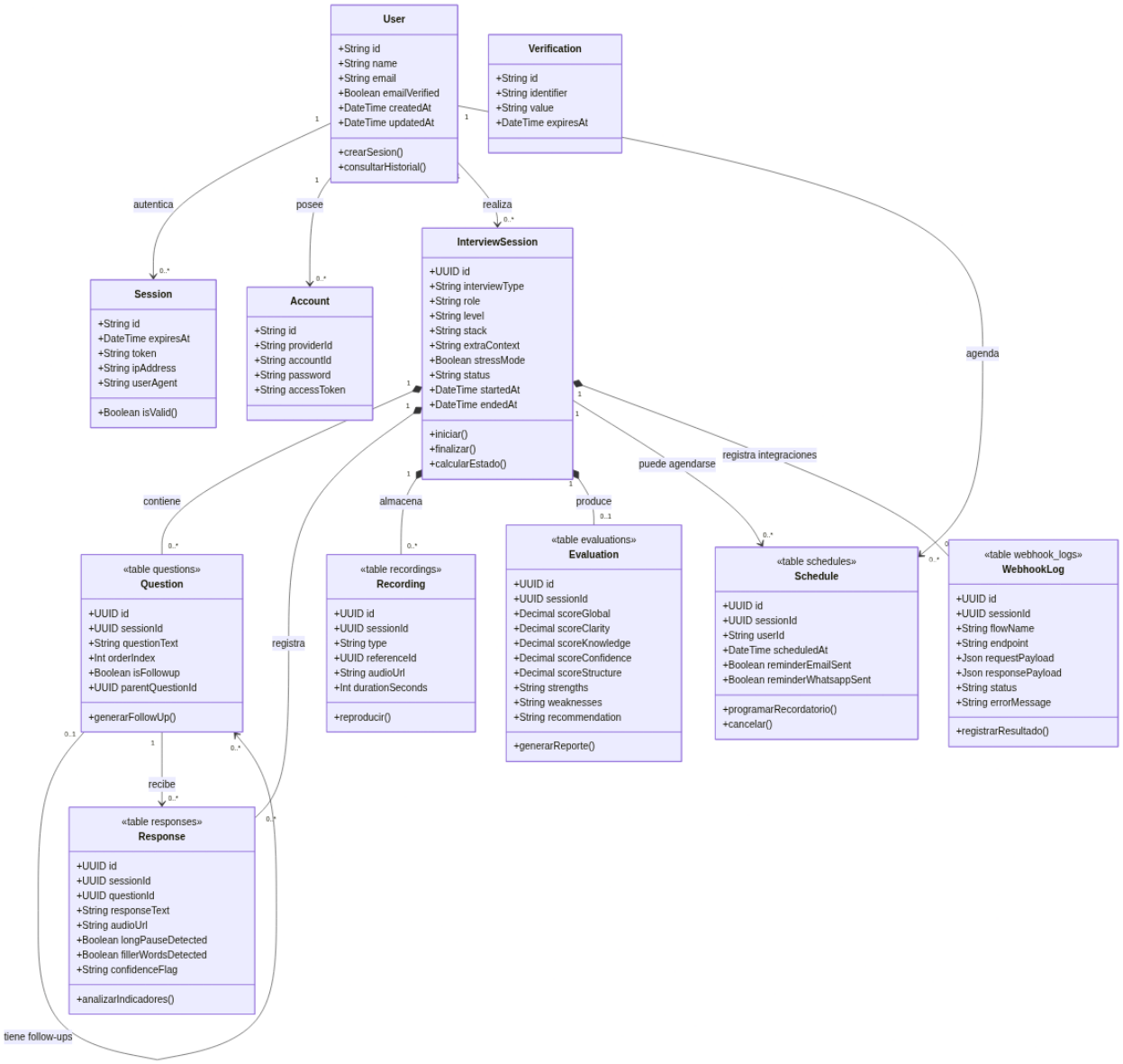


Diagrama de Clases del Dominio en svg

Reglas y cardinalidades del modelo

Relación	**Regla de negocio**
User - Session	Un usuario puede tener varias sesiones de autenticación; cada sesión pertenece a un único usuario y se elimina en cascada al eliminarlo.
User - InterviewSession	Un candidato puede realizar muchas entrevistas. La relación admite userId nulo para soportar sesiones públicas o datos heredados, aunque el flujo autenticado debe asociarlo siempre.

InterviewSession - Question	Una entrevista contiene cero o muchas preguntas persistidas; una pregunta puede tener preguntas hijas para follow-ups.
Question - Response	Una pregunta puede recibir varias respuestas registradas por reintentos o turnos; cada respuesta debe apuntar a una pregunta y a su sesión.
InterviewSession - Evaluation	Una sesión tiene como máximo una evaluación final (sessionId es único).
InterviewSession - Recording	Una sesión puede conservar múltiples audios de preguntas, respuestas o feedback.
Schedule	Una reserva puede vincularse a una sesión y a un usuario; la fecha debe ser futura y sus indicadores controlan los recordatorios.
WebhookLog	Registra cada integración y permite auditar respuestas, errores y reintentos sin bloquear el flujo principal.

Las clases User, Session, Account y Verification pertenecen a la autenticación. Las clases InterviewSession, Question, Response, Recording, Evaluation, Schedule y WebhookLog pertenecen al dominio de entrevistas. CurriculumVitae no se incluye todavía porque la carga y extracción del CV están fuera del MVP actual.

Diagrama de casos de uso

Identifica los actores principales y sus interacciones funcionales con el sistema.

Figura 1

Diagrama de casos de uso del sistema Nervio.

Nota. Representa los actores principales y sus interacciones funcionales con la plataforma. Fuente: elaboración propia.

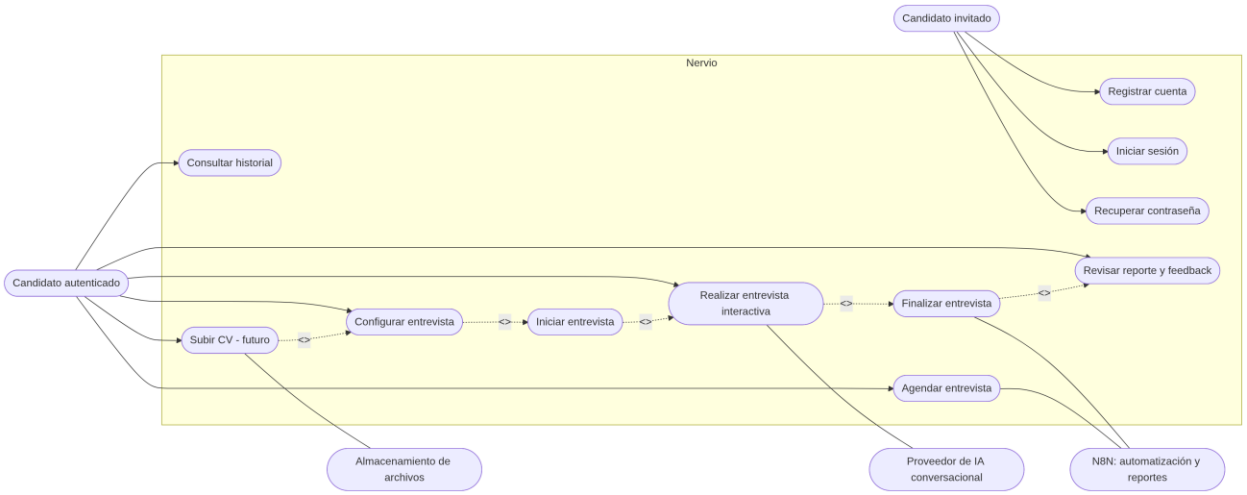


Diagrama de casos de uso del sistema Nervio en svg

Actores.

Tabla 1

Actores del sistema e interacción principal.

Actor	**Tipo**	**Responsabilidad e interacción**
Candidato invitado	Primario	Se registra, inicia sesión o solicita recuperación de contraseña. No puede acceder a sesiones ni reportes privados.
Candidato autenticado	Primario	Configura, inicia, realiza y finaliza entrevistas; consulta reportes e historial y puede agendar una sesión.
Proveedor de IA conversacional	Secundario externo	Mantiene el canal de voz, formula preguntas y devuelve respuestas de la entrevista en tiempo real.

Almacenamiento de archivos	Secundario externo planificado	Conserva los CV cargados cuando esta capacidad sea habilitada.

Catálogo y trazabilidad de casos de uso.

Tabla 2

Catálogo y trazabilidad de casos de uso.

ID	**Caso de uso**	**Actor principal**	**Prioridad / estado**	**Requisitos funcionales relacionados**
CU-01	Registrar cuenta	Candidato invitado	MVP	RF-AUTH-01, RF-AUTH-02
CU-02	Iniciar sesión	Candidato invitado	MVP	RF-AUTH-03, RF-AUTH-04
CU-03	Recuperar contraseña	Candidato invitado	Planificado	RF-AUTH-05
CU-04	Configurar entrevista	Candidato autenticado	MVP	RF-INT-01, RF-INT-02
CU-05	Iniciar entrevista	Candidato autenticado	MVP	RF-INT-03, RF-INT-04
CU-06	Realizar entrevista interactiva	Candidato autenticado, IA	MVP	RF-INT-05, RF-INT-06, RF-INT-07
CU-07	Finalizar entrevista y generar reporte	Candidato autenticado	MVP	RF-INT-08, RF-REP-01, RF-REP-02
CU-08	Revisar reporte y feedback	Candidato autenticado	MVP	RF-REP-03
CU-09	Consultar historial	Candidato autenticado	MVP	RF-HIS-01

CU-10	Agendar entrevista	Candidato autenticado	MVP	RF-SCH-01, RF-SCH-02
CU-11	Subir CV	Candidato autenticado, almacenamiento	Futuro	RF-CV-01, RF-CV-02

Fichas detalladas de casos de uso.

Las fichas siguientes constituyen la especificación verificable del diagrama. El **resultado esperado** es la salida principal que debe cumplirse para considerar exitoso el caso; el **resultado secundario** es una salida adicional, una notificación o una consecuencia persistida que también debe contemplarse.

CU-01. Registrar cuenta

Campo	**Especificación**
Objetivo	Crear una cuenta de candidato para acceder a las funciones privadas.
Actores	**Principal**: Candidato invitado. **Interno**: módulo de autenticación y base de datos.
Precondiciones	**El candidato** no tiene **una cuenta** con el mismo correo y se encuentra en la pantalla de registro.
Requerimientos	**RF-AUTH-01:** solicitar nombre, correo y contraseña. **RF-AUTH-02:** validar formato, unicidad y política mínima de contraseña; almacenar la contraseña de forma segura.
Flujo principal	1. El candidato envía sus datos. 2. Nervio valida los campos. 3. El módulo de autenticación crea el usuario.

	4. Se establece una sesión segura.
Resultado esperado	La cuenta queda creada y **el candidato queda autenticado** en el panel principal.
Resultado secundario	Se muestra confirmación y se registra la fecha de alta; ante correo duplicado se informa el error sin crear otra cuenta.

CU-02. Iniciar sesión

Campo	**Especificación**
Objetivo	Verificar la identidad y **permitir el acceso** a los datos propios del candidato.
Actores	**Principal:** Candidato invitado. Interno: módulo de autenticación y base de datos.
Precondiciones	La cuenta existe y el candidato no tiene una sesión válida.
Requerimientos	**RF-AUTH-03:** validar correo y contraseña. **RF-AUTH-04:** emitir cookie de sesión HttpOnly, Secure y con expiración controlada.
Flujo principal	1. El candidato envía credenciales. 2. Nervio las verifica. 3. Crea o renueva la sesión. 4. Redirige al panel.
Resultado esperado	El **candidato accede** únicamente a sus funciones y recursos privados.
Resultado secundario	Se actualiza la última actividad; ante credenciales inválidas se rechaza el acceso sin revelar si el correo existe.

CU-03. Recuperar contraseña

Campo	**Especificación**
Objetivo	Permitir que un candidato recupere el acceso a su cuenta.
Actores	Principal: Candidato invitado. Secundario: servicio de correo.
Precondiciones	El candidato conoce el correo asociado a la cuenta.
Requerimientos	**RF-AUTH-05:** generar un **token de un solo uso**, con expiración, y enviar un enlace de recuperación.
Flujo principal	1. El candidato solicita recuperación. 2. Nervio genera el token. 3. Envía el enlace. 4. El candidato define una nueva contraseña.
Resultado esperado	La contraseña se actualiza y el **token deja de ser utilizable**.
Resultado secundario	Se invalidan sesiones anteriores y se muestra una respuesta genérica aunque el correo no esté registrado.

CU-04. Configurar entrevista

Campo	**Especificación**
Objetivo	Definir el contexto con el que se personalizará la simulación.
Actores	Principal: Candidato autenticado. Extensión: almacenamiento de CV.
Precondiciones	Existe una sesión autenticada válida.
Requerimientos	**RF-INT-01:** solicitar tipo de entrevistador, rol, seniority y stack.

	RF-INT-02: aceptar contexto adicional y modo estrés; validar valores y longitud.
Flujo principal	1. El candidato selecciona parámetros. 2. Nervio valida la configuración. 3. Conserva la configuración en el formulario o sesión de preparación.
Resultado esperado	Existe una **configuración válida lista** para iniciar una entrevista.
Resultado secundario	Se muestran errores de validación por campo y el CV puede adjuntarse como extensión cuando se habilite.

CU-05. Iniciar entrevista

Campo	**Especificación**
Objetivo	Crear una sesión y preparar el canal de conversación con IA.
Actores	Principal: **Candidato autenticado**. Secundarios: proveedor de IA.
Precondiciones	El candidato está autenticado y la configuración es válida.
Requerimientos	**RF-INT-03:** crear sessionId asociado al userId. **RF-INT-04:** solicitar preguntas iniciales o token seguro del agente y cambiar el estado a in_progress.
Flujo principal	1. Nervio persiste la sesión. 2. Solicita la preparación necesaria. 3. Obtiene autorización del agente. 4. Informa que la entrevista comenzó.
Resultado esperado	La **sesión queda iniciada**, asociada al candidato y lista para intercambiar audio.

Resultado secundario	Se **registra la hora de inicio**; si falla un proveedor, la sesión no se presenta como activa y se informa una recuperación posible.

CU-06. Realizar entrevista interactiva

Campo	**Especificación**
Objetivo	Mantener el ciclo de pregunta, respuesta y réplica hablada adaptada al contexto.
Actores	Principal: Candidato autenticado. Secundario: Proveedor de IA conversacional.
Precondiciones	La sesión está en in_progress y el canal de audio está disponible.
Requerimientos	**RF-INT-05:** transmitir audio en tiempo real. **RF-INT-06:** transcribir y conservar cada respuesta. **RF-INT-07:** adaptar preguntas y activar el modo estrés según la configuración y las señales detectadas.
Flujo principal	1. La IA formula una pregunta. 2. El candidato escucha y responde. 3. Nervio transcribe y envía el contexto. 4. La IA procesa y responde. 5. El ciclo se repite.
Resultado esperado	Se **registra una conversación coherente** con preguntas, respuestas y tiempos mientras la sesión permanece activa.
Resultado secundario	Se **guardan métricas parciales** y se muestra el estado de conexión; ante corte se permite reconexión o finalización controlada.

CU-07. Finalizar entrevista y generar reporte

Campo	**Especificación**
Objetivo	Cerrar la interacción y solicitar la evaluación final.
Actores	Principal: Candidato autenticado.
Precondiciones	La sesión está activa o expiró por el límite de tiempo.
Requerimientos	**RF-INT-08:** cerrar el canal y marcar la sesión como finalizada. **RF-REP-01:** conservar la conversación. **RF-REP-02:** disparar el flujo de reporte final.
Flujo principal	1. El candidato finaliza o se alcanza el límite. 2. Nervio cierra la conexión. 3. Persiste respuestas y métricas. 4. Sistema analiza la sesión y guarda la evaluación.
Resultado esperado	La **sesión queda en estado completed** y existe un reporte final asociado o en estado de procesamiento.
Resultado secundario	El candidato recibe una notificación de procesamiento; si falla, la sesión permanece cerrada y el reporte puede reintentarse.

CU-08. Revisar reporte y feedback

Campo	**Especificación**
Objetivo	Consultar el desempeño obtenido en una entrevista finalizada.
Actores	Principal: Candidato autenticado.
Precondiciones	El candidato es propietario de la sesión y el reporte está disponible.

Requerimientos	**RF-REP-03:** mostrar puntuación global, categorías, fortalezas, debilidades y recomendación; impedir el acceso a sesiones de otros usuarios.
Flujo principal	1. El candidato selecciona una sesión. 2. Nervio verifica propiedad. 3. Recupera la evaluación. 4. Presenta el reporte y, si existe, el feedback hablado.
Resultado esperado	El **candidato visualiza el reporte** correcto y completo de su sesión.
Resultado secundario	Puede reproducir el feedback o ver un estado pendiente cuando la evaluación aún no termina.

CU-09. Consultar historial

Campo	**Especificación**
Objetivo	Consultar sesiones y resultados previos para observar el progreso.
Actores	Principal: Candidato autenticado.
Precondiciones	Existe una sesión autenticada válida.
Requerimientos	**RF-HIS-01:** listar únicamente sesiones propias, ordenadas por fecha, con estado, rol y puntuación disponible.
Flujo principal	1. El candidato abre el historial. 2. Nervio consulta sus sesiones. 3. Presenta el listado y permite abrir un reporte.
Resultado esperado	Se **muestra un historial privado**, ordenado y consistente con las sesiones almacenadas.

Resultado secundario	Si **no existen sesiones**, se muestra un estado vacío con la acción para configurar una entrevista.

CU-10. Agendar entrevista

Campo	**Especificación**
Objetivo	Reservar una sesión futura con una configuración determinada.
Actores	Principal: Candidato autenticado. Secundario: servicio de notificaciones.
Precondiciones	El candidato está autenticado, la configuración es válida y la fecha está en el futuro.
Requerimientos	**RF-SCH-01:** persistir fecha, configuración, usuario y estado. RF-SCH-02: notificar o programar recordatorios.
Flujo principal	1. El candidato selecciona fecha y hora. 2. Nervio valida y persiste la reserva. 3. Notifica a correo. 4. Confirma la reserva.
Resultado esperado	Se **crea una sesión agendada** con identificador, fecha y configuración asociada.
Resultado secundario	Se programa un recordatorio; si la notificación falla, la reserva se conserva y queda marcada para reintento.

CU-11. Subir CV (funcionalidad futura)

Campo	**Especificación**

Objetivo	Cargar un CV para enriquecer el contexto de una entrevista.
Actores	Principal: Candidato autenticado. Secundario: almacenamiento de archivos y motor de extracción.
Precondiciones	La cuenta está autenticada y el archivo es PDF o Word dentro del límite permitido.
Requerimientos	**RF-CV-01:** validar extensión, tamaño y tipo MIME. **RF-CV-02:** almacenar el archivo de forma privada y extraer texto cuando sea posible.
Flujo principal	1. El candidato selecciona el archivo. 2. Nervio valida y lo envía al almacenamiento. 3. Registra la referencia. 4. Extrae texto para futuras configuraciones.
Resultado esperado	El **CV queda asociado al candidato** y disponible para personalizar una entrevista.
Resultado secundario	Se **informa el estado de extracción**; los documentos escaneados o inválidos se rechazan sin conservar archivos incompletos.

Requisitos funcionales identificados.

Los identificadores usados en las fichas anteriores deben convertirse en requisitos verificables del backlog. Esta tabla evita que un caso de uso quede como una descripción aislada sin vínculo con implementación y pruebas.

Tabla 4

Requisitos funcionales identificados y criterios de aceptación.

ID	Requisito funcional	Casos relacionados	Criterio de aceptación resumido
RF-AUTH-01	Registrar datos básicos del candidato	CU-01	Se rechazan campos ausentes o con formato inválido.
RF-AUTH-02	Crear identidad y sesión segura después del registro	CU-01	El usuario creado puede acceder sin exponer su contraseña.
RF-AUTH-03	Validar credenciales de acceso	CU-02	Credenciales válidas permiten acceso; inválidas lo rechazan.
RF-AUTH-04	Proteger la sesión y los recursos privados	CU-02, CU-08, CU-09	Un usuario no puede consultar datos de otro usuario.
RF-AUTH-05	Recuperar contraseña mediante token temporal	CU-03	El token expira, es de un solo uso y actualiza la contraseña.
RF-INT-01	Capturar el contexto mínimo de la entrevista	CU-04	Se guardan tipo, rol, seniority y stack válidos.
RF-INT-02	Validar contexto adicional y modo estrés	CU-04	La configuración inválida no permite iniciar la sesión.
RF-INT-03	Crear sesión asociada al candidato	CU-05	La sesión tiene sessionId, userId, estado y fecha.

RF-INT-04	Preparar agente y canal seguro de IA	CU-05	La entrevista solo inicia si el canal está autorizado.
RF-INT-05	Intercambiar audio en tiempo real	CU-06	El sistema reproduce la pregunta y recibe la respuesta.
RF-INT-06	Transcribir y persistir respuestas	CU-06	Cada respuesta queda vinculada a su pregunta y sesión.
RF-INT-07	Adaptar la entrevista y modo estrés	CU-06	El comportamiento respeta la configuración y métricas disponibles.
RF-INT-08	Cerrar sesión y conservar el estado final	CU-07	Una sesión cerrada no acepta nuevas respuestas.
RF-REP-01	Persistir conversación y métricas	CU-07	El cierre conserva los datos necesarios para evaluar.
RF-REP-02	Solicitar reporte final asíncrono	CU-07	Sistema recibe la referencia y puede reintentar ante error.
RF-REP-03	Mostrar reporte privado y feedback	CU-08	Se muestran categorías, fortalezas, debilidades y recomendación.
RF-HIS-01	Listar historial propio	CU-09	El listado está ordenado y no filtra datos ajenos.

RF-SCH-01	Persistir una reserva futura	CU-10	Se rechazan fechas pasadas y se guarda la configuración.
RF-SCH-02	Programar recordatorios	CU-10	Sistema recibe la reserva y el estado de notificación.
RF-CV-01	Validar archivo de CV	CU-11	Solo se aceptan formatos y tamaños permitidos.
RF-CV-02	Almacenar y extraer texto del CV	CU-11	El archivo es privado y su texto queda asociado al candidato.

Diagramas de procesos de negocio

Flujo conceptual de autenticación

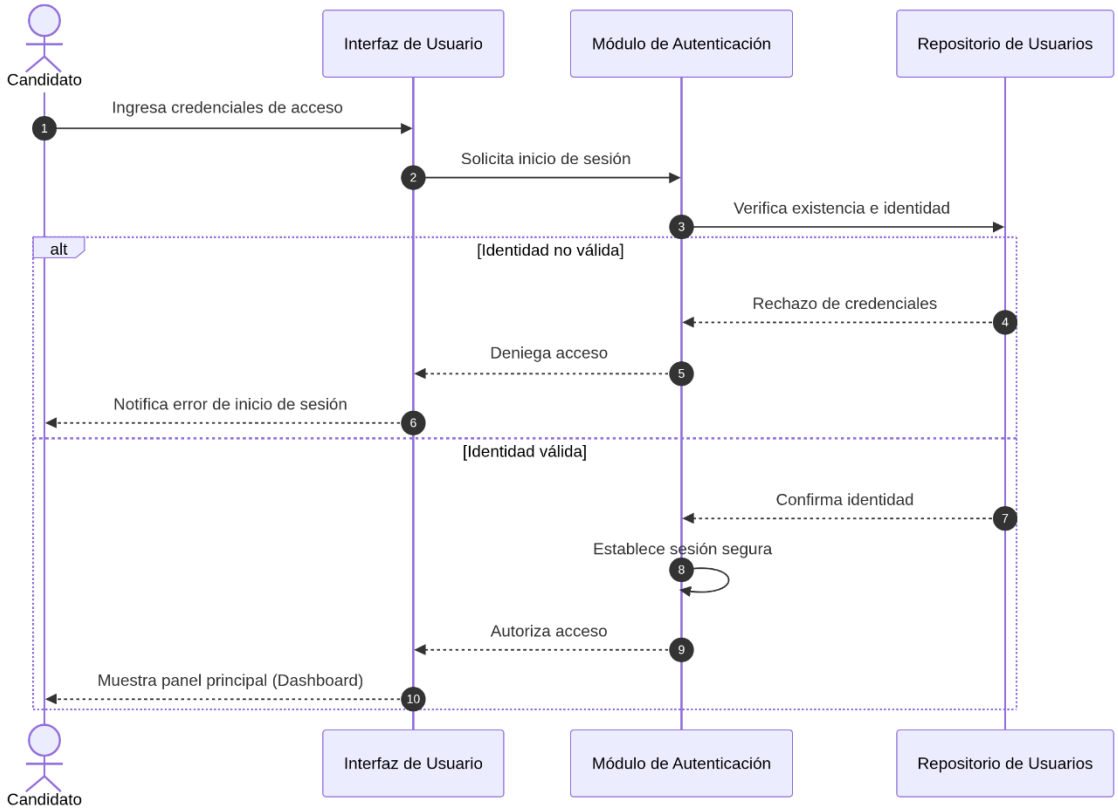
Describe el proceso de validación de identidad que garantiza que el sistema esté protegido, asegurando que solo usuarios reconocidos accedan a la plataforma principal.

Figura 2

Flujo conceptual de autenticación.

Nota. Describe el proceso de validación de identidad y acceso seguro a la plataforma.

Fuente: elaboración propia.



Flujo conceptual de autenticación en svg

Diagrama de actividades: jornada del usuario

Mapea la experiencia del usuario, desde el inicio hasta el final dentro del sistema, independientemente de la tecnología subyacente.

Figura 3

Diagrama de actividades de la jornada del usuario.

Nota. Mapea la experiencia del usuario desde el acceso hasta la finalización de la entrevista. Fuente: elaboración propia.



Diagrama de actividades de la jornada del usuario en svg

Diagrama de secuencia: entrevista interactiva

Ilustra las interacciones de los módulos lógicos principales y actores durante el desarrollo de una sesión de entrevista.

Figura 4

Diagrama de secuencia de entrevista interactiva.

Nota. Ilustra la interacción entre usuario, sistema y proveedor de IA durante una sesión de entrevista. Fuente: elaboración propia.

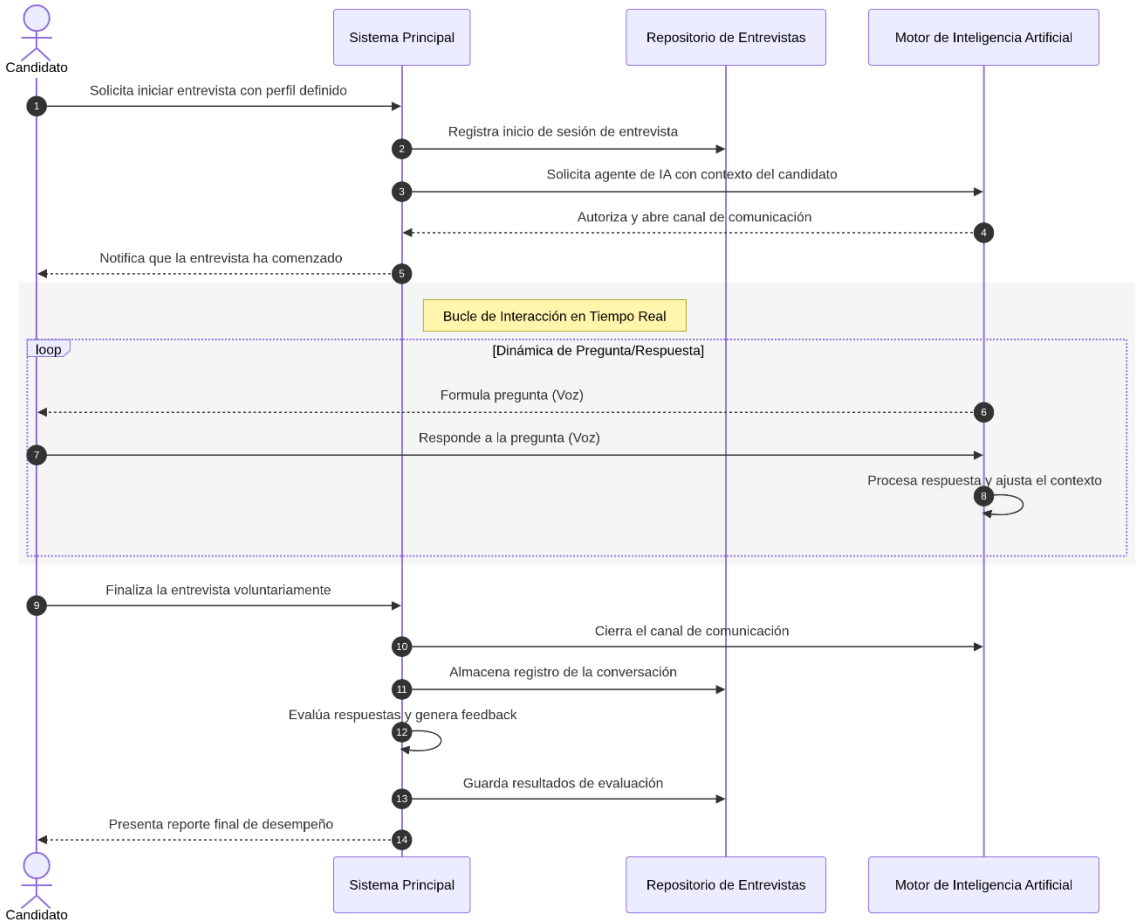


Diagrama de secuencia de entrevista interactiva en svg

Diagrama de canales: ejecución de una entrevista

Este diagrama distribuye las responsabilidades por canal y permite identificar con claridad los puntos de integración que podrían bloquear el desarrollo o la ejecución del MVP.

Figura 5

Diagrama de canales para la ejecución de una entrevista.

Nota. Distribuye las responsabilidades entre frontend, backend, servicios externos y base de datos. Fuente: elaboración propia.

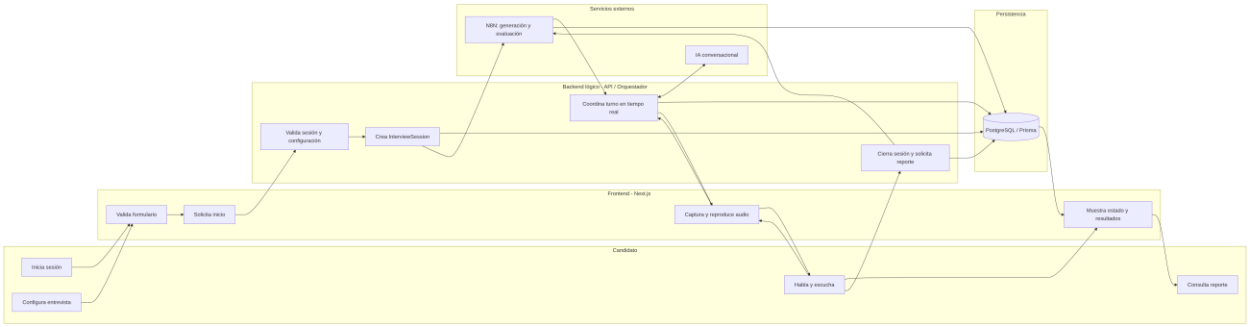


Diagrama Swinlane en svg

El flujo debe respetar estas responsabilidades: el frontend gestiona la interacción, el backend coordina las operaciones sensibles y el tiempo real, los servicios externos procesan IA o tareas diferidas, y la base de datos conserva el estado de la sesión y sus resultados.

Matriz de riesgos y mitigación

Para asegurar la viabilidad técnica del MVP, se han identificado los siguientes riesgos y estrategias de mitigación implementadas o planificadas.

Tabla 3

Matriz de riesgos técnicos y estrategias de mitigación.

#	Riesgo técnico / arquitectónico	Impacto	Probabilidad	Estrategia de mitigación actual / propuesta
1	**Seguridad en la autenticación** (filtración de sesiones o acceso no autorizado a entrevistas).	Alto	Baja	Uso de Better Auth con cookies HttpOnly y Secure. Validación de propiedad de la InterviewSession mediante userId en todas las rutas de API.

2	**Latencia y cortes de red con ElevenLabs** (pérdida de la simulación en tiempo real).	Alto	Media	Utilización de WebSockets directos desde el cliente para baja latencia. Manejo de estados de reconexión y alternativa visual en el componente ActiveInterviewView.
3	**Procesamiento de CV** (OCR / PDF parsing, falla en extracción de texto o formatos no soportados).	Medio	Alta	En la fase MVP, limitar el soporte a PDF generados digitalmente con texto seleccionable mediante pdf-parse en Node.js, descartando imágenes escaneadas por el momento.
4	**Costos de API** (abuso del sistema consumiendo minutos gratuitos o pagos de ElevenLabs).	Alto	Media	Implementar rate limiting en /api/interview/init y limitar la duración máxima de la sesión WebSocket a un tiempo fijo, por ejemplo, 15 minutos.

Estrategia de prevención y gestión de bloqueos de desarrollo

Para evitar que una dependencia o decisión pendiente detenga el avance del equipo, se establece el siguiente protocolo de prevención y gestión de bloqueos durante el desarrollo del MVP.

1. **Identificar:** todo impedimento técnico, funcional o de acceso se registra el mismo día en Jira con descripción, responsable, impacto y dependencia relacionada.
2. **Clasificar:** se etiqueta como BLOCKER, DEPENDENCY, DECISION o ACCESS, y se determina si afecta el camino crítico del sprint.
3. **Desacoplar:** mientras se resuelve la dependencia, se trabaja contra interfaces, mocks, fixtures o datos de ejemplo versionados. Las claves y servicios reales solo se requieren para la validación de integración.
4. **Escalar:** si el bloqueo no se resuelve en una jornada de trabajo, se escala al responsable técnico o al equipo completo y se define una decisión temporal para continuar.
5. **Validar:** una vez resuelto, se ejecuta una prueba focalizada y una prueba de integración para confirmar que el desbloqueo no introdujo regresiones.
6. **Cerrar:** se actualiza la tarea en Jira, se documenta la solución y se elimina el estado de bloqueo únicamente cuando existe evidencia verificable.

Criterio de continuidad: ningún bloqueo externo debe detener todo el sprint. Cada tarea crítica debe contar con una ruta alternativa documentada, como mock local, set de preguntas de respaldo, procesamiento degradado o ejecución manual controlada.

Sprint 1: Acceso seguro y arranque de entrevista

Este sprint se enfoca en entregar las funciones iniciales de acceso seguro: registro, inicio de sesión, recuperación de contraseña, configuración del contexto y creación de una sesión persistente.

Sprint Goal

Entregar acceso seguro y arranque de entrevista: registro, login, recuperación de contraseña, configuración del contexto y creación de sesión persistente.

Producto backlog:

Evidencia general del sprint: las siguientes capturas resumen el alcance planificado para las funcionalidades de autenticación y arranque de entrevista.

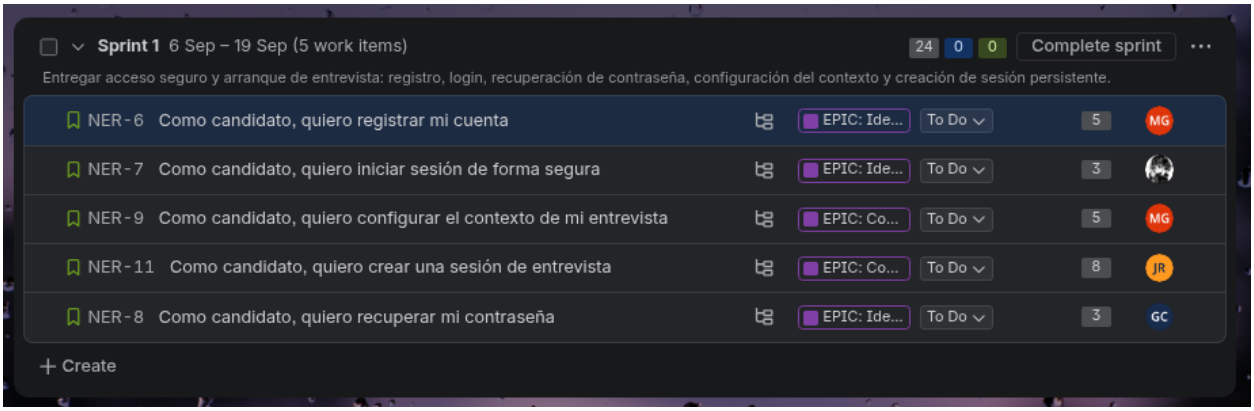


Figura 6

Resumen visual del alcance planificado del Sprint 1

Nota. La figura presenta una vista general del alcance inicial del Sprint 1, destacando las funcionalidades previstas para acceso seguro y arranque de entrevista. Según APA 7, las figuras deben numerarse en el orden en que aparecen y acompañarse de un título breve y descriptivo que permita identificar su propósito dentro del documento.

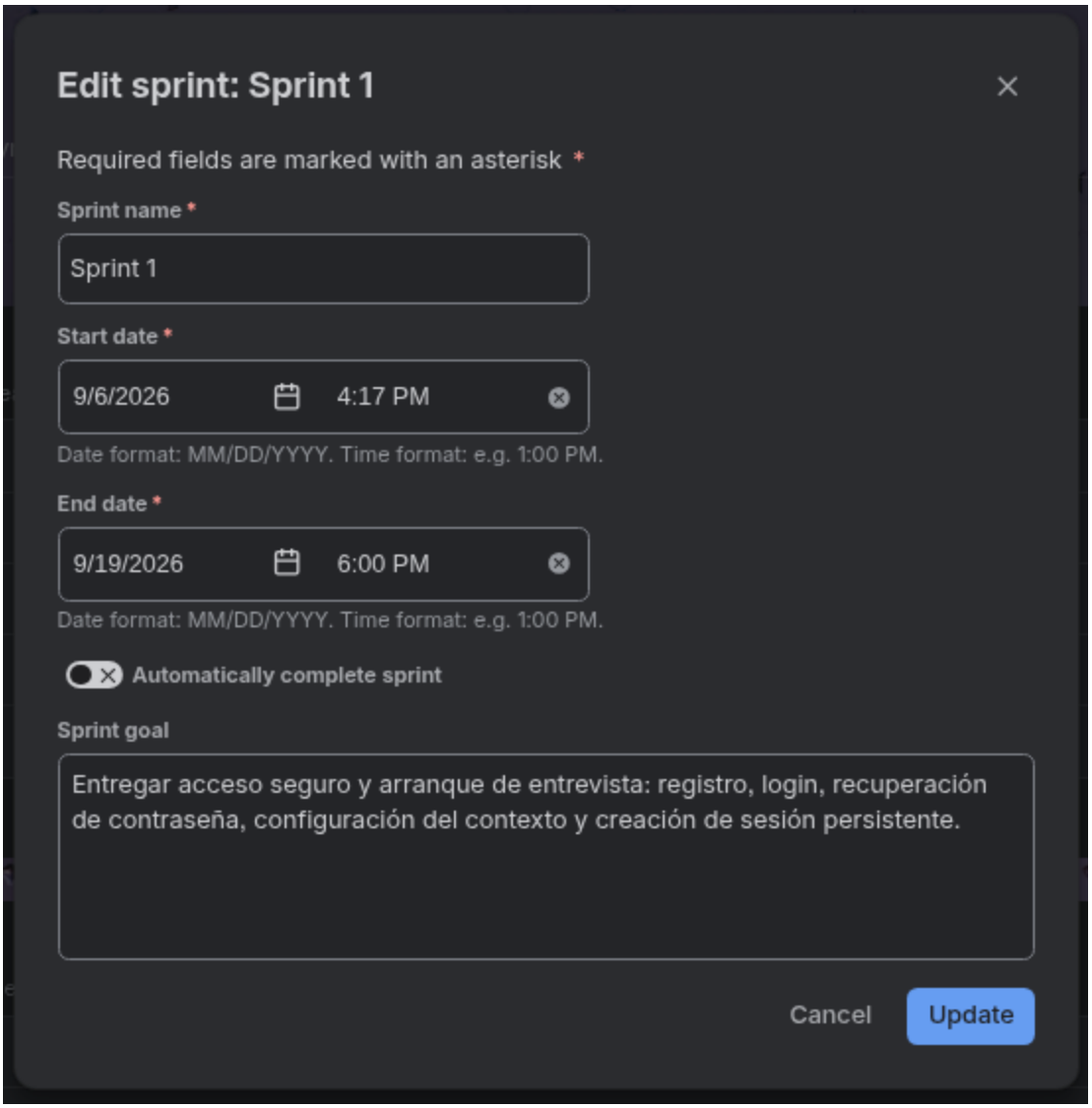


Figura 7

Product Backlog del Sprint 1 en Jira

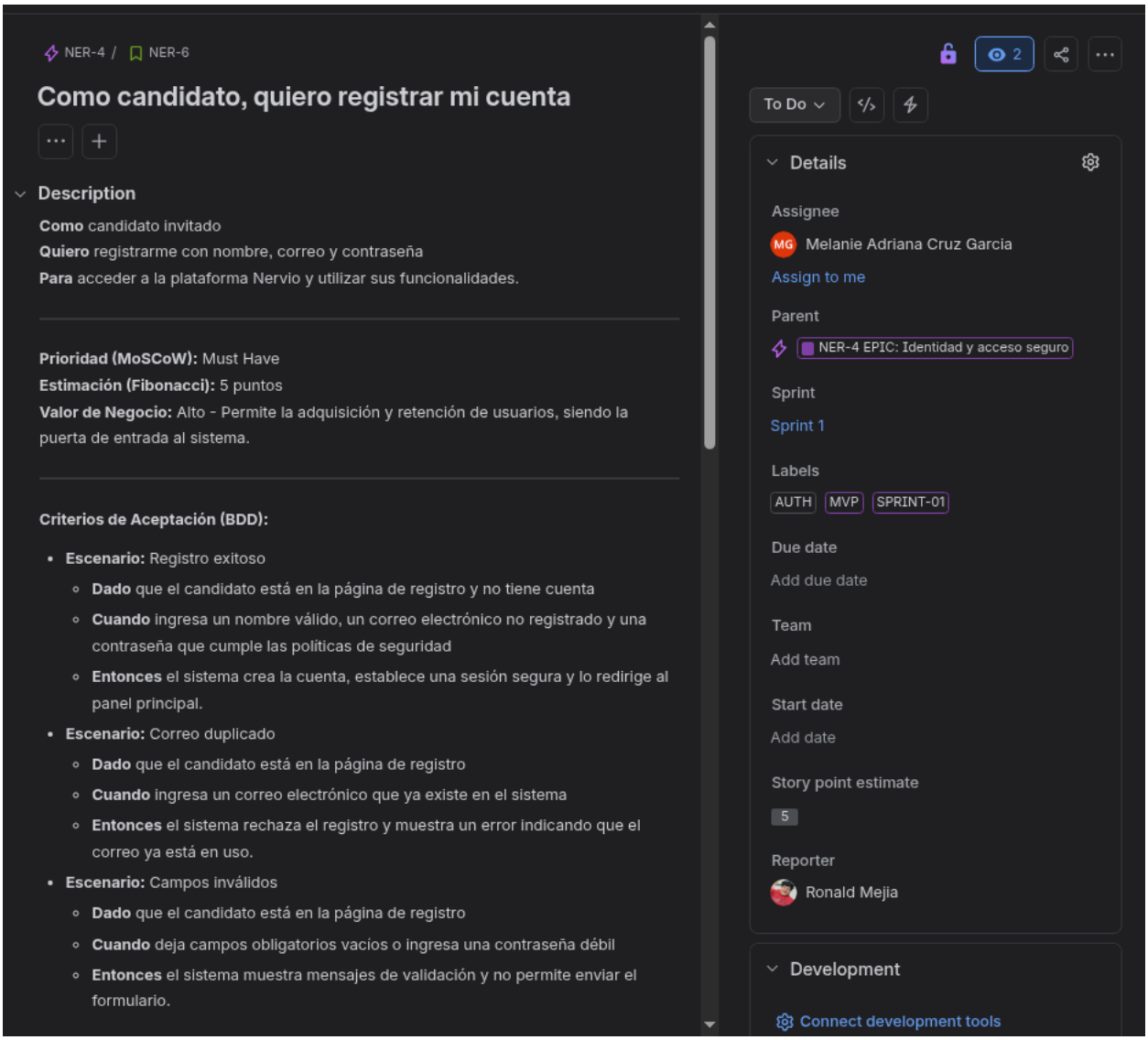
Nota. La figura muestra el backlog del proyecto Nervio en Jira, donde se agrupan y priorizan los elementos de trabajo del Sprint 1. Atlassian describe el backlog Scrum como un espacio para crear, actualizar, ordenar y asignar elementos de trabajo a sprints, lo que permite planificar y dar seguimiento al avance del equipo.

Historias de usuario

NER-6: Registro de cuenta del candidato

Historia de usuario: Como candidato, quiero registrar mi cuenta para poder acceder de forma segura a la plataforma.

Evidencia de registro: las siguientes capturas muestran la interfaz y el flujo principal de creación de cuenta del candidato.



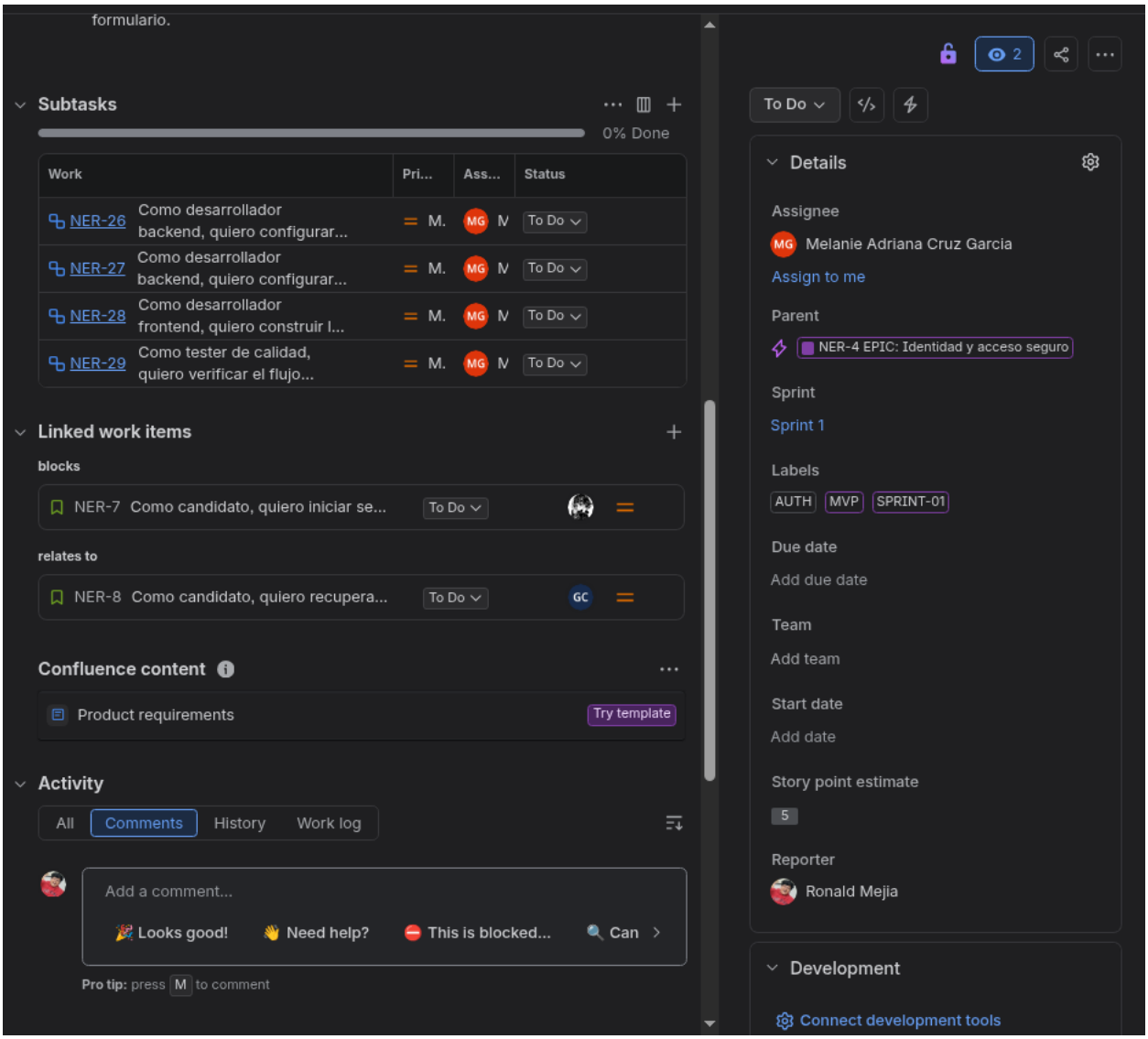


Figura 8

Flujo de registro de cuenta del candidato

Nota. La figura muestra el proceso visual que sigue el candidato para crear una cuenta en la plataforma Nervio, incluyendo pantallas de ingreso de datos y validaciones iniciales. Esta evidencia se relaciona con la historia de usuario NER-6, redactada bajo la lógica de expresar una necesidad desde la perspectiva del usuario final, práctica recomendada en la gestión ágil de requisitos.

Subtareas asociadas

Las subtareas se organizan por componente técnico para mostrar cómo backend, frontend y aseguramiento de calidad contribuyen al registro seguro del candidato.

NER-26: Configuración de Prisma y tablas de autenticación

Evidencia backend: esta captura documenta la configuración de Prisma y las tablas necesarias para soportar la autenticación.

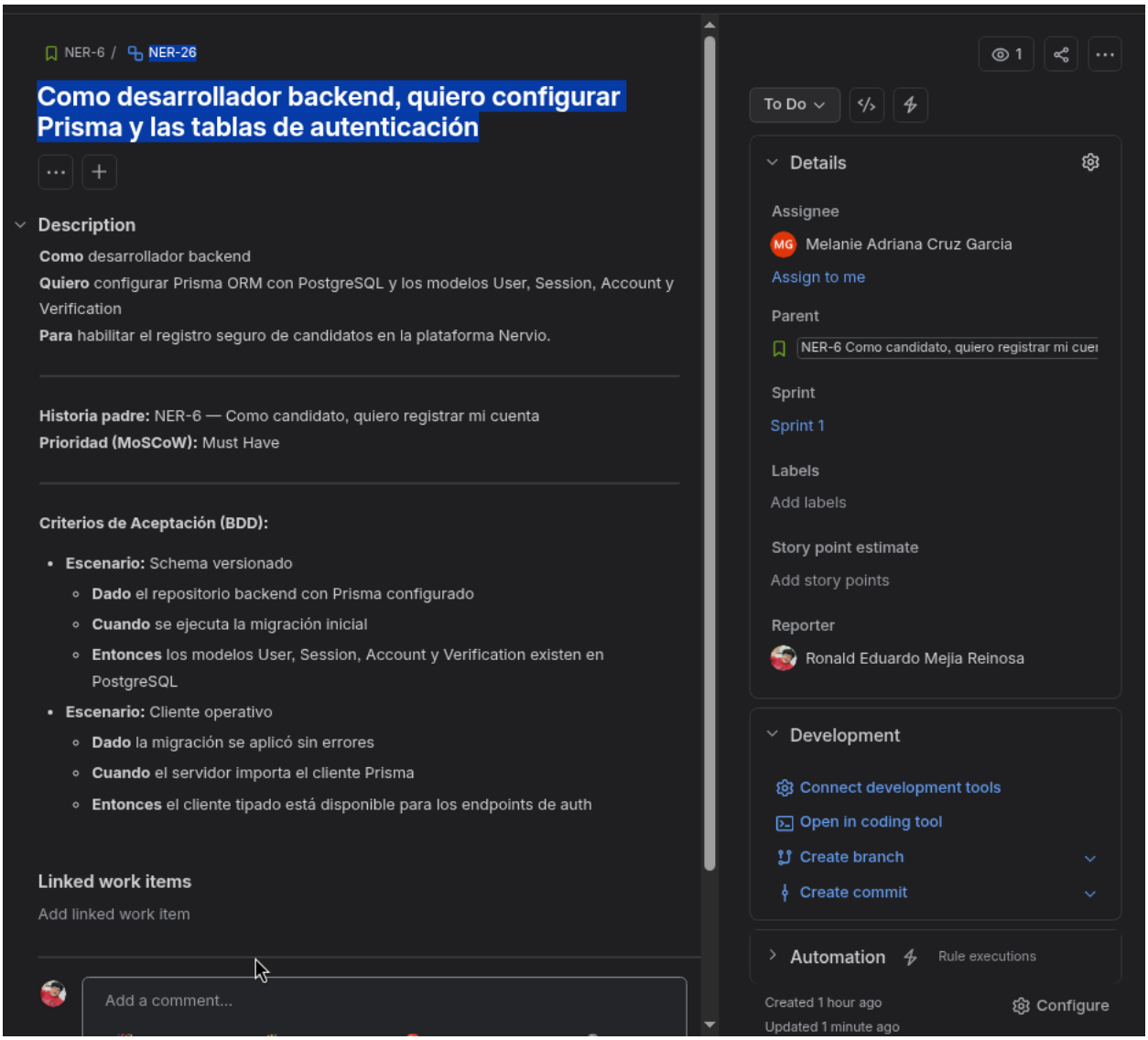


Figura 9

Configuración de Prisma y tablas de autenticación

Nota. La captura evidencia la definición del esquema de datos que soporta usuarios, cuentas, sesiones y credenciales de autenticación. En Prisma, el archivo de esquema centraliza la conexión a la base de datos, los generadores y el modelo de datos de la aplicación, por lo que funciona como referencia técnica para la persistencia del sistema.

NER-27: Configuración de Better Auth con email y contraseña

Evidencia de autenticación: esta captura muestra la configuración de Better Auth para habilitar el acceso mediante correo electrónico y contraseña.

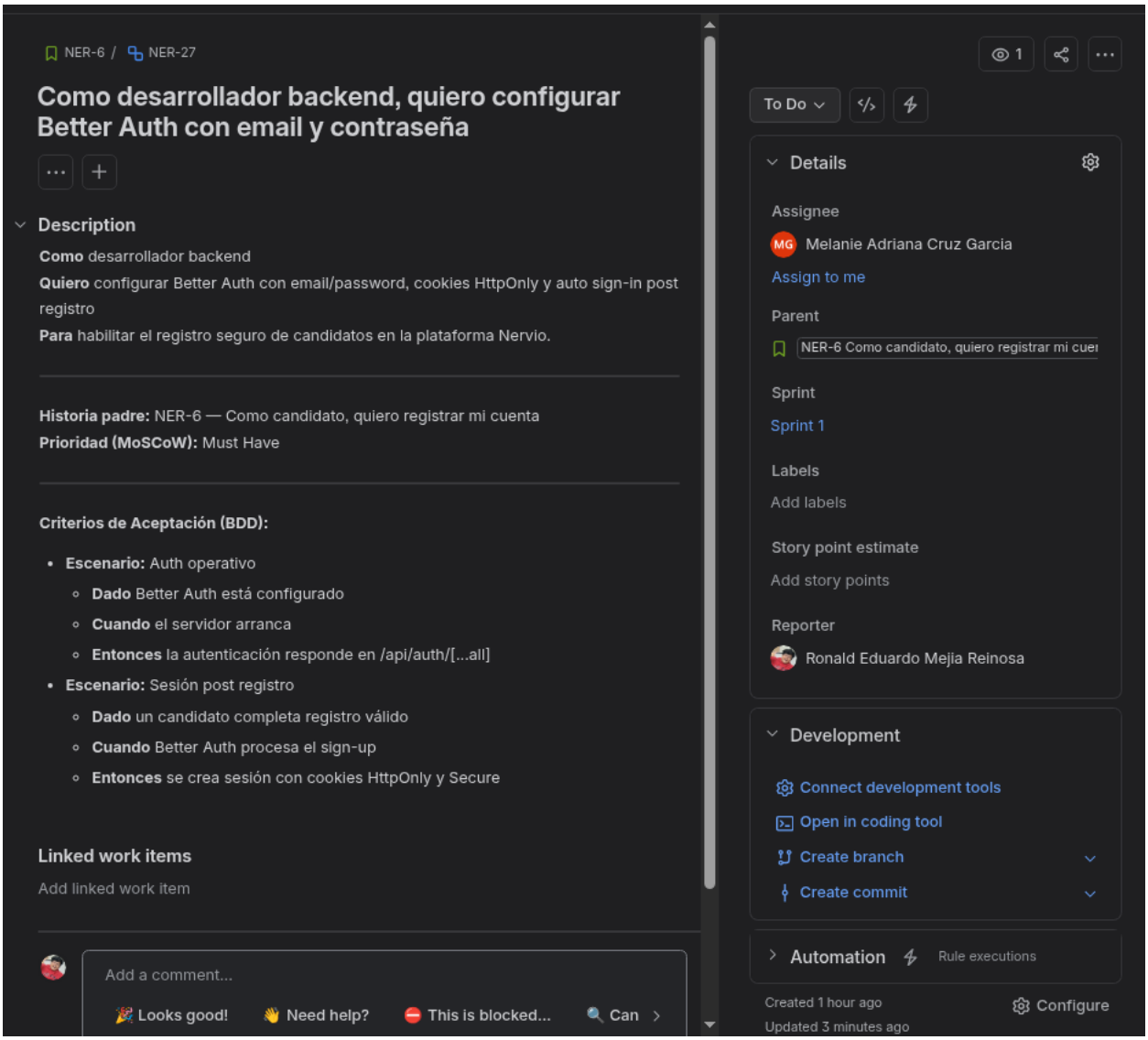


Figura 10

Configuración de Better Auth con correo electrónico y contraseña

Nota. La imagen documenta la habilitación del mecanismo de autenticación por correo electrónico y contraseña dentro de la configuración del proyecto. Better Auth indica que este método debe activarse explícitamente mediante la opción *emailAndPassword.enabled*, lo cual permite utilizar los flujos de registro e inicio de sesión basados en credenciales.

NER-28: Construcción de la página de registro con validación

Evidencia frontend: esta captura presenta la implementación visual del formulario de registro y sus validaciones principales.

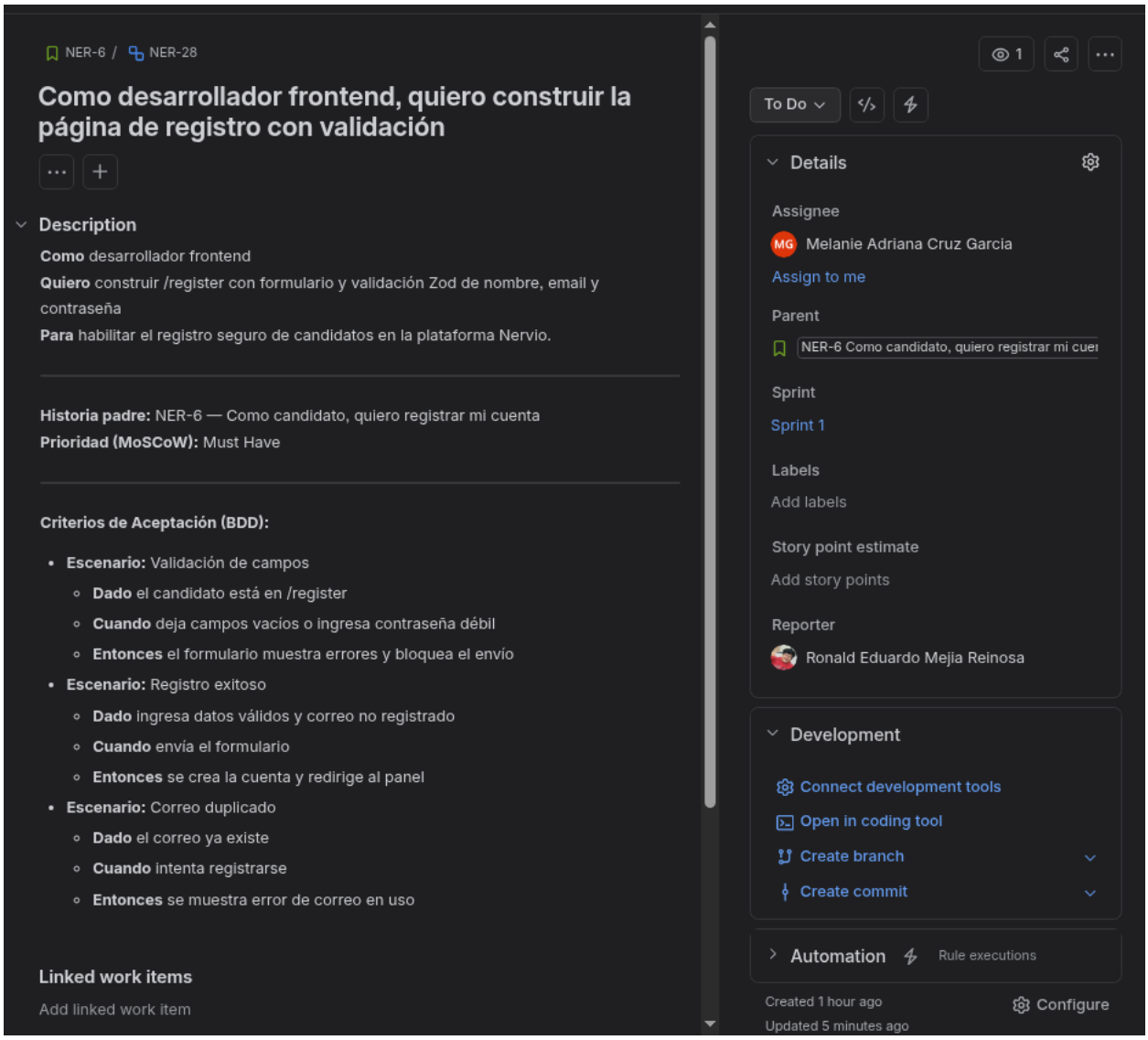


Figura 11

Formulario de registro con validación

Nota. La figura presenta la interfaz de registro implementada para capturar los datos del candidato y aplicar validaciones antes de crear la cuenta. Su propósito es evidenciar la

correspondencia entre la historia de usuario, la interfaz entregada y los criterios mínimos de calidad esperados para un flujo de alta de usuarios.

NER-29: Verificación del flujo completo de registro

Evidencia de QA: esta captura respalda la verificación del flujo completo de registro desde la perspectiva de aseguramiento de calidad.

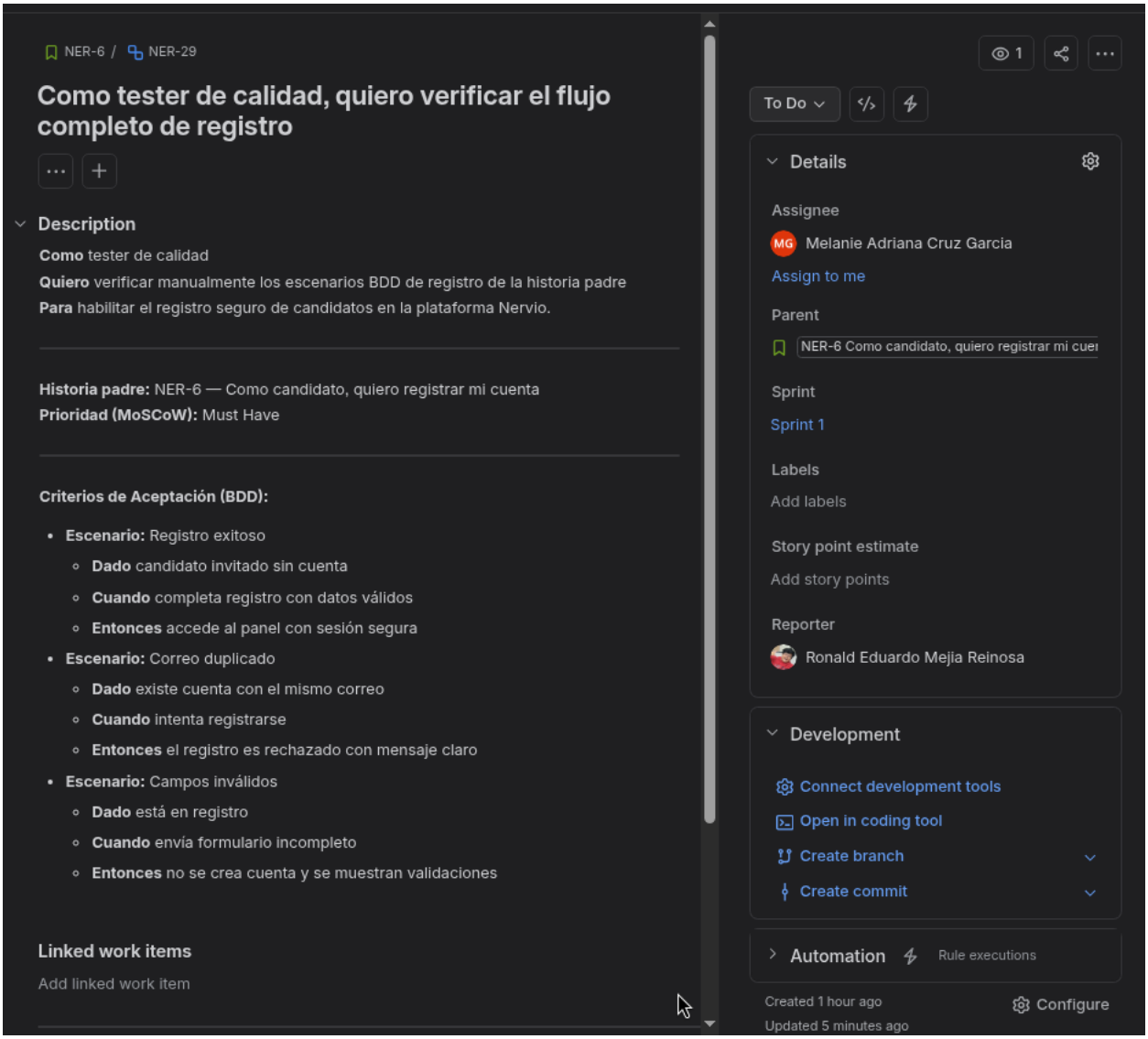


Figura 12

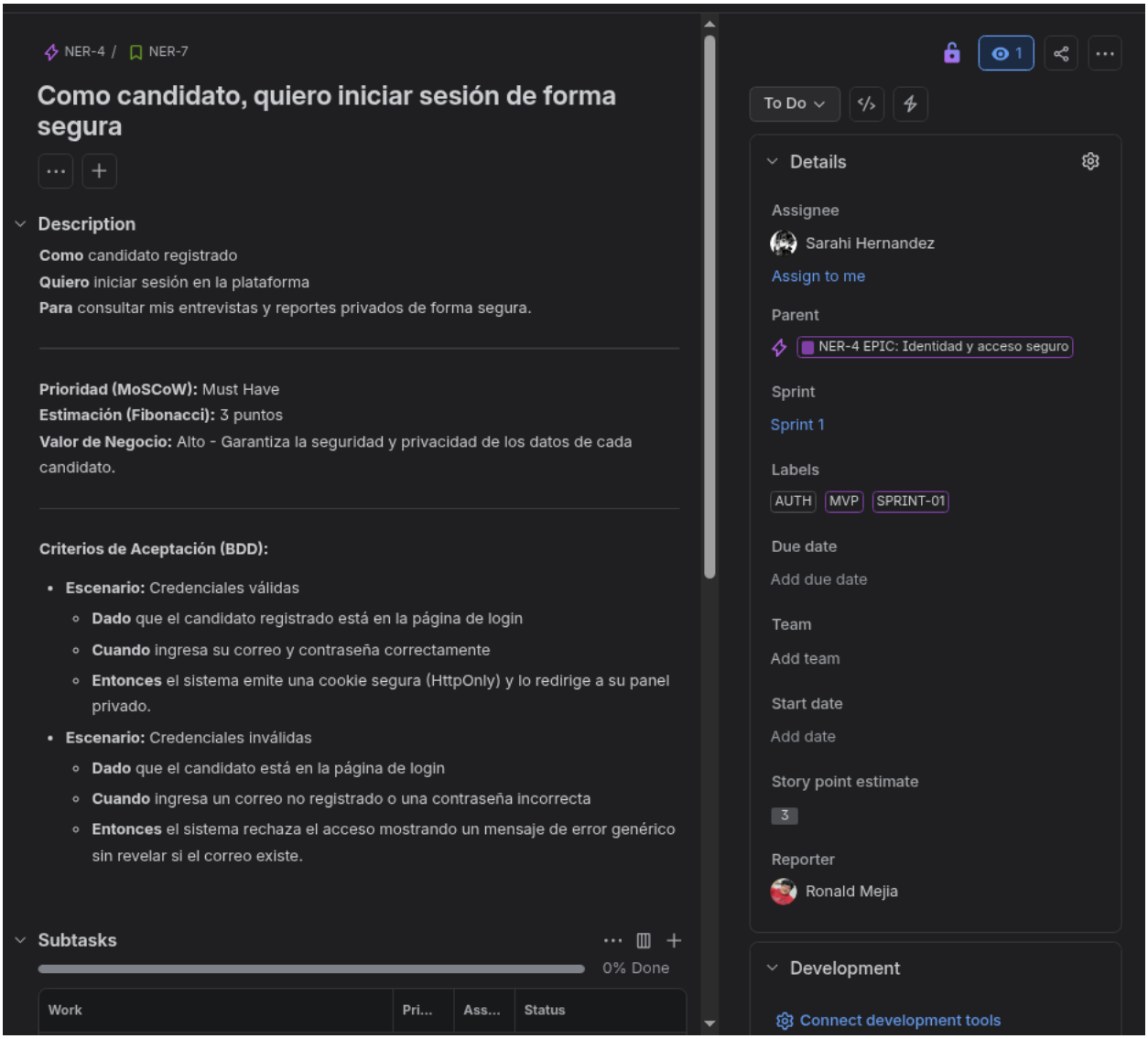
Verificación del flujo completo de registro

Nota. La figura documenta la revisión de aseguramiento de calidad aplicada al registro de cuenta, mostrando que el flujo fue validado desde la creación de usuario hasta la respuesta esperada del sistema. Esta evidencia fortalece la trazabilidad entre requisito, implementación y verificación funcional.

NER-7: Inicio de sesión seguro del candidato

Historia de usuario: Como candidato, quiero iniciar sesión de forma segura para acceder a la plataforma con mis credenciales.

Evidencia de inicio de sesión: las siguientes capturas muestran la interfaz y el flujo principal de acceso seguro del candidato.



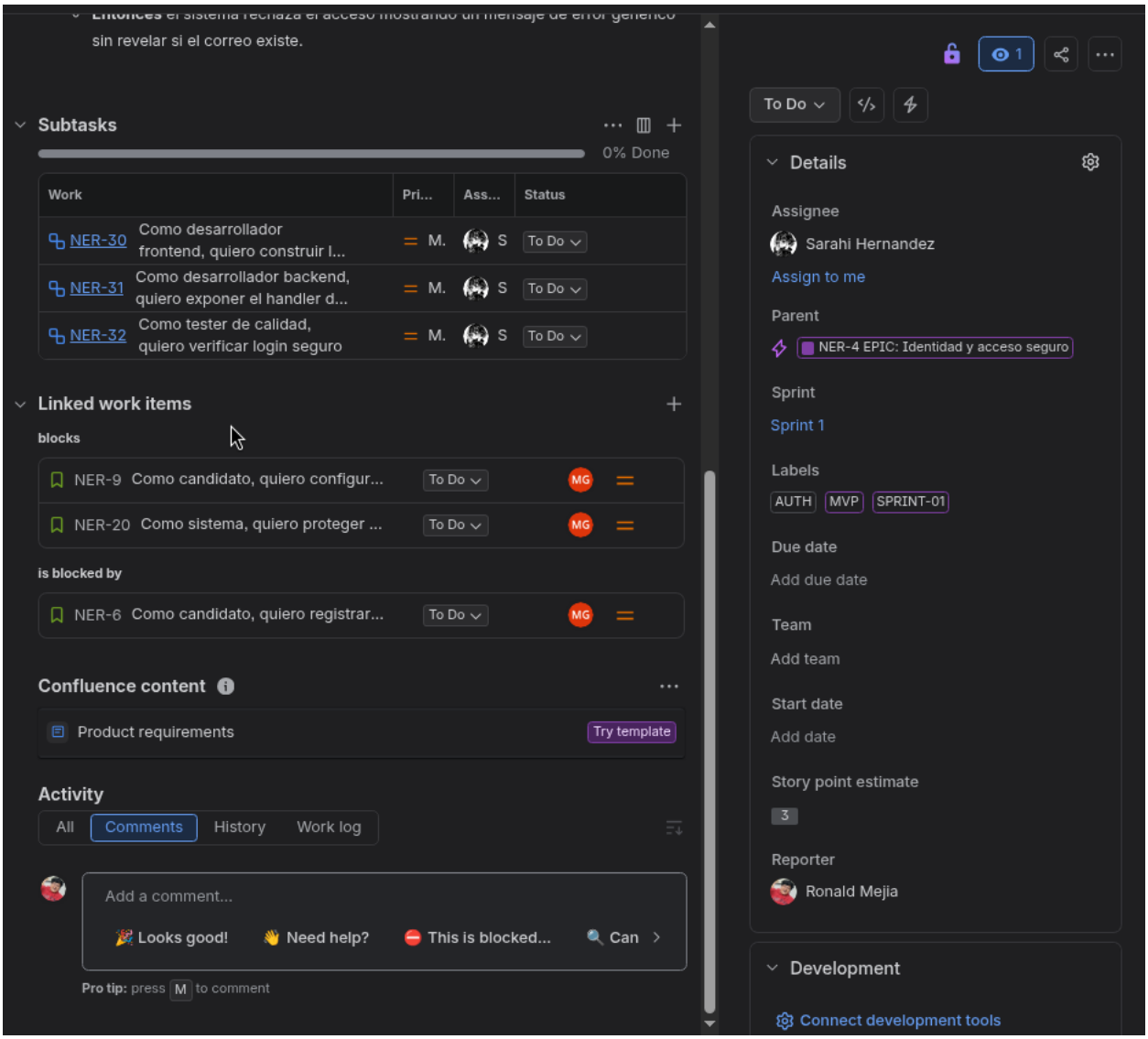


Figura 13

Inicio de sesión seguro del candidato

Nota. La figura representa el flujo mediante el cual el candidato ingresa sus credenciales para acceder a la plataforma. En términos de seguridad, la autenticación verifica la identidad del usuario mediante uno o más autenticadores, mientras que la gestión de sesiones permite mantener el estado del usuario durante su interacción con el sistema.

Subtareas asociadas

NER-30: Construcción de la página de login

Evidencia frontend: esta captura presenta la construcción de la página de login.

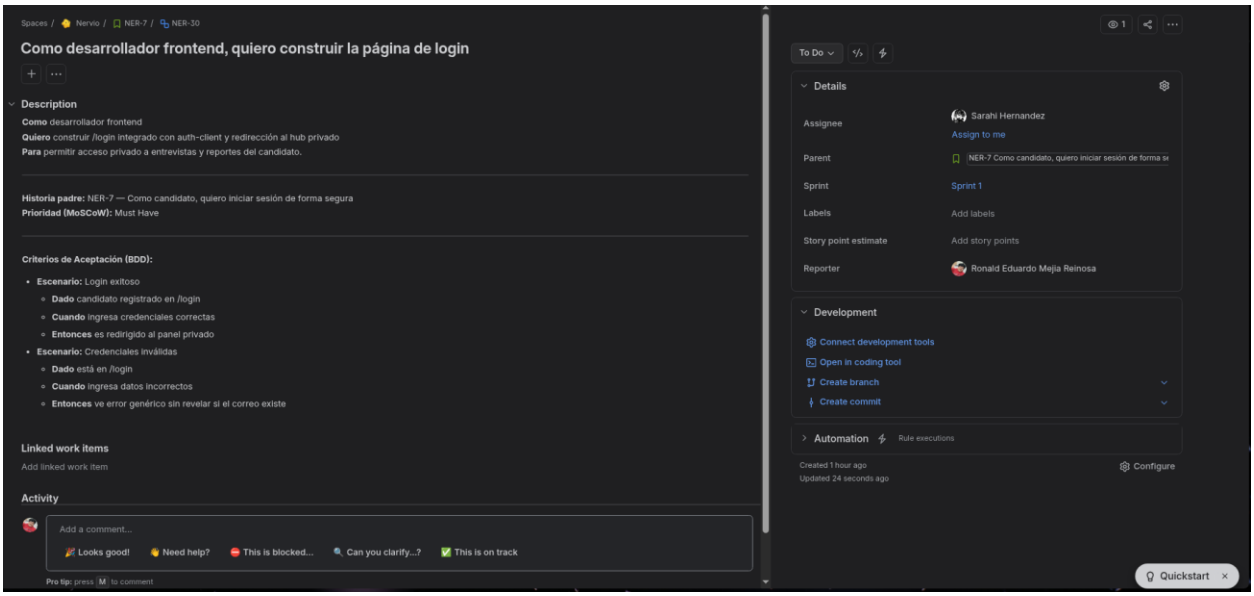


Figura 14

Construcción de la página de login

Nota. La captura muestra la implementación frontend de la pantalla de inicio de sesión, diseñada para recibir correo electrónico y contraseña como credenciales de acceso. Esta evidencia permite asociar la interfaz construida con la historia de usuario NER-7 y con el flujo general de autenticación basado en credenciales.

NER-31: Exposición del handler de autenticación

Evidencia backend: esta captura muestra la exposición del handler de autenticación.

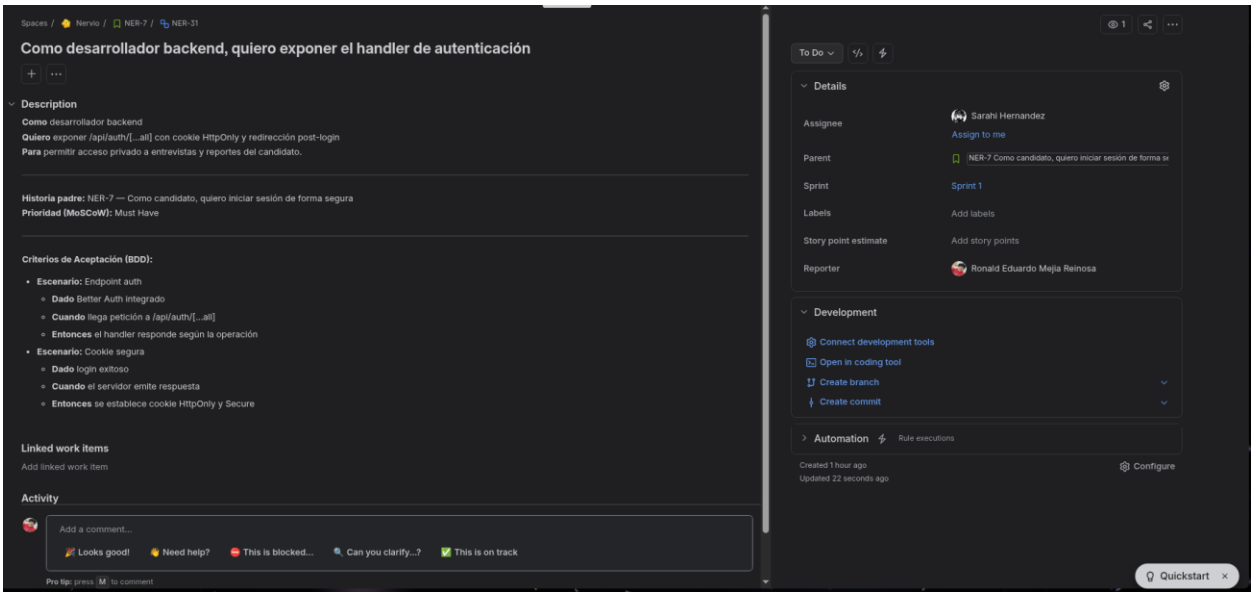


Figura 15

Exposición del handler de autenticación

Nota. La imagen evidencia la configuración backend responsable de recibir y procesar las solicitudes relacionadas con autenticación. Esta capa conecta la interfaz de usuario con los servicios encargados de validar credenciales, crear sesiones y responder al cliente, por lo que constituye un componente clave del acceso seguro.

NER-32: Verificación de login seguro

Evidencia de QA: esta captura respalda la verificación del login seguro.

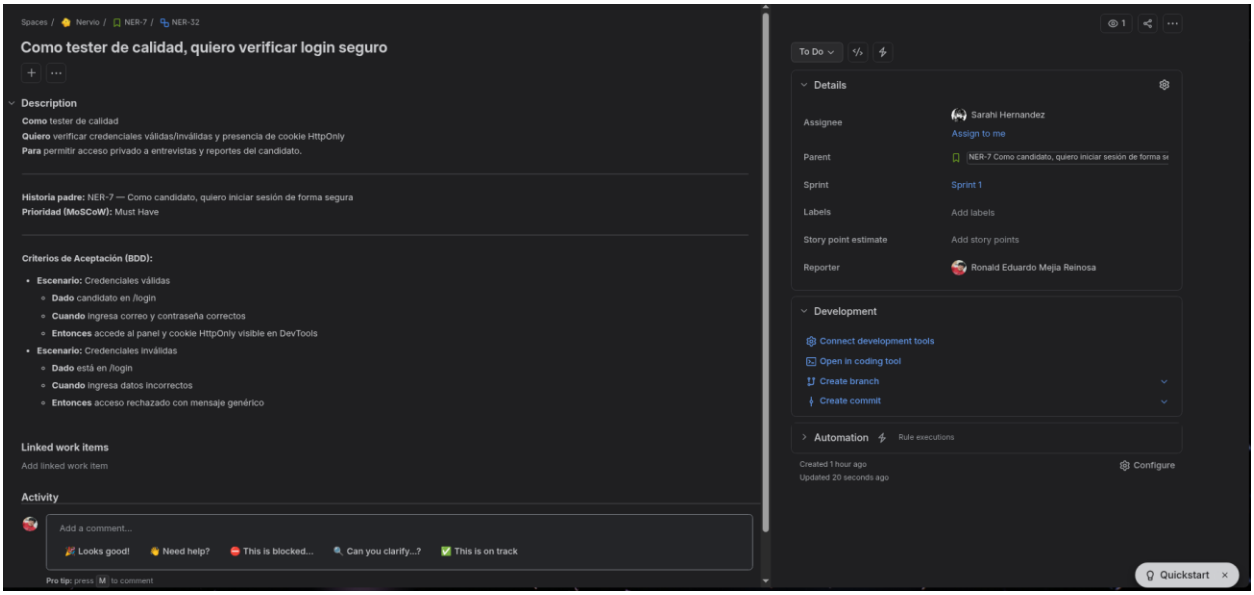


Figura 16

Verificación de login seguro

Nota. La figura presenta la evidencia de QA correspondiente al inicio de sesión seguro, confirmando que la autenticación responde de forma adecuada ante credenciales válidas y que el candidato puede acceder a la plataforma según lo definido en la historia de usuario NER-7.

NER-9: Configuración del contexto de entrevista

Historia de usuario: Como candidato, quiero configurar el contexto de mi entrevista para personalizar la experiencia antes de iniciar.

Evidencia de configuración: las siguientes capturas muestran el flujo de configuración del contexto de entrevista.

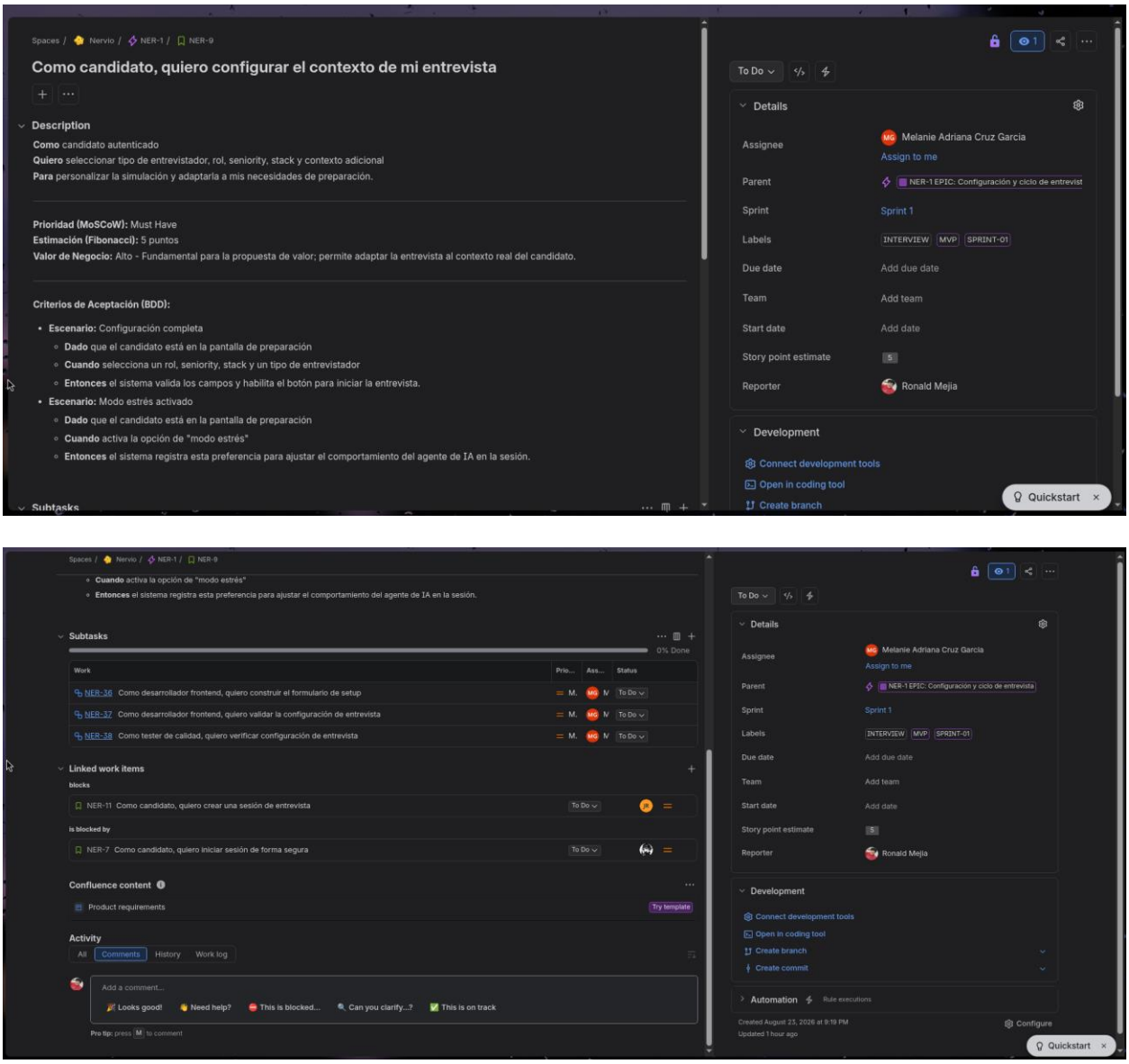


Figura 17

Configuración del contexto de entrevista

Nota. La figura muestra el flujo mediante el cual el candidato proporciona información previa para personalizar la experiencia de entrevista. Su inclusión permite explicar visualmente la transición entre el acceso a la plataforma y la preparación de una sesión individualizada, cumpliendo con el principio APA de que las figuras deben aportar información clara, necesaria y comprensible para el lector.

Subtareas asociadas

NER-36: Construcción del formulario de setup

Evidencia frontend: esta captura presenta el formulario de configuración inicial de entrevista.

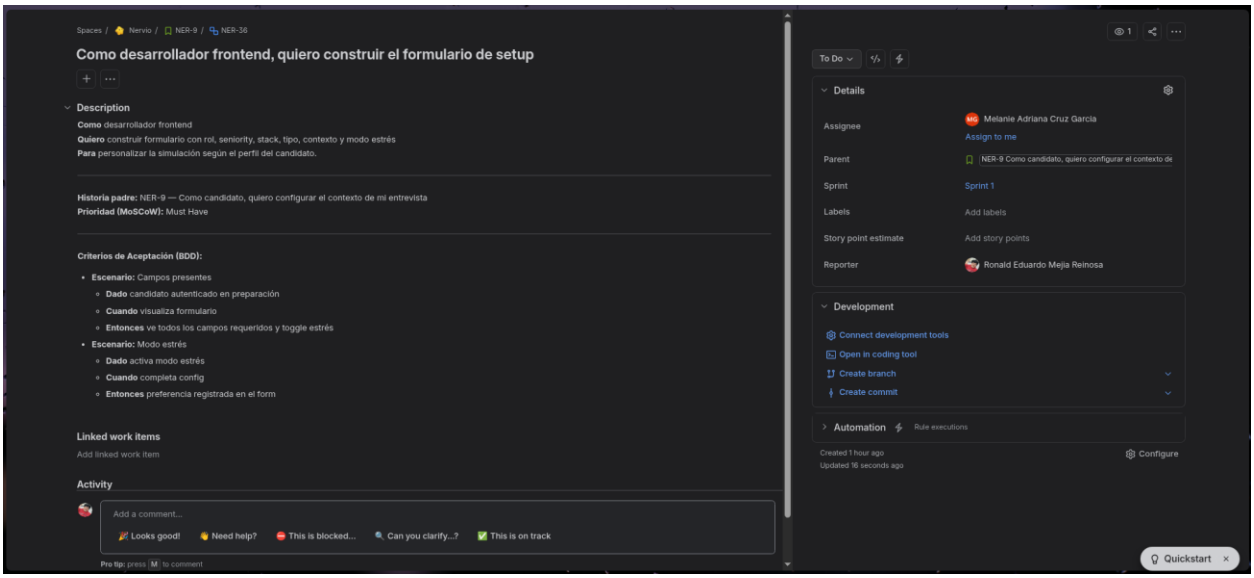


Figura 18

Formulario de configuración inicial de entrevista

Nota. La captura presenta el formulario donde el candidato registra los datos necesarios para preparar la entrevista. Esta evidencia documenta el componente frontend encargado de recopilar el contexto inicial y de habilitar la personalización del flujo antes de crear una sesión.

NER-37: Validación de la configuración de entrevista

Evidencia de validación: esta captura muestra la validación de los datos necesarios para configurar la entrevista.

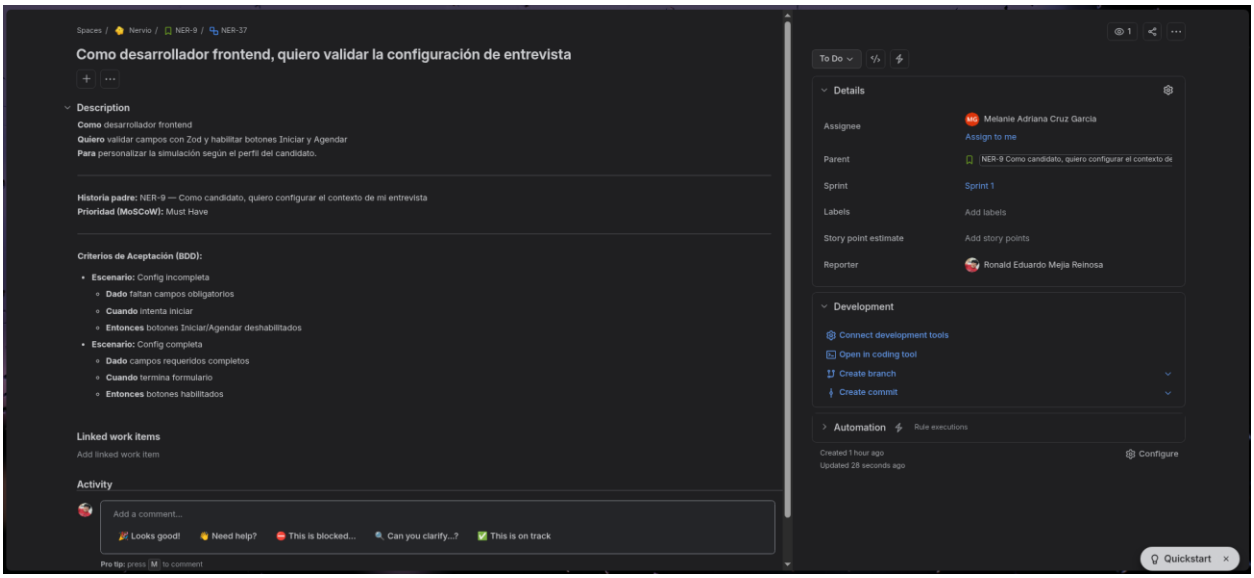


Figura 19

Validación de datos para configurar la entrevista

Nota. La figura evidencia los controles aplicados para verificar que la información introducida por el candidato sea completa y coherente antes de continuar con la entrevista. Esta validación reduce errores del flujo posterior y contribuye a que la sesión se genere con datos suficientes para su ejecución.

NER-38: Verificación de configuración de entrevista

Evidencia de QA: esta captura respalda la verificación del flujo de configuración de entrevista.

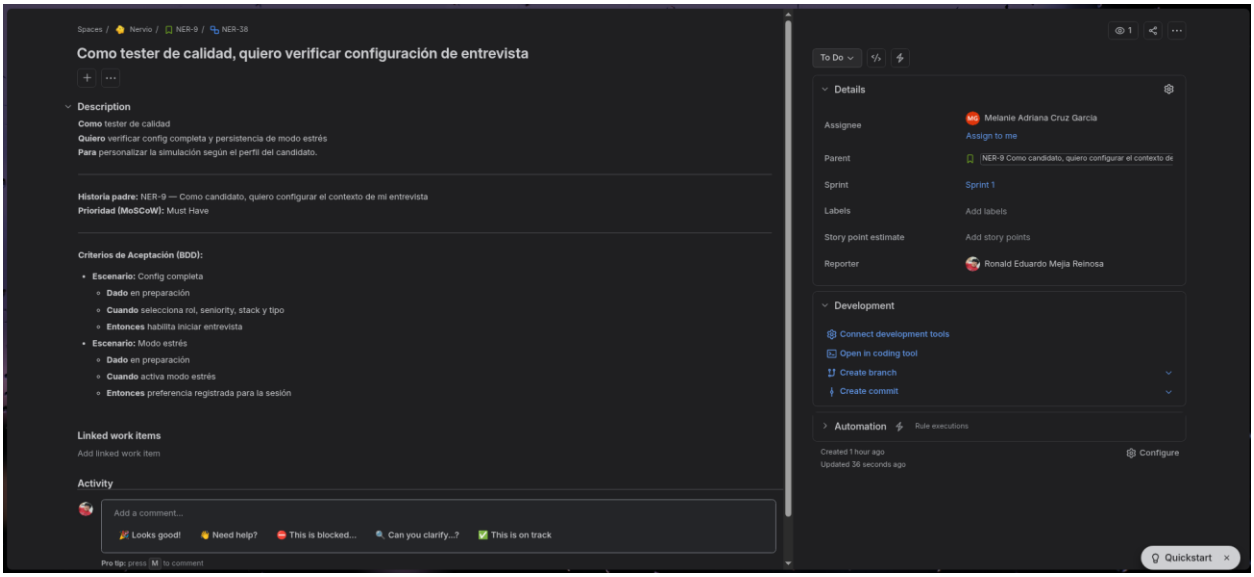


Figura 20

Verificación de configuración de entrevista

Nota. La imagen documenta la revisión de calidad del flujo de configuración de entrevista, evidenciando que la funcionalidad fue evaluada antes de considerarse parte del entregable del sprint. Esta validación permite relacionar la historia NER-9 con los criterios de aceptación definidos para la personalización de la entrevista.

NER-11: Creación de sesión de entrevista

Historia de usuario: Como candidato, quiero crear una sesión de entrevista para iniciar una experiencia de entrevista dentro de la plataforma.

Evidencia de creación de sesión: las siguientes capturas muestran el flujo principal para crear una sesión de entrevista.

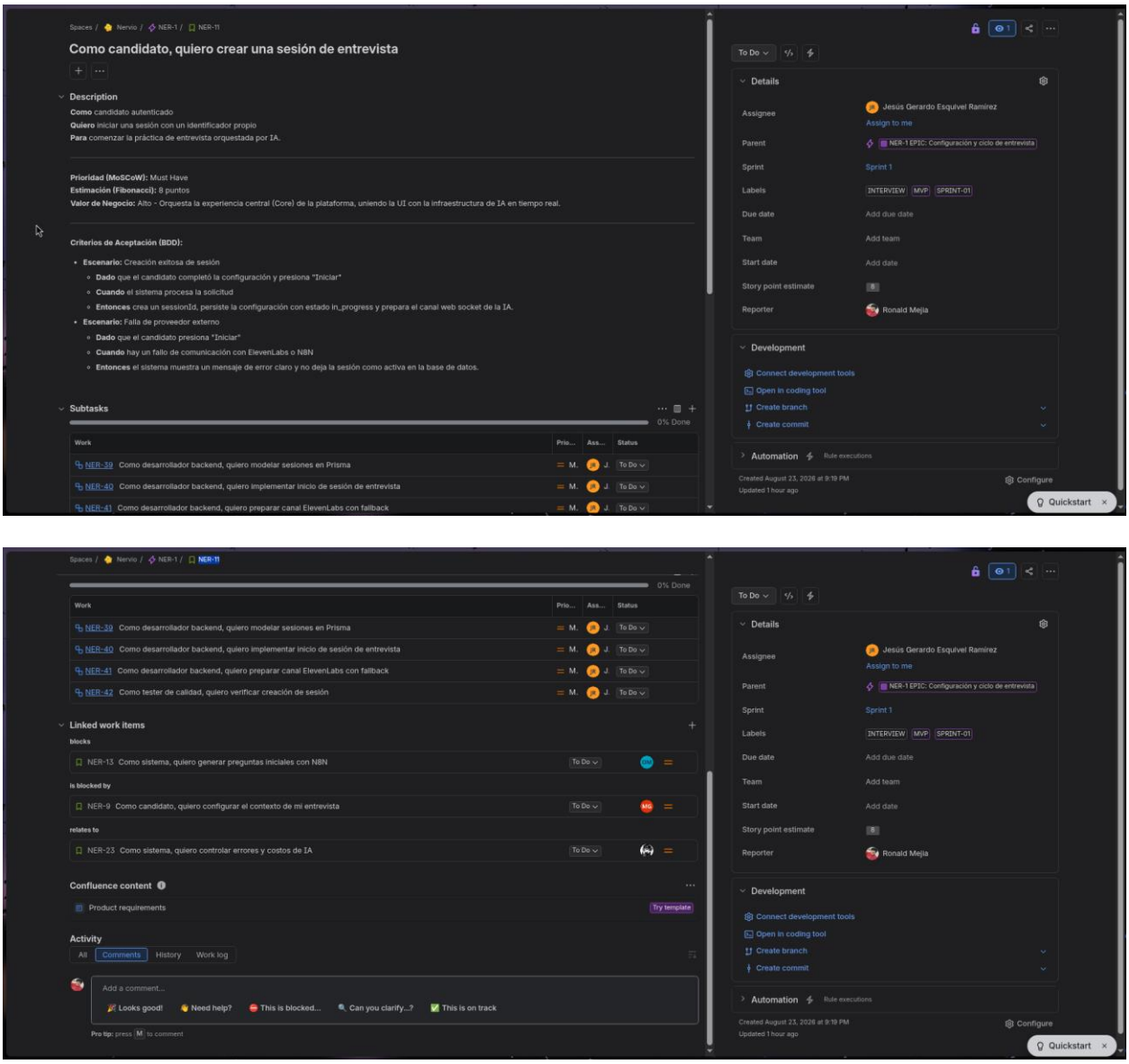


Figura 21

Creación de sesión de entrevista

Nota. La figura presenta el flujo principal para generar una sesión de entrevista a partir del contexto previamente configurado por el candidato. Esta evidencia muestra la continuidad funcional entre la preparación de la entrevista y el inicio de una experiencia persistente dentro de la plataforma.

Subtareas asociadas

NER-39: Modelado de sesiones en Prisma

Evidencia backend: esta captura documenta el modelado de sesiones en Prisma.

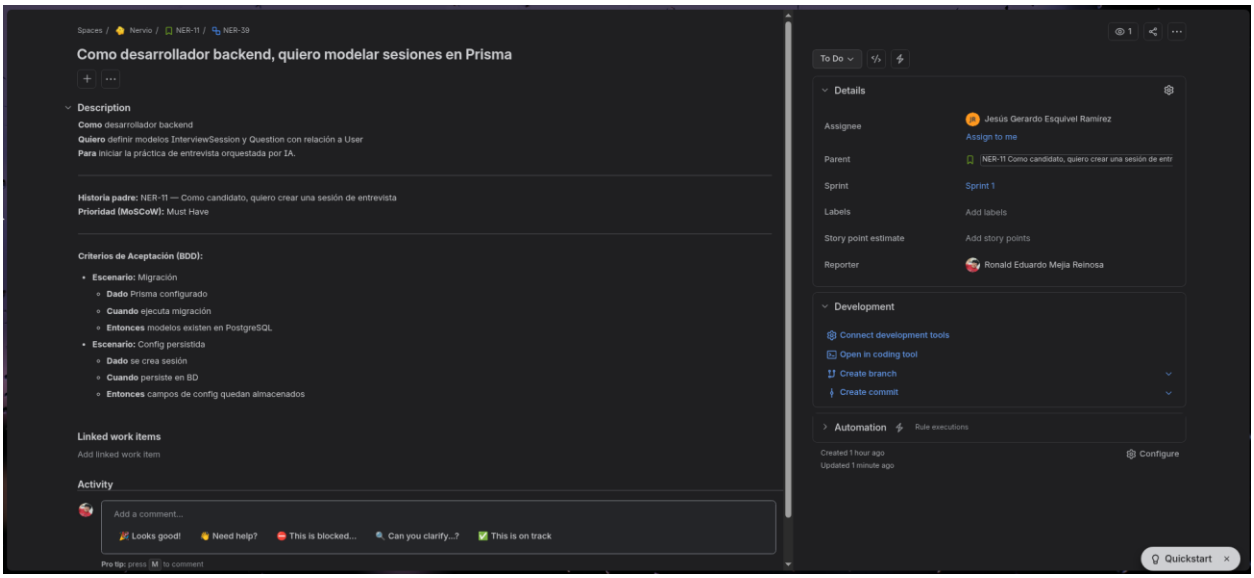


Figura 22

Modelado de sesiones en Prisma

Nota. La captura documenta el modelo de datos utilizado para representar sesiones persistentes de entrevista. Prisma establece que los modelos describen las entidades del dominio de la aplicación y se asignan a tablas o colecciones de la base de datos, lo que permite estructurar la persistencia y generar consultas tipadas mediante Prisma Client.

NER-40: Implementación del inicio de sesión de entrevista

Evidencia backend: esta captura muestra la implementación del inicio de una sesión de entrevista.

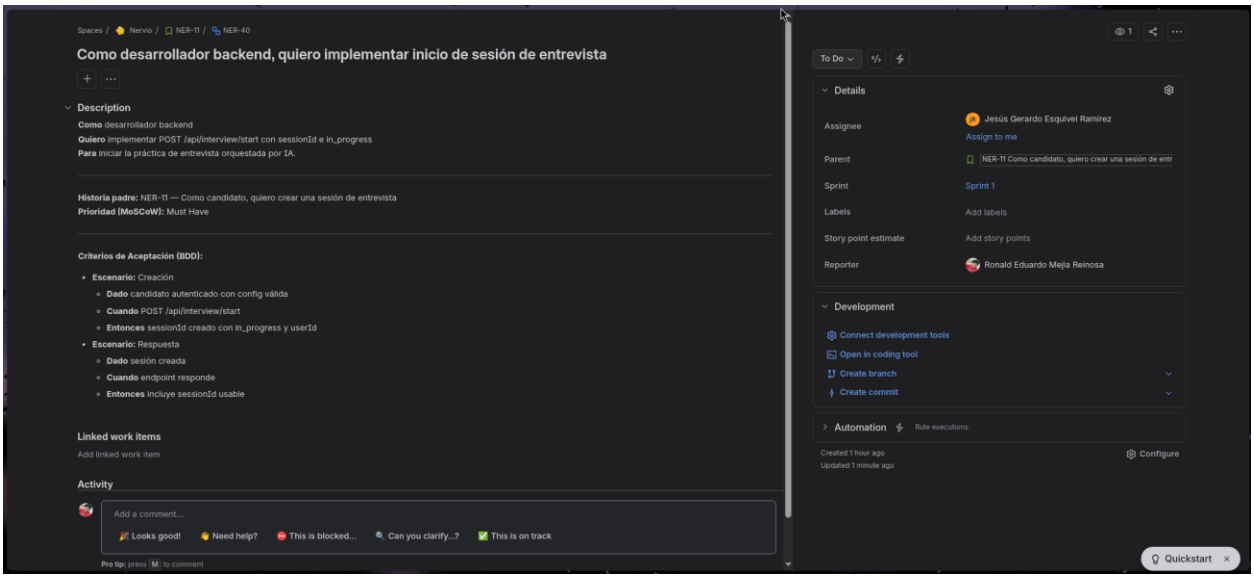


Figura 23

Implementación del inicio de una sesión de entrevista

Nota. La imagen evidencia la lógica backend que crea o inicia una sesión de entrevista a partir de la configuración previa. Esta implementación conecta los datos persistidos con el flujo funcional que experimenta el candidato, permitiendo que la entrevista se ejecute como una sesión diferenciada dentro del sistema.

NER-41: Preparación del canal ElevenLabs con fallback

Evidencia de integración: esta captura presenta la preparación del canal ElevenLabs con mecanismo de respaldo.

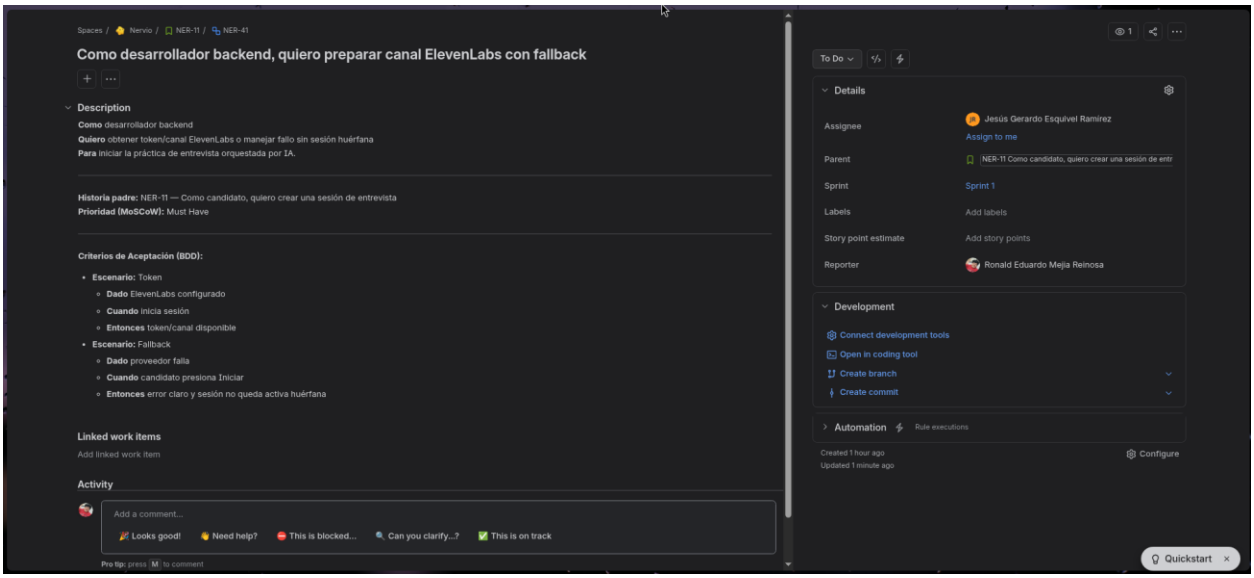


Figura 24

Preparación del canal ElevenLabs con mecanismo de respaldo

Nota. La captura presenta la preparación del canal de voz utilizado durante la entrevista, junto con un mecanismo de respaldo para reducir interrupciones ante posibles fallos de integración. Esta evidencia documenta una dependencia técnica crítica del flujo conversacional y su relación con la continuidad del servicio.

NER-42: Verificación de creación de sesión

Evidencia de QA: esta captura respalda la verificación de la creación de sesión de entrevista.

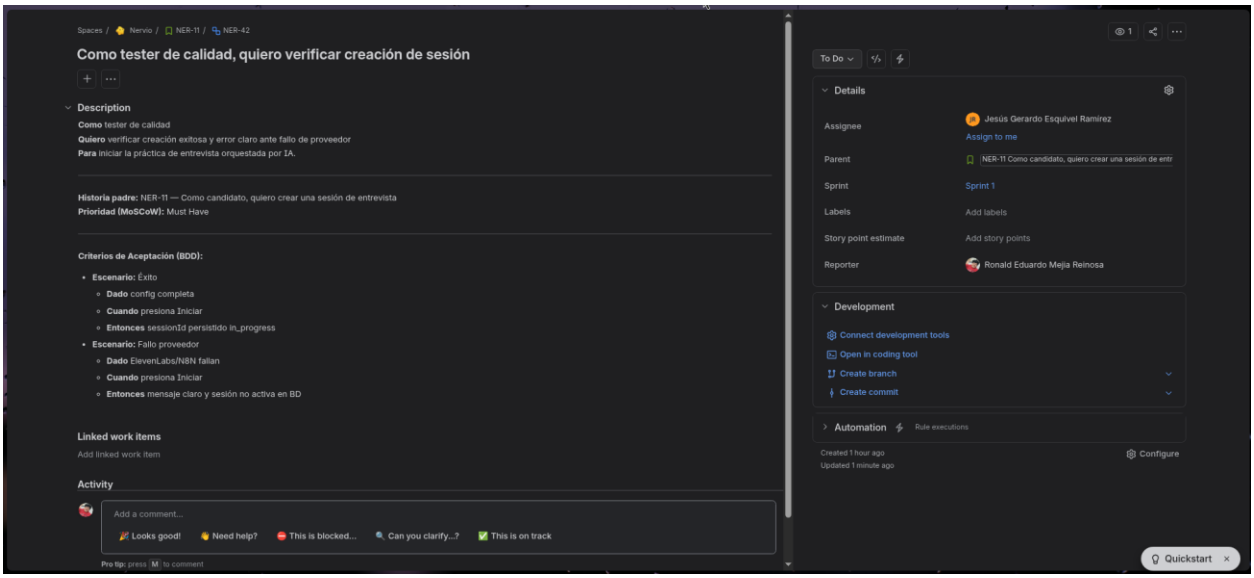


Figura 25

Verificación de creación de sesión de entrevista

Nota. La figura muestra la evidencia de aseguramiento de calidad relacionada con la creación de sesiones de entrevista. Su función es confirmar que la implementación cumple con la historia NER-11 y que el flujo opera correctamente desde la configuración previa hasta la disponibilidad de una sesión iniciada.

NER-8: Recuperación de contraseña

Historia de usuario: Como candidato, quiero recuperar mi contraseña para volver a acceder a la plataforma si olvido mis credenciales.

Evidencia de recuperación: las siguientes capturas muestran el flujo principal de recuperación de contraseña.

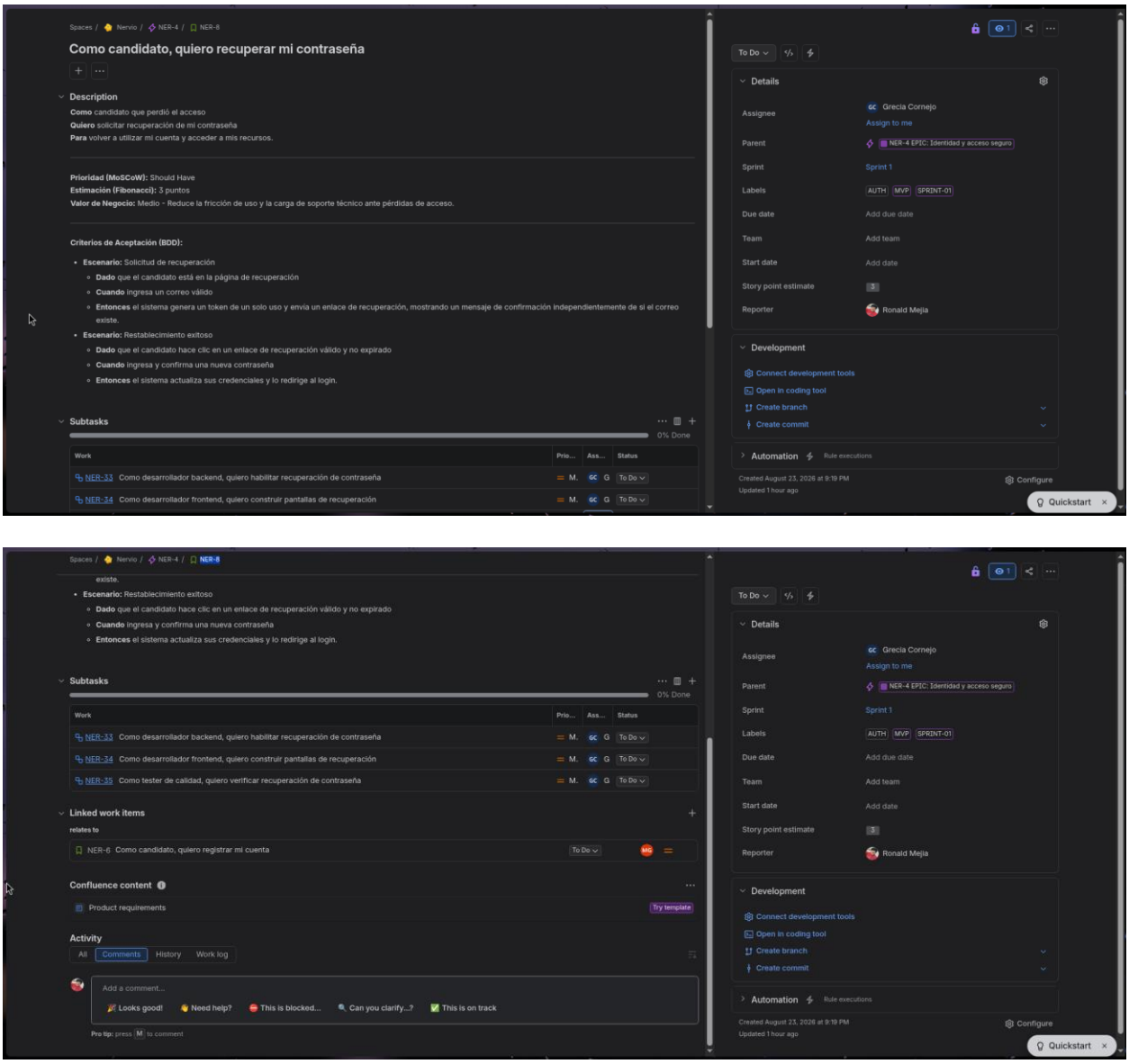


Figura 26

Flujo de recuperación de contraseña

Nota. La figura representa el proceso que permite al candidato recuperar el acceso a su cuenta cuando olvida sus credenciales. OWASP recomienda que estos flujos utilicen mensajes consistentes para cuentas existentes y no existentes, tiempos de respuesta uniformes, tokens aleatorios de un solo uso, expiración adecuada y protección contra solicitudes automatizadas.

Subtareas asociadas

NER-33: Habilitación de recuperación de contraseña

Evidencia backend: esta captura muestra la habilitación de recuperación de contraseña.

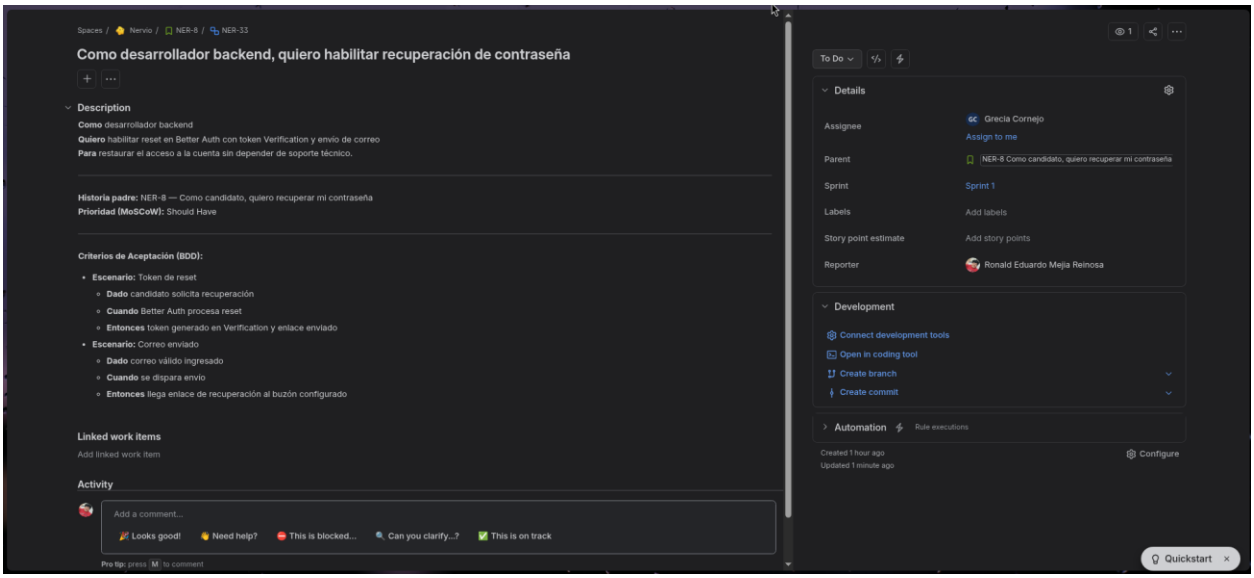


Figura 27

Habilitación de recuperación de contraseña

Nota. La captura evidencia la configuración backend que habilita el proceso de recuperación de contraseña. Esta funcionalidad debe apoyarse en mecanismos seguros para emitir, validar y expirar tokens de restablecimiento, evitando que el sistema revele la existencia de cuentas o permita solicitudes automatizadas excesivas.

NER-34: Construcción de pantallas de recuperación

Evidencia frontend: esta captura presenta las pantallas de recuperación de contraseña.

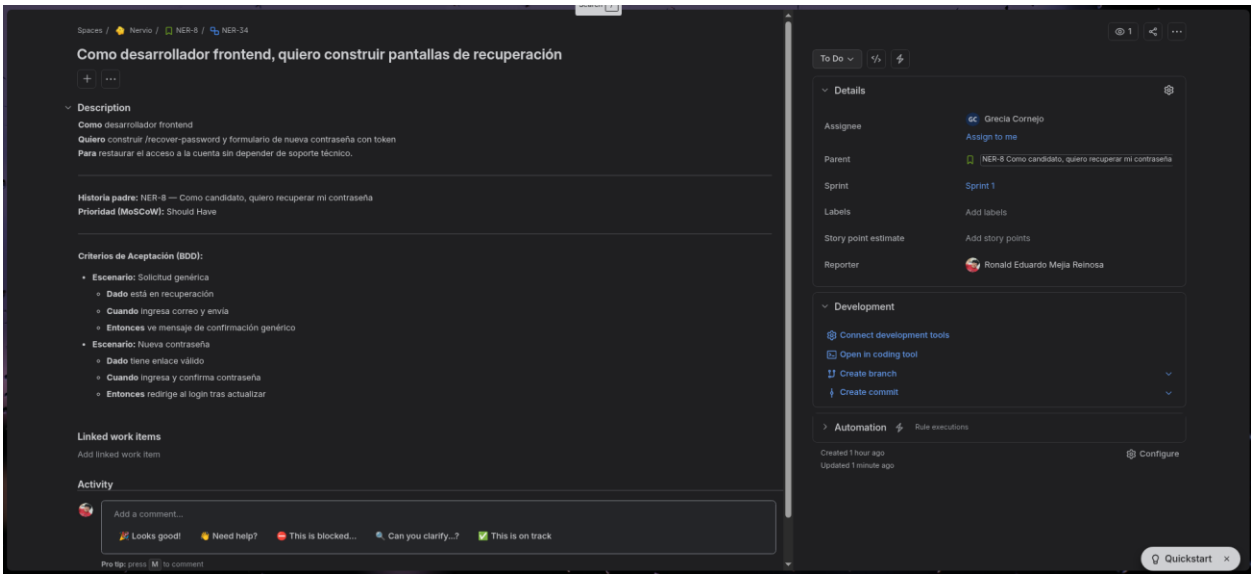


Figura 28

Pantallas de recuperación de contraseña

Nota. La figura presenta la interfaz mediante la cual el candidato solicita el restablecimiento de su contraseña. Desde una perspectiva de seguridad, esta pantalla debe ofrecer instrucciones claras sin confirmar si la cuenta existe, manteniendo respuestas uniformes para reducir riesgos de enumeración de usuarios.

NER-35: Verificación de recuperación de contraseña

Evidencia de QA: esta captura respalda la verificación del flujo de recuperación de contraseña.

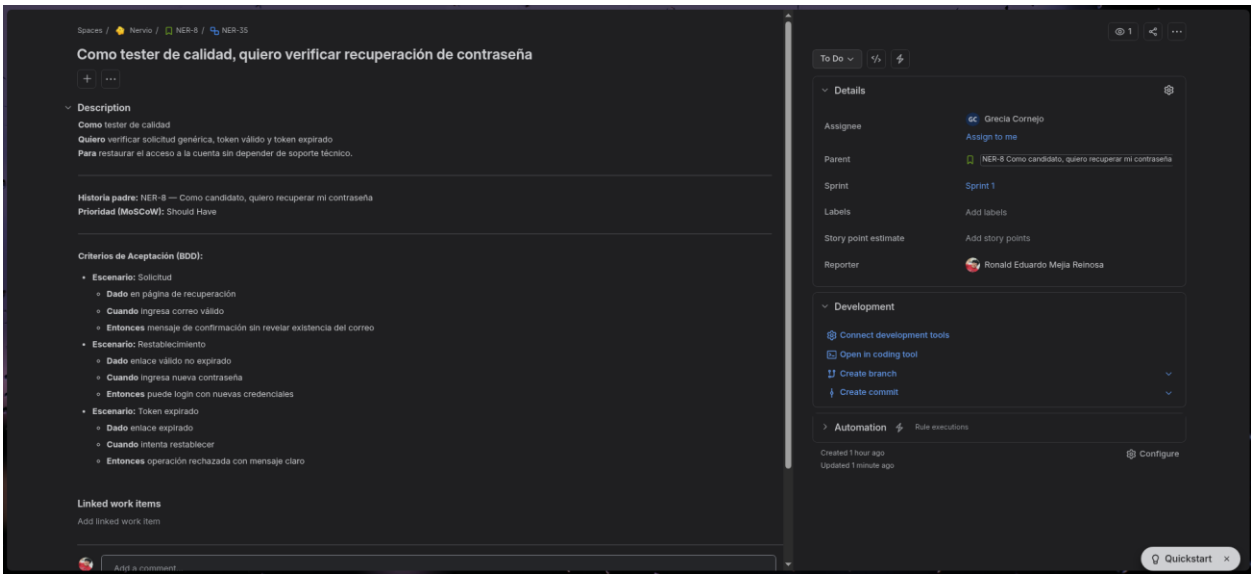


Figura 29

Verificación de recuperación de contraseña

Nota. La captura presenta la evidencia de QA utilizada para validar el flujo de recuperación de contraseña. Esta verificación confirma que la funcionalidad fue revisada frente a los criterios de aceptación de la historia NER-8 y que el proceso responde adecuadamente ante la solicitud de restablecimiento.

Nervio Board:

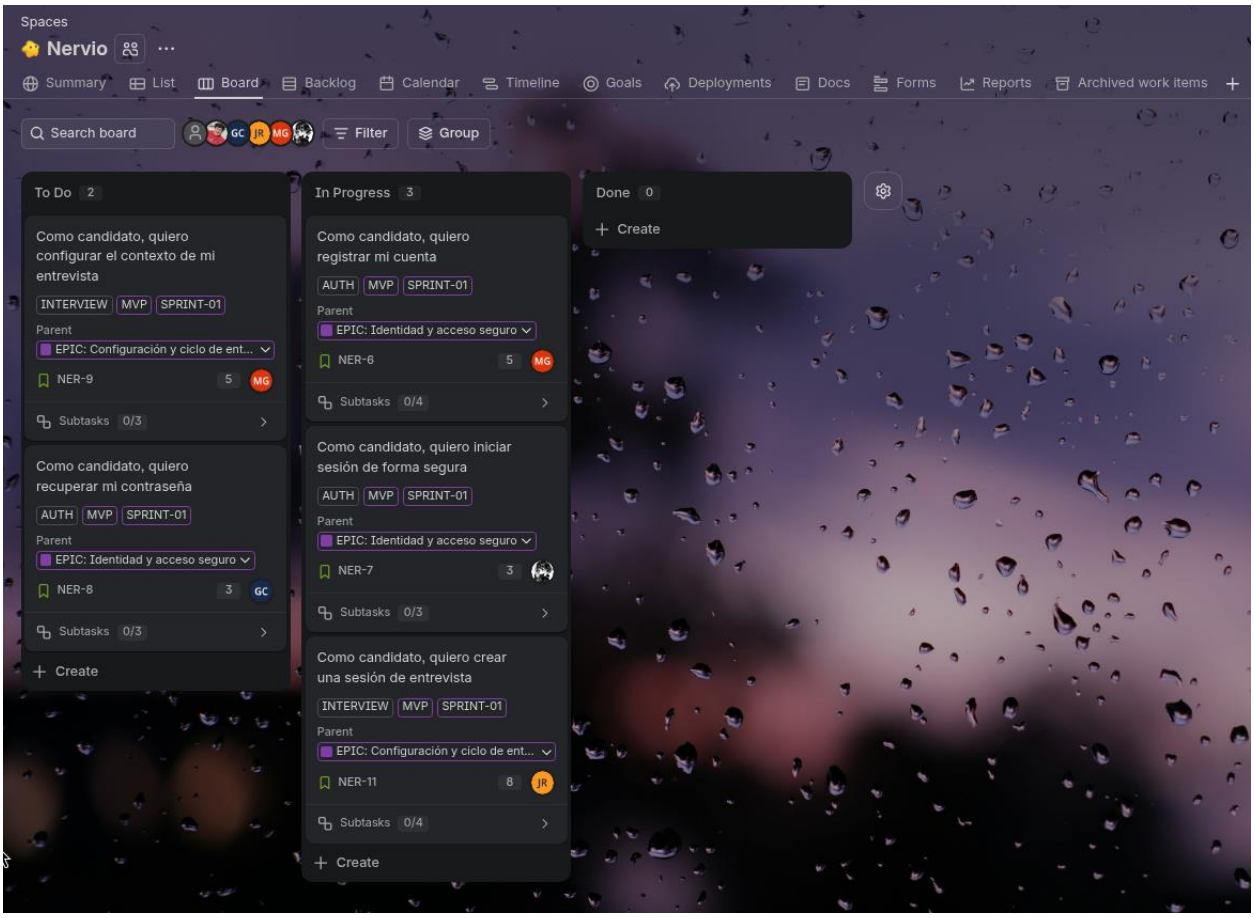


Figura 30

Tablero del proyecto Nervio en Jira

Nota. La figura muestra el tablero de trabajo utilizado para monitorear el avance del Sprint 1. En Jira, el backlog Scrum permite crear, actualizar, priorizar y asignar elementos de trabajo a sprints, mientras que el tablero facilita visualizar el progreso durante la ejecución del proyecto.

Partición del Modelo en Subsistemas y Componentes

El propósito de esta sección es transformar el modelo conceptual de análisis desarrollado anteriormente en una arquitectura técnica rigurosa, modular y verificable para la plataforma Nervio. Esta especificación describe la partición en subsistemas y componentes de software correspondientes a la implementación de Nervio, estableciendo una clara separación de responsabilidades, contratos de interfaces, políticas de seguridad y mecanismos de tolerancia a fallos.

Arquitectura del sistema

Patrón Arquitectónico Adoptado: Nervio implementa una arquitectura Cliente-Servidor en Capas con diseño de Monolito Modular Full-Stack estructurado sobre el framework Next.js (App Router, React 19 y Node.js), desacoplado de servicios especializados externos para procesamiento de voz en tiempo real (ElevenLabs) y orquestación de tareas asíncronas y evaluación analítica.

La elección de este patrón responde a las necesidades críticas del producto identificadas en el análisis previo:

- **Latencia ultrabaja en la interacción conversacional:** La simulación de entrevista requiere tiempos de respuesta inferiores a un segundo. El frontend se conecta directamente con el motor conversacional de ElevenLabs mediante WebSockets y streaming de audio bidireccional, evitando que el tráfico multimedia sature el servidor web intermedio.
- **Seguridad y custodia estricta de credenciales:** Toda operación que involucra persistencia de base de datos, sesiones de usuario, tokens efímeros y claves secretas de integración se ejecuta en el entorno del servidor de Next.js mediante Route Handlers (/api/*), impidiendo la exposición de credenciales en el cliente.

- **Separación estricta entre tiempo real y procesos analíticos pesados:** La orquestación conversacional en vivo se ejecuta de forma síncrona y fluida en la aplicación, mientras que la generación de preguntas iniciales, los recordatorios programados y las evaluaciones con IA profunda se delegan al servidor mediante procesos asíncronos y no bloqueantes.
- **Mantenibilidad y tipado de extremo a extremo:** El uso de TypeScript junto con Prisma ORM y esquemas de validación Zod garantiza consistencia de tipos entre la base de datos PostgreSQL, los controladores de API y las interfaces de usuario.

Estructura de Capas Lógicas del Sistema: El sistema se divide en cuatro capas arquitectónicas bien diferenciadas:

1. **Capa de Presentación (UI / Client Layer):** Compuesta por Server Components y Client Components de React, estilos con Tailwind CSS y componentes de interfaz basados en Shadcn UI. Es responsable de la captura de audio del micrófono, renderizado del orbe visual reactivo, línea de tiempo de la entrevista, formularios de configuración y visualización gráfica de reportes.
2. **Capa de Aplicación y Control (Application / Route Handlers Layer):** Implementada mediante las rutas de API (/api/interview/*, /api/auth/*, /api/elevenlabs/*). Actúa como controlador de negocio: valida esquemas de datos con Zod, verifica la autenticación y propiedad de la sesión, coordina la llamada a servicios y emite respuestas HTTP estructuradas.
3. **Capa de Dominio y Servicios (Domain Layer):** Centralizada en src/lib/interview/, encapsula la lógica pura de negocio: ciclo de vida de la sesión (pending, scheduled, in_progress, ended), cálculo de métricas, normalización de preguntas y respuestas, y lógica de respaldo (fallback) ante fallas externas.

4. **Capa de Persistencia e Infraestructura (Data Access & Infrastructure Layer):** Gestiona el acceso a la base de datos relacional PostgreSQL a través de Prisma Client con pooling de conexiones (@prisma/adapter-pg), control de migraciones y adaptadores de integración HTTP para webhooks de APIs de ElevenLabs.

Descomposición en Módulos y Subsistemas

El sistema se descompone en seis subsistemas funcionales autónomos y de alta cohesión. A continuación, se detallan las responsabilidades técnicas, componentes de software e interfaces de cada uno:

Subsistema de Presentación y Experiencia de Usuario (UI/UX)

Encargado de la interacción directa con el candidato. Aprovecha las capacidades híbridas de Next.js combinando Server Components (para obtención segura y pre-renderizada de datos) y Client Components con la directiva "use client" (para gestión de eventos, streams de audio y animaciones reactivas).

- **Páginas y Vistas Principales:** src/app/page.tsx (Landing y resumen de características), src/app/login/page.tsx y register/page.tsx (Acceso y registro), src/app/interview/setup/page.tsx (Configuración previa de entrevista), src/app/interview/[sessionId]/page.tsx (Entorno inmersivo de entrevista en vivo) y src/app/interview/reports/page.tsx (Historial y reportes de desempeño).
- **Componentes Especializados de Entrevista:** InterviewLiveClient (orquestador del ciclo de vida en el cliente), InterviewLiveShell (estructura visual de cabina de entrevista), InterviewOrb (esfera 3D reactiva al tono y volumen del entrevistador), InterviewControls (control de micrófono y finalización manual) e InterviewTimeline (seguimiento cronológico de turnos).
- **Custom Hook useInterviewConversation:** Encapsula la máquina de estados en el cliente (fases: conectando, hablando, escuchando, finalizando), sincroniza transcripciones en tiempo real y detecta problemas de calidad de enlace.

Subsistema de Autenticación y Control de Acceso

Garantiza la identidad del candidato, custodia sus credenciales y restringe el acceso a las entrevistas y reportes únicamente a su propietario legítimo.

- **Better Auth con Adaptador Prisma:** Implementado en src/lib/auth.ts, gestiona hashing seguro de contraseñas, emisión de sesiones basadas en tokens criptográficos y persistencia en las tablas User, Session, Account y Verification.
- **Protección de Rutas (requireAuth):** Función servidora en src/lib/auth-server.ts que valida la sesión HTTP y redirige automáticamente al flujo de login ante accesos no autenticados, conservando la URL de retorno.
- **Endpoint Unificado de Autenticación:** src/app/api/auth/[...all]/route.ts expone los controladores nativos de Better Auth para registro, login, logout y validación de sesión.

Subsistema de Gestión del Ciclo de Vida de Entrevistas

Controla la máquina de estados de las simulaciones y coordina la transición entre estados: configuración inicial, sesión programada o activa, registro de turnos y cierre de evaluación.

- **InterviewSetupForm y Validaciones:** Valida entradas mediante esquemas Zod (interviewSetupSchema): rol laboral, seniority (junior, mid, senior), stack tecnológico, contexto adicional, idioma y activación de "Modo Estrés".
- **Orquestador de Inicio (POST /api/interview/start):** Crea el registro persistente InterviewSession, invoca a un procedimiento en el servidor de Eleven Labs para generar las preguntas contextuales y sintetiza el primer audio.
- **Controlador de Mensajes (POST /api/interview/[sessionId]/messages):** Registra en la base de datos cada turno conversacional, vinculando las intervenciones del entrevistador (questions) y las respuestas del candidato (responses).
- **Controlador de Cierre (POST /api/interview/[sessionId]/end):** Finaliza la sesión con marca temporal ended_at, vincula el conversationId de ElevenLabs y ejecuta la persistencia de las métricas de evaluación.

Subsistema de Conversación y Voz en Tiempo Real

Proporciona la experiencia conversacional por voz. Gestiona la conexión bidireccional de baja latencia con los modelos de habla e inteligencia conversacional.

- **SDK @elevenlabs/react y ConversationProvider:** Establece canal WebSocket directo con el agente conversacional, transmitiendo audio PCM del micrófono y recibiendo audio en streaming con baja latencia.

- **Emisión de Credenciales Efímeras (POST /api/elevenlabs/token):** Genera tokens temporales firmados para que el cliente se autentique ante ElevenLabs sin exponer la clave privada del servidor.
- **Servicio de Síntesis de Voz (src/lib/interview/elevenlabs.ts):** Provee síntesis de texto a voz (TTS) para preguntas introductorias y mensajes del sistema mediante el modelo eleven_multilingual_v2.

Subsistema de Persistencia y Acceso a Datos

Administra el almacenamiento relacional, la integridad referencial y las consultas tipadas hacia la base de datos PostgreSQL.

- **Prisma ORM y Conexión Centralizada:** Definido en prisma/schema.prisma y gestionado en src/lib/prisma.ts mediante el adaptador @prisma/adapter-pg, implementando el patrón Singleton para reutilizar conexiones activas.
- **Entidades del Dominio de Entrevista:** InterviewSession (sesión principal), questions (preguntas base y follow-ups con jerarquía parentQuestion), responses (respuestas transcritas, banderas de pausas largas y muletillas), evaluations (scores cuantitativos de claridad, conocimiento, confianza y estructura), recordings (audios de soporte), schedules (reservas futuras) y webhook_logs (auditoría de integraciones).

Subsistema de Automatización, Evaluaciones y Procesos Asíncronos

Coordina los flujos diferidos y la persistencia de métricas profundas generadas por modelos de lenguaje analítico.

- **Módulo de Análisis de Evaluación (src/lib/interview/elevenlabs-analysis.ts):** Extrae y normaliza el reporte final estructurado desde la conversación analizada, almacenando fortalezas, debilidades, recomendación final y puntaje global en la base de datos.
- **Tabla de Trazabilidad webhook_logs:** Almacena cada envío y respuesta hacia la base de datos con estado (sent, error), código HTTP y cuerpo de error para auditoría y reintentos.

Tabla 1 Descomposición técnica de subsistemas y componentes de la plataforma Nervio.

Subsistema	**Componente Principal**	**Capa / Ubicación**	**Responsabilidad Técnica Principal**
Presentación	InterviewLiveClient & Shell	Cliente (React/Next.js)	Control de cabina interactiva, visualización de orbe y audio.

Subsistema	Componente Principal	Capa / Ubicación	Responsabilidad Técnica Principal
Presentación	InterviewSetupForm	Cliente (React/Zod)	Captura y validación de parámetros de entrevista y modo estrés.
Presentación	InterviewReportsPage	Servidor (RSC)	Renderizado de métricas cuantitativas e historial de sesiones.
Autenticación	Better Auth Handler	Servidor (/api/auth)	Gestión de credenciales, tokens de sesión y cookies seguras.
Autenticación	requireAuth Guard	Servidor (lib/auth-server)	Protección de rutas de navegación y redirección asistida.
Ciclo de Vida	Route Handler /start	Servidor (/api/interview)	Creación de sesión, invocación de interview y primer turno.
Ciclo de Vida	Route Handler /messages	Servidor (/api/interview)	Persistencia ordenada de preguntas y respuestas del candidato.
Ciclo de Vida	Route Handler /end	Servidor (/api/interview)	Cierre de sesión, cálculo de tiempos y solicitud de evaluación.
Conversación/Voz	ElevenLabs ConvAI Client	Cliente / WebSocket	Streaming bidireccional de voz, detección de habla y tono.
Persistencia	Prisma Client Singleton	Servidor (lib/prisma)	Acceso tipado a PostgreSQL, transacciones y pool de conexiones.

Nota. *Elaboración propia basada en la arquitectura implementada en el repositorio de Nervio.*

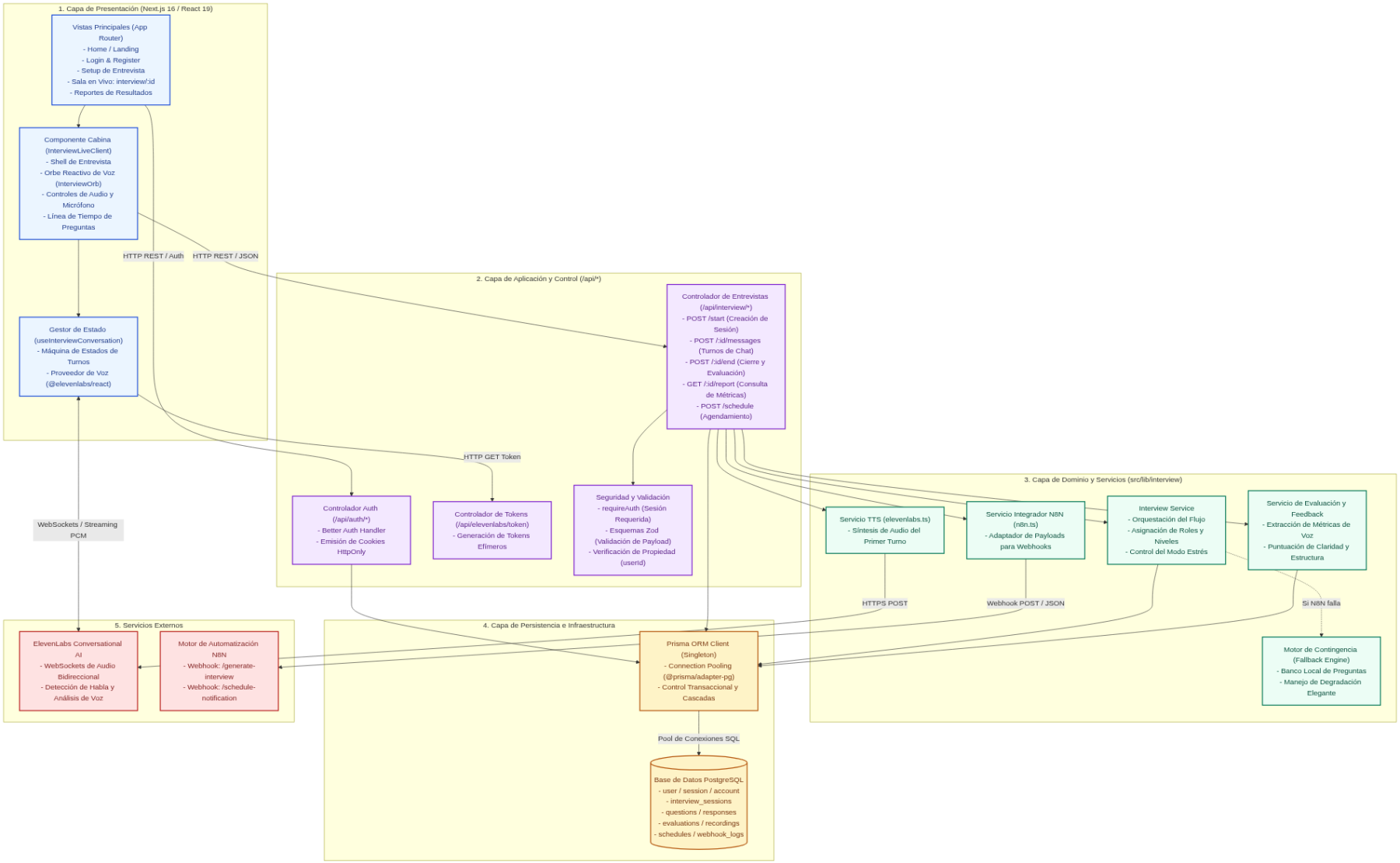
Tabla 2 *Catálogo de interfaces y endpoints de la capa de aplicación (Route Handlers).*

Método	Ruta / Endpoint	Autenticación	Cuerpo / Parámetros	Respuesta Principal
POST	/api/auth/[...all]	Pública / Sesión	Credenciales / Token	Cookie de sesión

Método	Ruta / Endpoint	Autenticación	Cuerpo / Parámetros	Respuesta Principal
				HttpOnly y perfil de usuario.
POST	/api/interview/start	Requerida (Cookie)	JSON: rol, nivel, stack, estrés	JSON: sessionId, questions[], primer audioUrl.
POST	/api/interview/[id]/messages	Requerida (Cookie)	JSON: role, text	JSON: estado de turno { isComplete, phase }.
POST	/api/interview/[id]/end	Requerida (Cookie)	JSON: conversationId, reason	JSON: estado { ended, reportReady }.
GET	/api/interview/[id]/report	Requerida (Cookie)	Parámetro de ruta: id	JSON: scores, fortalezas, debilidades.
POST	/api/interview/schedule	Requerida (Cookie)	JSON: fecha, hora, contacto	JSON: scheduleId, confirmación de cita.
GET	/api/elevenlabs/token	Requerida (Cookie)	Encabezados de sesión	JSON: token efímero firmado para WebSocket.

Nota. *Todos los endpoints de entrevista validan que la sesión pertenezca al usuario autenticado*

Diagrama de Componentes del Sistema: A continuación, se define la estructura lógica en capas del sistema. Conforme a las directrices de la actividad, este esquema se complementará con su respectivo diagrama modelado en Mermaid dentro del documento final.


1. Capa de Presentación (Next.js 16 / React 19)
Vistas Principales (App Router)
- Home / Landing
- Login & Register
- Setup de Entrevista
- Sala en Vivo: interview/:id
- Reportes de Resultados
Componente Cabina (InterviewLiveClient)
- Shell de Entrevista
- Orbe Reactivo de Voz (InterviewOrb)
- Controles de Audio y Micrófono
- Línea de Tiempo de Preguntas
Gestor de Estado (useInterviewConversation)
- Máquina de Estados de Turnos
- Proveedor de Voz (@elevenlabs/react)
HTTP REST / Auth
HTTP REST / JSON
2. Capa de Aplicación y Control (/api/*)
Controlador de Entrevistas (/api/interview/*)
- POST /start (Creación de Sesión)
- POST /:id/messages (Turnos de Chat)
- POST /:id/end (Cierre y Evaluación)
- GET /:id/report (Consulta de Métricas)
- POST /schedule (Agendamiento)
HTTP GET Token
Controlador Auth (/api/auth/*)
- Better Auth Handler
- Emisión de Cookies HttpOnly
Controlador de Tokens (/api/elevenlabs/token)
- Generación de Tokens Efímeros
Seguridad y Validación
- requireAuth (Sesión Requerida)
- Esquemas Zod (Validación de Payload)
- Verificación de Propiedad (userid)
3. Capa de Dominio y Servicios (src/lib/interview)
Servicio TTS (elevenlabs.ts)
- Síntesis de Audio del Primer Turno
Servicio Integrador N8N (n8n.ts)
- Adaptador de Payloads para Webhooks
Interview Service
- Orquestación del Flujo
- Asignación de Roles y Niveles
- Control del Modo Estrés
Servicio de Evaluación y Feedback
- Extracción de Métricas de Voz
- Puntuación de Claridad y Estructura
HTTPS POST
Webhook POST / JSON
Si N8N falla
WebSockets / Streaming PCM
5. Servicios Externos
ElevenLabs Conversational AI
- WebSockets de Audio Bidireccional
- Detección de Habla y Análisis de Voz
Motor de Automatización N8N
- Webhook: /generate-interview
- Webhook: /schedule-notification
4. Capa de Persistencia e Infraestructura
Prisma ORM Client (Singleton)
- Connection Pooling (@prisma/adapter-pg)
- Control Transaccional y Cascadas
Motor de Contingencia (Fallback Engine)
- Banco Local de Preguntas
- Manejo de Degradación Elegante
Pool de Conexiones SQL
Base de Datos PostgreSQL
- user / session / account
- interview_sessions
- questions / responses
- evaluations / recordings
- schedules / webhook_logs

Mecanismos de Control y Recursos Globales

Para garantizar la robustez, seguridad y operatividad ininterrumpida de Nervio en entornos de producción, se han especificado formalmente los siguientes mecanismos transversales de control:

Gestión de Seguridad (Autenticación y Autorización)

La seguridad del sistema se rige bajo el principio de defensa en profundidad y privilegio mínimo:

- **Almacenamiento Seguro de Credenciales:** Better Auth aplica algoritmos criptográficos robustos (scrypt/bcrypt con salt aleatorio) para la custodia de contraseñas, impidiendo el almacenamiento de texto plano en la base de datos.
- **Gestión de Sesiones e Identidad:** Las credenciales de acceso se administran mediante cookies protegidas con directivas HttpOnly (inaccesibles para JavaScript del navegador, previniendo ataques XSS), Secure (transmisión forzada por HTTPS) y SameSite=Lax (mitigación de ataques CSRF).
- **Control de Acceso Basado en Propiedad (Prevención de IDOR):** En cada endpoint de entrevista (/api/interview/[sessionId]/*), el servidor no solo verifica que exista una sesión activa, sino que valida explícitamente que interviewSession.userId coincida con el id del usuario autenticado en la consulta de base de datos. Ningún usuario puede consultar o alterar entrevistas ajenas aunque conozca su identificador UUID.
- **Protección de Secretos y Llaves de API:** Las variables de entorno sensibles (DATABASE_URL, BETTER_AUTH_SECRET, ELEVENLABS_API_KEY) residen exclusivamente en el entorno del servidor y nunca son expuestas al cliente a través de prefijos públicos (como NEXT_PUBLIC_).
- **Validación Estricta de Entradas con Zod:** Toda carga útil (payload) enviada por el cliente es parseada y validada mediante esquemas estrictos de Zod antes de alcanzar la lógica de negocio o la base de datos, rechazando entradas con campos no permitidos o formatos inválidos (HTTP 400).

Manejo Global de Errores y Tolerancia a Fallos

El sistema implementa una arquitectura resiliente con estrategias de degradación elegante (graceful degradation) para garantizar la continuidad del servicio ante contingencias:

- **Estrategia de Fallback en Generación de Entrevistas:** Si el webhook del sistema está caído, sobrecargado o tarda más de 5 segundos en responder, el Route Handler POST /api/interview/start captura la excepción mediante un bloque try/catch y carga automáticamente un banco curado de preguntas predefinidas según el tipo de entrevista (técnica, HR o estrés). Esto asegura que el candidato siempre pueda iniciar su práctica sin bloqueos.
- **Gestión de Fallos de Red y Reconexión en Audio:** El hook useInterviewConversation monitorea activamente la latencia del WebSocket de ElevenLabs. Ante una pérdida prolongada de paquetes o caída de red, el sistema detecta el estado "poor_connection", finaliza la sesión ordenadamente y redirige al usuario a una pantalla asistida (/ended?reason=poor-connection) con opción de reanudar.
- **Respuestas HTTP Semánticas y Estandarizadas:** Todos los Route Handlers devuelven estructuras uniformes { error: string, details?: any } con códigos de estado conformes al estándar RFC: 400 (Datos Inválidos), 401 (No Autorizado), 404 (Recurso No Encontrado), 500 (Error Interno) y 502 (Fallo de Integración Externa).
- **Auditoría y Trazabilidad en webhook_logs:** Cada llamada a servicios externos (ElevenLabs) registra en la base de datos la fecha, flujo, endpoint, carga útil enviada, respuesta recibida y mensaje de error, facilitando el diagnóstico y reintento de operaciones críticas.

Gestión de Conexiones a la Base de Datos

El acceso a PostgreSQL está optimizado para alto rendimiento y tolerancia a concurrencia:

- **Patrón Singleton para Prisma Client:** En src/lib/prisma.ts se almacena la instancia de Prisma en globalThis durante el modo de desarrollo. Esto evita que los ciclos de Hot Module Replacement (HMR) y Fast Refresh de Next.js creen conexiones concurrentes duplicadas hasta agotar el límite de sockets del servidor PostgreSQL.
- **Pool de Conexiones Optimizado:** La conexión utiliza el adaptador nativo @prisma/adapter-pg acoplado a la librería pg de Node.js, configurado para reutilizar conexiones inactivas, administrar timeouts de socket y evitar bloqueos por transacciones colgadas.
- **Integridad Referencial y Eliminación en Cascada:** El modelo de datos en schema.prisma implementa onDelete: Cascade en las relaciones hijas (questions -> responses, interview_sessions -> evaluations, recordings). Si una sesión es eliminada por solicitud del usuario, la base de datos garantiza la purga consistente de sus datos derivados sin dejar registros huérfanos.

Tabla 3 *Matriz de mecanismos de control, políticas de seguridad y gestión técnica de recursos.*

Mecanismo de Control	Tecnología / Módulo	Estrategia Técnica Implementada en Nervio
Autenticación	Better Auth + Prisma	Hashing seguro (scrypt), tokens con expiración y soporte de sesiones persistentes.
Autorización	Route Handlers / requireAuth	Validación estricta de propiedad mediante cláusula WHERE id = $id AND userId = $userId.
Protección de Sesión	Next.js Cookies API	Cookies HttpOnly, Secure y SameSite=Lax para prevenir intercepción XSS y CSRF.
Validación de Entradas	Zod Runtime Validation	Validación estricta de esquemas antes de ejecutar lógica de negocio o queries SQL.
Tolerancia a Fallos IA	Fallback Engine (lib/interview)	Banco de preguntas estáticas ante indisponibilidad del Eleven Labs.
Resiliencia de Audio	ElevenLabs React SDK	Monitoreo de estado de WebSocket, detección de desconexión y cierre asistido.
Pool de Base de Datos	Prisma Singleton + pg Adapter	Reutilización global de sockets para evitar saturación en desarrollo y producción.
Auditoría de Enlaces	Modelo webhook_logs	Persistencia de solicitudes salientes, códigos de respuesta y tiempos de ejecución.

Nota. *Diseño técnico verificado con las capacidades del proyecto Buildathon Nervio*

Matriz de Consistencia y Trazabilidad con la Fase 1

Para garantizar la total consistencia exigida en los criterios de aceptación académica, se establece la trazabilidad directa entre las historias de usuario y casos de uso aprobados en la Fase 1 y los componentes técnicos diseñados en esta fase:

Tabla 4 *Trazabilidad directa entre requerimientos de Fase 1 y subsistemas técnicos de Fase 2.*

ID Fase 1	Requerimiento / Caso de Uso	Subsistema Responsable	Componente / Módulo Técnico Implementado
NER-6 / CU-01	Registro de cuenta de candidato	Autenticación	register/page.tsx, Better Auth Handler, Modelo User/Account.
NER-7 / CU-02	Inicio de sesión seguro	Autenticación	login/page.tsx, Better Auth Adapter, Cookie Session HttpOnly.

ID Fase 1	Requerimiento / Caso de Uso	Subsistema Responsable	Componente / Módulo Técnico Implementado
NER-8 / CU-03	Recuperación de contraseña	Autenticación	Verification Table, Token Generator, Reset Password Route.
NER-9 / CU-04	Configuración de contexto	Gestión de Entrevistas	InterviewSetupForm, interviewSetupSchema (Zod).
NER-11 / CU-05	Creación de sesión persistente	Ciclo de Vida / Datos	POST /api/interview/start, Modelo InterviewSession (Prisma).
RF-INT-03 / CU-06	Simulación con entrevistador IA	Conversación y Voz	InterviewLiveClient, @elevenlabs/react ConversationProvider.
RF-INT-04 / CU-06	Visualización de orbe reactivo	Presentación	InterviewOrb, animación CSS y volumen de audio en tiempo real.
RF-INT-05 / CU-06	Modo Estrés configurable	Ciclo de Vida / Conversación	stressMode en InterviewSession y parámetros dinámicos del agente.
RF-INT-08 / CU-07	Finalización y cierre de sesión	Ciclo de Vida	POST /api/interview/[sessionId]/end, transition to ended.
RF-REP-01 / CU-08	Generación de reporte final	Automatización / Datos	elevenlabs-analysis.ts, Modelo evaluations (Scores y feedback).
RF-HIS-01 / CU-09	Consulta de historial personal	Presentación / Persistencia	interview/reports/page.tsx, Prisma findMany con filtro userId.
RF-SCH-01 / CU-10	Agendamiento de entrevista	Ciclo de Vida	POST /api/interview/schedule, Modelo schedules y webhook Eleven Labs.

Con esta partición técnica, el sistema Nervio cuenta con una arquitectura completamente especificada, modular y desacoplada, lista para la fase de construcción y para el desarrollo de los diagramas de soporte.

Diseño de la Base de Datos (Estrategia de Administración de Datos)

Modelo Relacional (MR) / Esquema Físico:

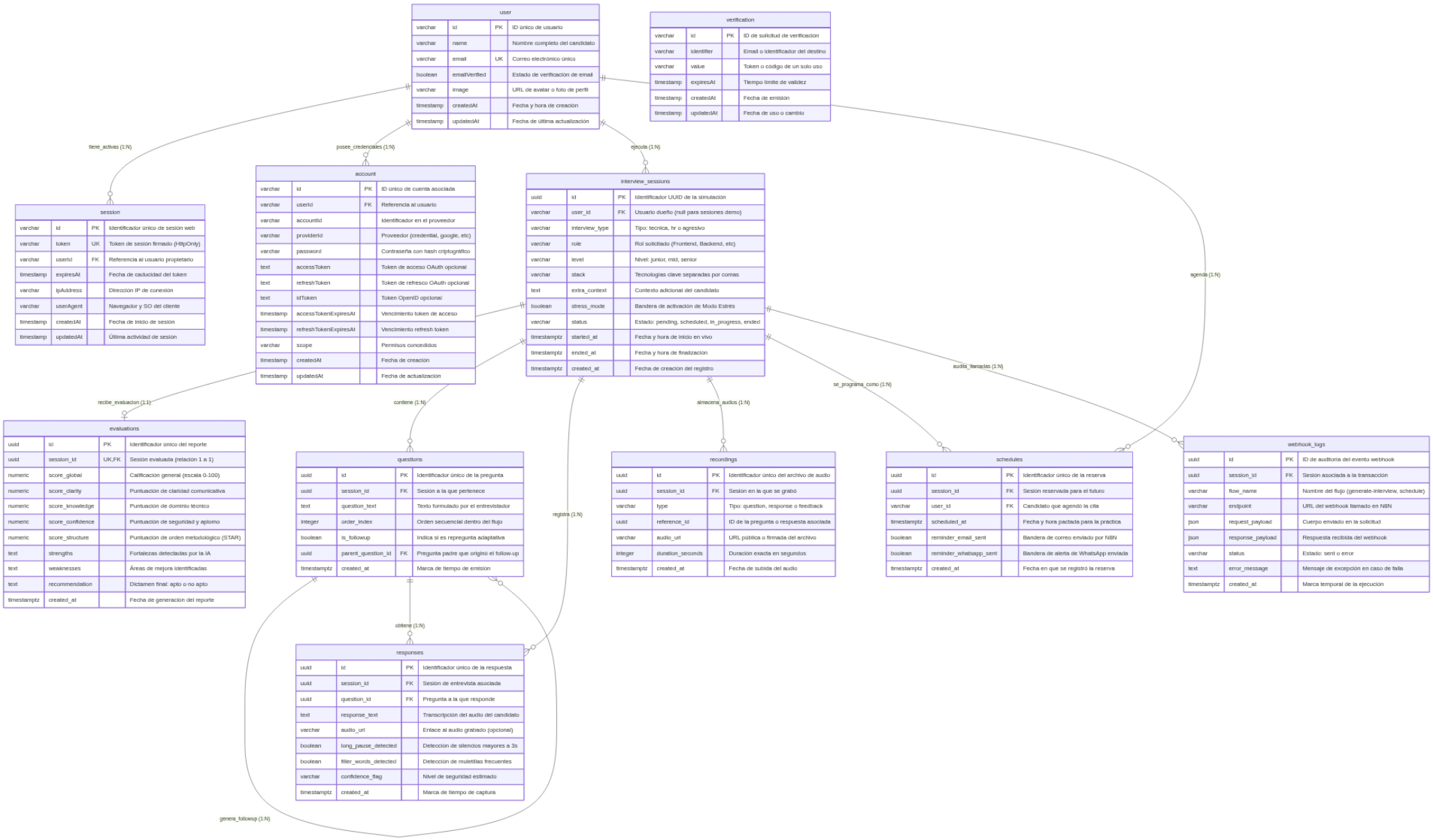
El modelo físico de datos se estructura en torno a dos núcleos funcionales interconectados:

- **Núcleo de Autenticación y Cuentas:** Compuesto por las entidades user, session, account y verification. Administra identidades de candidatos, cuentas vinculadas mediante proveedores locales o federados, y sesiones web activas.
- **Núcleo del Dominio de Entrevistas:** Compuesto por interview_sessions, questions, responses, evaluations, recordings, schedules y webhook_logs. Almacena las configuraciones de entrevista, el histórico secuencial de preguntas y respuestas, las grabaciones de audio, los reportes cuantitativos y las reservas futuras.

Reglas de Integridad Referencial y Cascadas: Para mantener la consistencia lógica de los datos y evitar registros huérfanos, el esquema aplica políticas explícitas en sus relaciones foráneas:

- **user -> session / account:** La eliminación de un candidato purga automáticamente en cascada sus credenciales y sesiones de navegación activas (onDelete: Cascade).
- **user -> interview_sessions:** La relación admite user_id nulo para posibilitar simulaciones de prueba anónimas. En caso de eliminación del usuario, se aplica onUpdate: NoAction para preservar historiales si se requiere anonimización.
- **interview_sessions -> questions, responses, evaluations, recordings, schedules, webhook_logs:** Toda entidad subordinada a una simulación implementa onDelete: Cascade. La purga de una entrevista borra coherentemente sus turnos, grabaciones, evaluaciones analíticas y registros de auditoría.
- **questions -> questions (Autorreferencia):** Permite modelar follow-ups o repreguntas adaptativas anidadas a una pregunta padre (parent_question_id).

Figura 31. Modelo Relacional (Esquema Físico) de la base de datos de Nervio en PostgreSQL.


user
varchar | id | PK | ID único de usuario
varchar | name | | Nombre completo del candidato
varchar | email | UK | Correo electrónico único
boolean | emailVerified | | Estado de verificación de email
varchar | image | | URL de avatar o foto de perfil
timestamp | createdAt | | Fecha y hora de creación
timestamp | updatedAt | | Fecha de última actualización
verification
varchar | id | PK | ID de solicitud de verificación
varchar | identifier | | Email o identificador del destino
varchar | value | | Token o código de un solo uso
timestamp | expiresAt | | Tiempo límite de validez
timestamp | createdAt | | Fecha de emisión
timestamp | updatedAt | | Fecha de uso o cambio
tiene_activas (1:N)
posee_credenciales (1:N)
ejecuta (1:N)
session
varchar | id | PK | Identificador único de sesión web
varchar | token | UK | Token de sesión firmado (HttpOnly)
varchar | userId | FK | Referencia al usuario propietario
timestamp | expiresAt | | Fecha de caducidad del token
varchar | ipAddress | | Dirección IP de conexión
varchar | userAgent | | Navegador y SO del cliente
timestamp | createdAt | | Fecha de inicio de sesión
timestamp | updatedAt | | Última actividad de sesión
account
varchar | id | PK | ID único de cuenta asociada
varchar | userId | FK | Referencia al usuario
varchar | accountId | | Identificador en el proveedor
varchar | providerId | | Proveedor (credential, google, etc)
varchar | password | | Contraseña con hash criptográfico
text | accessToken | | Token de acceso OAuth opcional
text | refreshToken | | Token de refresco OAuth opcional
text | idToken | | Token OpenID opcional
timestamp | accessTokenExpiresAt | | Vencimiento token de acceso
timestamp | refreshTokenExpiresAt | | Vencimiento refresh token
varchar | scope | | Permisos concedidos
timestamp | createdAt | | Fecha de creación
timestamp | updatedAt | | Fecha de actualización
interview_sessions
uuid | id | PK | Identificador UUID de la simulación
varchar | user_id | FK | Usuario dueño (null para sesiones demo)
varchar | interview_type | | Tipo: tecnica, hr o agresivo
varchar | role | | Rol solicitado (Frontend, Backend, etc)
varchar | level | | Nivel: junior, mid, senior
varchar | stack | | Tecnologías clave separadas por comas
text | extra_context | | Contexto adicional del candidato
boolean | stress_mode | | Bandera de activación de Modo Estrés
varchar | status | | Estado: pending, scheduled, in_progress, ended
timestamptz | started_at | | Fecha y hora de inicio en vivo
timestamptz | ended_at | | Fecha y hora de finalización
timestamptz | created_at | | Fecha de creación del registro
agenda (1:N)
audita_llamadas (1:N)
se_programa_como (1:N)
recibe_evaluacion (1:1)
contiene (1:N)
almacena_audios (1:N)
evaluations
uuid | id | PK | Identificador único del reporte
uuid | session_id | UK,FK | Sesión evaluada (relación 1 a 1)
numeric | score_global | | Calificación general (escala 0-100)
numeric | score_clarity | | Puntuación de claridad comunicativa
numeric | score_knowledge | | Puntuación de dominio técnico
numeric | score_confidence | | Puntuación de seguridad y aplomo
numeric | score_structure | | Puntuación de orden metodológico (STAR)
text | strengths | | Fortalezas detectadas por la IA
text | weaknesses | | Áreas de mejora identificadas
text | recommendation | | Dictamen final: apto o no apto
timestamptz | created_at | | Fecha de generación del reporte
questions
uuid | id | PK | Identificador único de la pregunta
uuid | session_id | FK | Sesión a la que pertenece
text | question_text | | Texto formulado por el entrevistador
integer | order_index | | Orden secuencial dentro del flujo
boolean | is_followup | | Indica si es repregunta adaptativa
uuid | parent_question_id | FK | Pregunta padre que originó el follow-up
timestamptz | created_at | | Marca de tiempo de emisión
registra (1:N)
obtiene (1:N)
genera_followup (1:N)
responses
uuid | id | PK | Identificador único de la respuesta
uuid | session_id | FK | Sesión de entrevista asociada
uuid | question_id | FK | Pregunta a la que responde
text | response_text | | Transcripción del audio del candidato
varchar | audio_url | | Enlace al audio grabado (opcional)
boolean | long_pause_detected | | Detección de silencios mayores a 3s
boolean | filler_words_detected | | Detección de muletillas frecuentes
varchar | confidence_flag | | Nivel de seguridad estimado
timestamptz | created_at | | Marca de tiempo de captura
recordings
uuid | id | PK | Identificador único del archivo de audio
uuid | session_id | FK | Sesión en la que se grabó
varchar | type | | Tipo: question, response o feedback
uuid | reference_id | | ID de la pregunta o respuesta asociada
varchar | audio_url | | URL pública o firmada del archivo
integer | duration_seconds | | Duración exacta en segundos
timestamptz | created_at | | Fecha de subida del audio
schedules
uuid | id | PK | Identificador único de la reserva
uuid | session_id | FK | Sesión reservada para el futuro
varchar | user_id | FK | Candidato que agendó la cita
timestamptz | scheduled_at | | Fecha y hora pactada para la práctica
boolean | reminder_email_sent | | Bandera de correo enviado por N8N
boolean | reminder_whatsapp_sent | | Bandera de alerta de WhatsApp enviada
timestamptz | created_at | | Fecha en que se registró la reserva
webhook_logs
uuid | id | PK | ID de auditoría del evento webhook
uuid | session_id | FK | Sesión asociada a la transacción
varchar | flow_name | | Nombre del flujo (generate-interview, schedule)
varchar | endpoint | | URL del webhook llamado en N8N
json | request_payload | | Cuerpo enviado en la solicitud
json | response_payload | | Respuesta recibida del webhook
varchar | status | | Estado: sent o error
text | error_message | | Mensaje de excepción en caso de falla
timestamptz | created_at | | Marca temporal de la ejecución

Nota. El diagrama representa la transformación física del modelo de datos normalizado implementado con Prisma ORM sobre PostgreSQL. Se distinguen dos dominios: el subsistema de autenticación (user, session, account, verification) y el dominio central de entrevistas (interview_sessions, questions, responses, evaluations, recordings, schedules y webhook_logs). Todas las relaciones dependientes cuentan con integridad referencial y eliminación en cascada (onDelete: Cascade), garantizando que la purga de una sesión elimine coherentemente preguntas, respuestas, evaluaciones y registros multimedia asociados. Fuente: elaboración propia.

Diccionario de datos

A continuación, se desglosan las especificaciones técnicas completas de cada tabla del esquema físico implementado en PostgreSQL:

Tabla 5 *Diccionario de datos: Entidad user (Cuentas y perfiles de candidatos).*

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
id	VARCHAR(255)	PK	NOT NULL	Clave Primaria	Identificador único de usuario generado por Better Auth.
name	VARCHAR(255)	-	NOT NULL	-	Nombre completo del candidato.
email	VARCHAR(255)	-	NOT NULL	UNIQUE (user_email_key)	Correo electrónico único utilizado para acceso e identificación.
emailVerified	BOOLEAN	-	NOT NULL	DEFAULT false	Indica si el correo electrónico ha sido verificado.
image	VARCHAR(500)	-	NULL	-	URL de la fotografía o avatar de perfil del usuario.
createdAt	TIMESTAMP(3)	-	NOT NULL	DEFAULT now()	Fecha y hora de creación de la cuenta en el sistema.
updatedAt	TIMESTAMP(3)	-	NOT NULL	Auto-actualizable	Fecha y hora de la última

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
					modificación del perfil.

Nota. *Tabla central de identidad de candidatos administrada por Better Auth.*

Tabla 6 *Diccionario de datos: Entidad session (Sesiones activas de navegación).*

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
id	VARCHAR(255)	PK	NOT NULL	Clave Primaria	Identificador único de la sesión web.
token	VARCHAR(500)	-	NOT NULL	UNIQUE (session_token_key)	Token criptográfico emitido y verificado en cookies HttpOnly.
userId	VARCHAR(255)	FK	NOT NULL	user(id) ON DELETE CASCADE	Usuario propietario de la sesión. Índice idx_session_userId.
expiresAt	TIMESTAMP(3)	-	NOT NULL	-	Fecha y hora límite de expiración de la sesión.
ipAddress	VARCHAR(45)	-	NULL	-	Dirección IP de conexión del cliente (soporte IPv4/IPv6).
userAgent	VARCHAR(500)	-	NULL	-	Cadena del agente de usuario (navegador y sistema operativo).
createdAt	TIMESTAMP(3)	-	NOT NULL	DEFAULT now()	Momento de emisión de la sesión.
updatedAt	TIMESTAMP(3)	-	NOT NULL	Auto-actualizable	Momento del último refresh o actividad.

Nota. *Controla el acceso persistente a las páginas protegidas mediante cookies cifradas.*

Tabla 7 *Diccionario de datos: Entidad account (Credenciales y métodos de autenticación).*

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
id	VARCHAR(255)	PK	NOT NULL	Clave Primaria	Identificador único del registro de autenticación.
accountId	VARCHAR(255)	-	NOT NULL	-	Identificador en el proveedor (correo o identificador OAuth).
providerId	VARCHAR(100)	-	NOT NULL	-	Estrategia de acceso: credential, google, github, etc.
userId	VARCHAR(255)	FK	NOT NULL	user(id) ON DELETE CASCADE	Usuario asociado. Índice idx_account_userId.
password	VARCHAR(255)	-	NULL	Hash seguro	Contraseña cifrada con algoritmo scrypt/bcrypt. Solo en credenciales locales.
accessToken	TEXT	-	NULL	-	Token de acceso temporal concedido por proveedores externos.
refreshToken	TEXT	-	NULL	-	Token de actualización otorgado por proveedores externos.
idToken	TEXT	-	NULL	-	Token de identidad OpenID Connect.
accessTokenExpiresAt	TIMESTAMP(3)	-	NULL	-	Fecha de expiración del token de acceso.

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
refreshTokenExpiresAt	TIMESTAMP(3)	-	NULL	-	Fecha de expiración del token de actualización.
scope	VARCHAR(255)	-	NULL	-	Alcance de permisos autorizados por el usuario en el proveedor.
createdAt	TIMESTAMP(3)	-	NOT NULL	DEFAULT now()	Fecha de vinculación de la cuenta.
updatedAt	TIMESTAMP(3)	-	NOT NULL	Auto-actualizable	Fecha de última renovación de tokens.

Nota. *Almacena métodos de acceso soportando esquemas locales y federados.*

Tabla 8 *Diccionario de datos: Entidad verification (Tokens de validación y restablecimiento).*

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
id	VARCHAR(255)	PK	NOT NULL	Clave Primaria	Identificador de la solicitud de verificación.
identifier	VARCHAR(255)	-	NOT NULL	-	Correo electrónico u objetivo de la verificación. Índice de búsqueda.
value	VARCHAR(255)	-	NOT NULL	-	Token o código temporal de un solo uso.
expiresAt	TIMESTAMP(3)	-	NOT NULL	-	Límite de validez del código emitido.
createdAt	TIMESTAMP(3)	-	NOT NULL	DEFAULT now()	Fecha y hora de generación del token.

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
updatedAt	TIMESTAMP(3)	-	NOT NULL	Auto-actualizable	Fecha de actualización o invalidación.

Nota. *Soporta los flujos de verificación de correo y recuperación de contraseña (NER-8).*

Tabla 9 *Diccionario de datos: Entidad interview_sessions (Simulaciones de entrevista).*

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
id	UUID	PK	NOT NULL	DEFAULT gen_random_uuid()	Identificador global de la sesión de entrevista.
user_id	VARCHAR(255)	FK	NULL	user(id) ON UPDATE NO ACTION	Candidato dueño de la sesión. Índice idx_sessions_user_id.
interview_type	VARCHAR(50)	-	NOT NULL	Check: tecnica, hr, agresivo	Modalidad y personalidad del entrevistador virtual.
role	VARCHAR(100)	-	NOT NULL	-	Rol laboral evaluado (ej. Frontend Developer, QA Engineer).
level	VARCHAR(50)	-	NOT NULL	Check: junior, mid, senior	Seniority configurado para el nivel de exigencia técnica.
stack	VARCHAR(255)	-	NULL	-	Tecnologías clave separadas por comas (ej. React, Next.js, Postgres).
extra_context	TEXT	-	NULL	-	Requisitos o aclaraciones introducidas por el candidato en el setup.
stress_mode	BOOLEAN	-	NOT NULL	DEFAULT false	Activa preguntas hostiles e interrupciones en la simulación.

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
status	VARCHAR(50)	-	NOT NULL	DEFAULT pending	Estados: pending, scheduled, in_progress, ended. Índice idx_sessions_status.
started_at	TIMESTAMPTZ(6)	-	NULL	-	Marca temporal del inicio formal de la interacción de voz.
ended_at	TIMESTAMPTZ(6)	-	NULL	-	Marca temporal de la conclusión o cierre de la sesión.
created_at	TIMESTAMPTZ(6)	-	NOT NULL	DEFAULT now()	Fecha y hora de registro de la simulación.

Nota. *Entidad central del dominio. Controla el ciclo de vida de la práctica.*

Tabla 10 *Diccionario de datos: Entidad questions (Preguntas formuladas en la sesión).*

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
id	UUID	PK	NOT NULL	DEFAULT gen_random_uuid()	Identificador único de la pregunta.
session_id	UUID	FK	NOT NULL	interview_sessions(id) CASCADE	Sesión a la que pertenece. Índice idx_questions_session_id.
question_text	TEXT	-	NOT NULL	-	Enunciado de la pregunta formulado por el entrevistador.
order_index	INTEGER	-	NOT NULL	Check: order_index >= 1	Número secuencial de la pregunta dentro de la entrevista.
is_followup	BOOLEAN	-	NOT NULL	DEFAULT false	Indica si es una repregunta derivada de la respuesta anterior.

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
parent_question_id	UUID	FK	NULL	questions(id) ON UPDATE NO ACTION	Autorreferencia a la pregunta origen que motivó el follow-up.
created_at	TIMESTAMPTZ(6)	-	NOT NULL	DEFAULT now()	Fecha y hora de formulación de la pregunta.

Nota. *Soporta preguntas base generadas por el backend y follow-ups en tiempo real.*

Tabla 11 *Diccionario de datos: Entidad responses (Respuestas del candidato).*

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
id	UUID	PK	NOT NULL	DEFAULT gen_random_uuid()	Identificador único de la respuesta.
session_id	UUID	FK	NOT NULL	interview_sessions (id) CASCADE	Sesión asociada. Índice idx_responses_session_id.
question_id	UUID	FK	NOT NULL	questions(id) CASCADE	Pregunta que se responde. Índice idx_responses_question_id.
response_text	TEXT	-	NULL	-	Transcripción textual del habla capturada mediante Whisper/ElevenLabs.
audio_url	VARCHAR(500)	-	NULL	-	Enlace al archivo de audio con la grabación de la respuesta.
long_pause_detected	BOOLEAN	-	NULL	DEFAULT false	Indicador de silencio prolongado (> 3 segundos) previo a responder.

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
filler_words_detected	BOOLEAN	-	NULL	DEFAULT false	Indicador de uso reiterado de muletillas (eh, este, um).
confidence_flag	VARCHAR(50)	-	NULL	-	Calificación estimada de seguridad (alta, media, baja).
created_at	TIMESTAMPTZ (6)	-	NOT NULL	DEFAULT now()	Marca temporal de captura de la respuesta.

Nota. *Almacena transcripciones y variables de estrés analizadas durante la interacción.*

Tabla 12 *Diccionario de datos: Entidad evaluations (Reportes y evaluaciones de desempeño).*

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
id	UUID	PK	NOT NULL	DEFAULT gen_random_uuid()	Identificador único del reporte analítico.
session_id	UUID	FK, UK	NOT NULL	interview_sessions(id) CASCADE	Sesión evaluada. UNIQUE (1:1). Índice idx_evaluations_session_id.
score_global	DECIMAL(5,2)	-	NULL	Check: 0.00 - 100.00	Puntaje ponderado general de la simulación.
score_clarity	DECIMAL(5,2)	-	NULL	Check: 0.00 - 100.00	Calificación de elocuencia y claridad comunicativa.
score_knowledge	DECIMAL(5,2)	-	NULL	Check: 0.00 - 100.00	Calificación de profundidad y precisión técnica demostrada.
score_confidence	DECIMAL(5,2)	-	NULL	Check: 0.00 - 100.00	Calificación de temple, seguridad y manejo de estrés.

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
score_structure	DECIMAL(5,2)	-	NULL	Check: 0.00 - 100.00	Calificación de organización y método (metodología STAR).
strengths	TEXT	-	NULL	-	Síntesis textual de fortalezas y aspectos destacados del candidato.
weaknesses	TEXT	-	NULL	-	Síntesis de debilidades y áreas técnicas de mejora recomendadas.
recommendation	TEXT	-	NULL	-	Dictamen cualitativo final (Apto, Prometedor, Requiere Práctica).
createdAt	TIMESTAMPTZ(6)	-	NOT NULL	DEFAULT now()	Fecha y hora de computación de la evaluación.

Nota. *Guarda los resultados cualitativos y cuantitativos generados por la IA analítica.*

Tabla 13 *Diccionario de datos: Entidad recordings (Archivos de audio de la sesión).*

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
id	UUID	PK	NOT NULL	DEFAULT gen_random_uuid()	Identificador del fragmento de audio.
session_id	UUID	FK	NOT NULL	interview_sessions(id) CASCADE	Sesión a la que pertenece. Índice idx_recordings_session_id.
type	VARCHAR(50)	-	NOT NULL	Check: question, response, feedback	Propósito del audio. Índice idx_recordings_type.

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
reference_id	UUID	-	NULL	-	Identificador de la pregunta, respuesta o reporte vinculado.
audio_url	VARCHAR(500)	-	NOT NULL	-	URL segura de acceso al archivo multimedia en almacenamiento.
duration_seconds	INTEGER	-	NULL	Check: duration_seconds >= 0	Duración exacta del audio en segundos.
created_at	TIMESTAMPTZ(6)	-	NOT NULL	DEFAULT now()	Fecha y hora de almacenamiento del recurso de audio.

Nota. *Almacena pistas de audio para reproducir la sesión completa en modo llamada.*

Tabla 14 *Diccionario de datos: Entidad schedules (Reservas de entrevistas futuras).*

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
id	UUID	PK	NOT NULL	DEFAULT gen_random_uuid()	Identificador único de la cita agendada.
session_id	UUID	FK	NULL	interview_sessions(id) CASCADE	Sesión de entrevista pre-configurada vinculada a la cita.
user_id	VARCHAR(255)	FK	NULL	user(id) ON UPDATE NO ACTION	Candidato que solicitó la reserva.
scheduled_at	TIMESTAMPTZ(6)	-	NOT NULL	Fecha futura	Fecha y hora programada para la prueba. Índice idx_schedules_scheduled_at.

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
reminder_email_sent	BOOLEAN	-	NOT NULL	DEFAULT false	Bandera de correo enviado por. Índice compuesto idx_schedules_reminders.
reminder_whatsapp_sent	BOOLEAN	-	NOT NULL	DEFAULT false	Bandera de alerta de WhatsApp enviada. Índice compuesto idx_schedules_reminders.
created_at	TIMESTAMPTZ(6)	-	NOT NULL	DEFAULT now()	Fecha y hora en que se registró la cita en la plataforma.

Nota. *Optimizado para disparar alertas automáticas de mediante crons periódicos.*

Tabla 15 *Diccionario de datos: Entidad webhook_logs (Auditoría de enlaces e integraciones).*

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
id	UUID	PK	NOT NULL	DEFAULT gen_random_uuid()	Identificador de la transacción auditada.
session_id	UUID	FK	NULL	interview_sessions (id) CASCADE	Sesión ligada al evento. Índice idx_webhook_logs_session_id.
flow_name	VARCHAR(100)	-	NOT NULL	-	Nombre del flujo remoto (generate-interview, schedule-notification). Índice idx_webhook_logs_flow_name.

Columna	Tipo de Dato	PK / FK	Nulo	Restricción / Regla	Descripción e Índices
endpoint	VARCHAR(500)	-	NOT NULL	-	URL exacta del webhook llamado en el motor de.
request_payload	JSON / JSONB	-	NULL	-	Cuerpo de la petición HTTP transmitido al servicio externo.
response_payload	JSON / JSONB	-	NULL	-	Respuesta recibida del webhook externo.
status	VARCHAR(50)	-	NOT NULL	DEFAULT sent	Estado de la llamada: sent o error.
error_message	TEXT	-	NULL	-	Detalle o traza del fallo en caso de respuesta anómala.
created_at	TIMESTAMPTZ(6)	-	NOT NULL	DEFAULT now()	Fecha y hora exacta de la invocación del webhook.

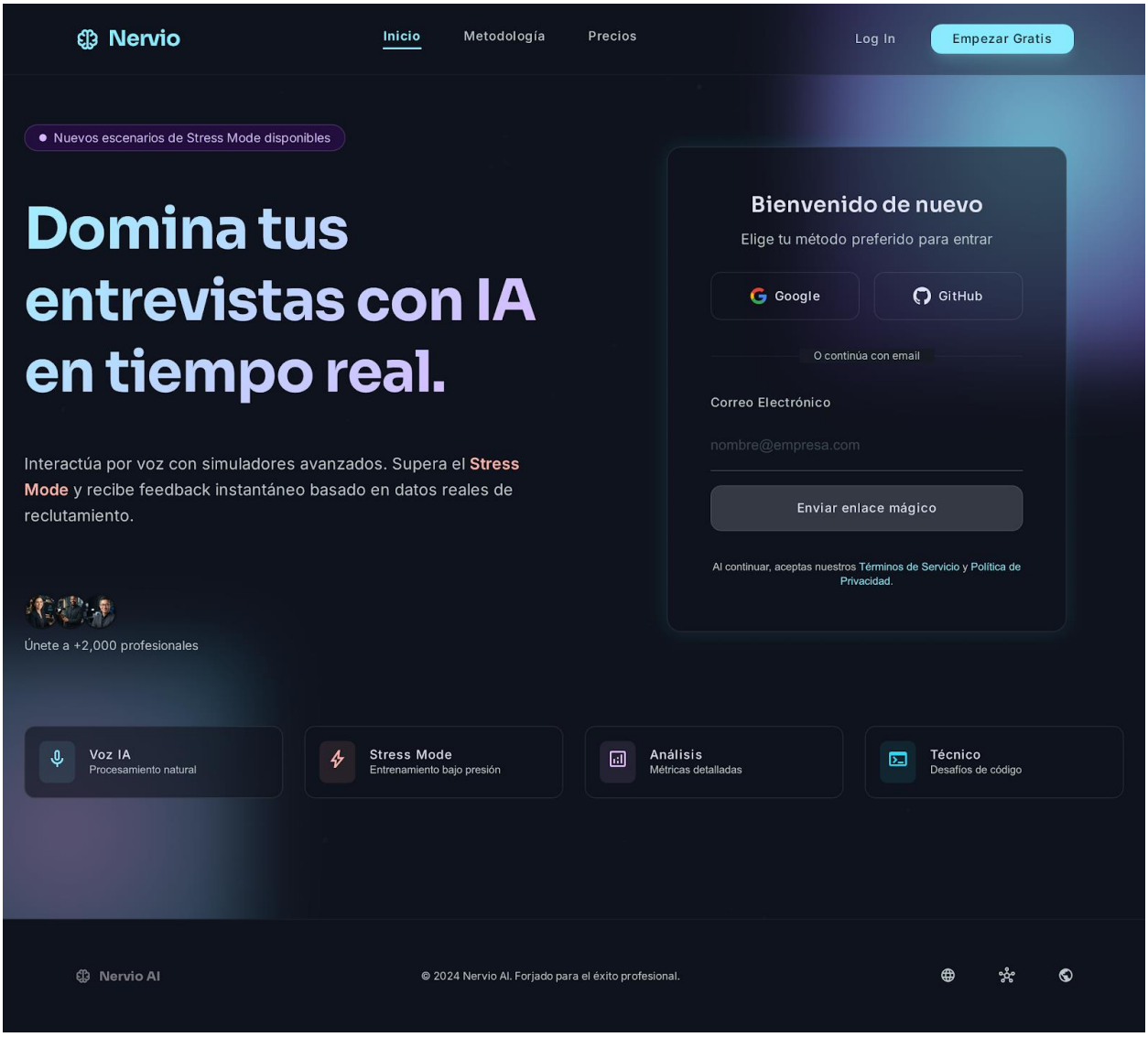
Nota. *Garantiza la trazabilidad y auditoría de eventos asíncronos sin bloquear al usuario.*

Con estas especificaciones físicas y relacionales, la base de datos de Nervio cumple con la totalidad de los criterios de normalización en Tercera Forma Normal (3FN), integridad referencial robusta y optimización mediante índices especializados.

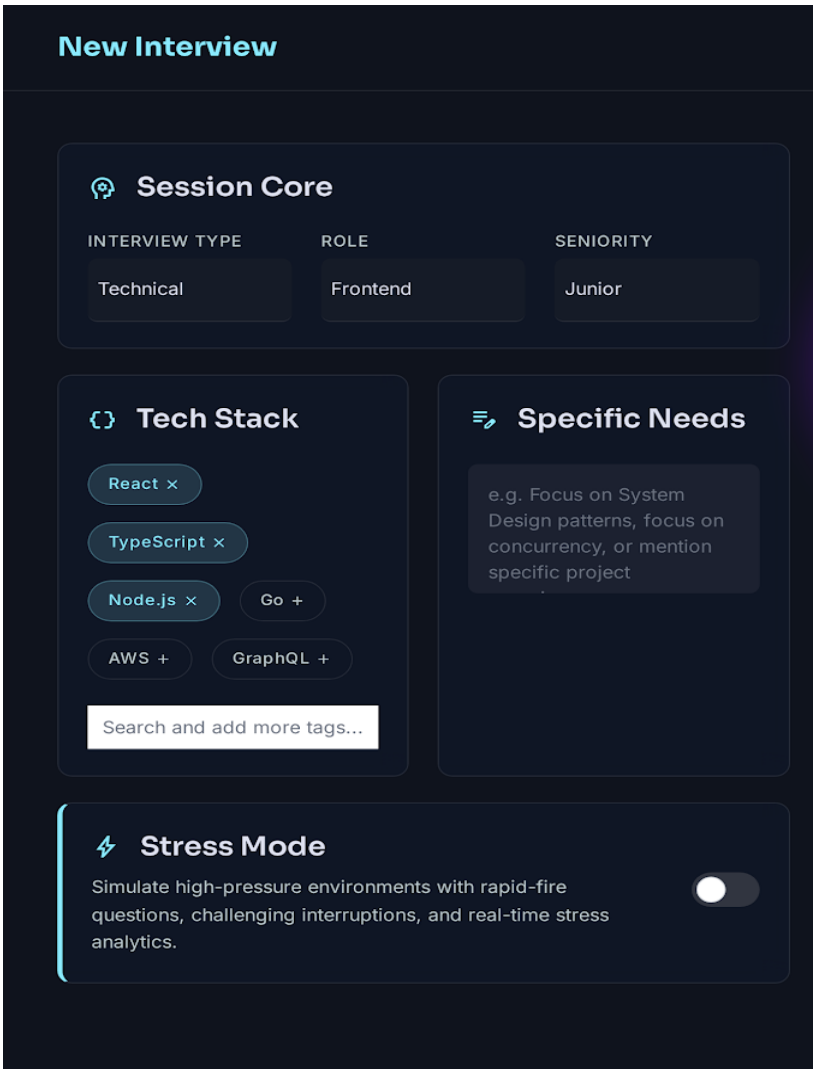
Diseño de la interfaz de usuario (UI/UX)

Prototipos de alta fidelidad / wireframes

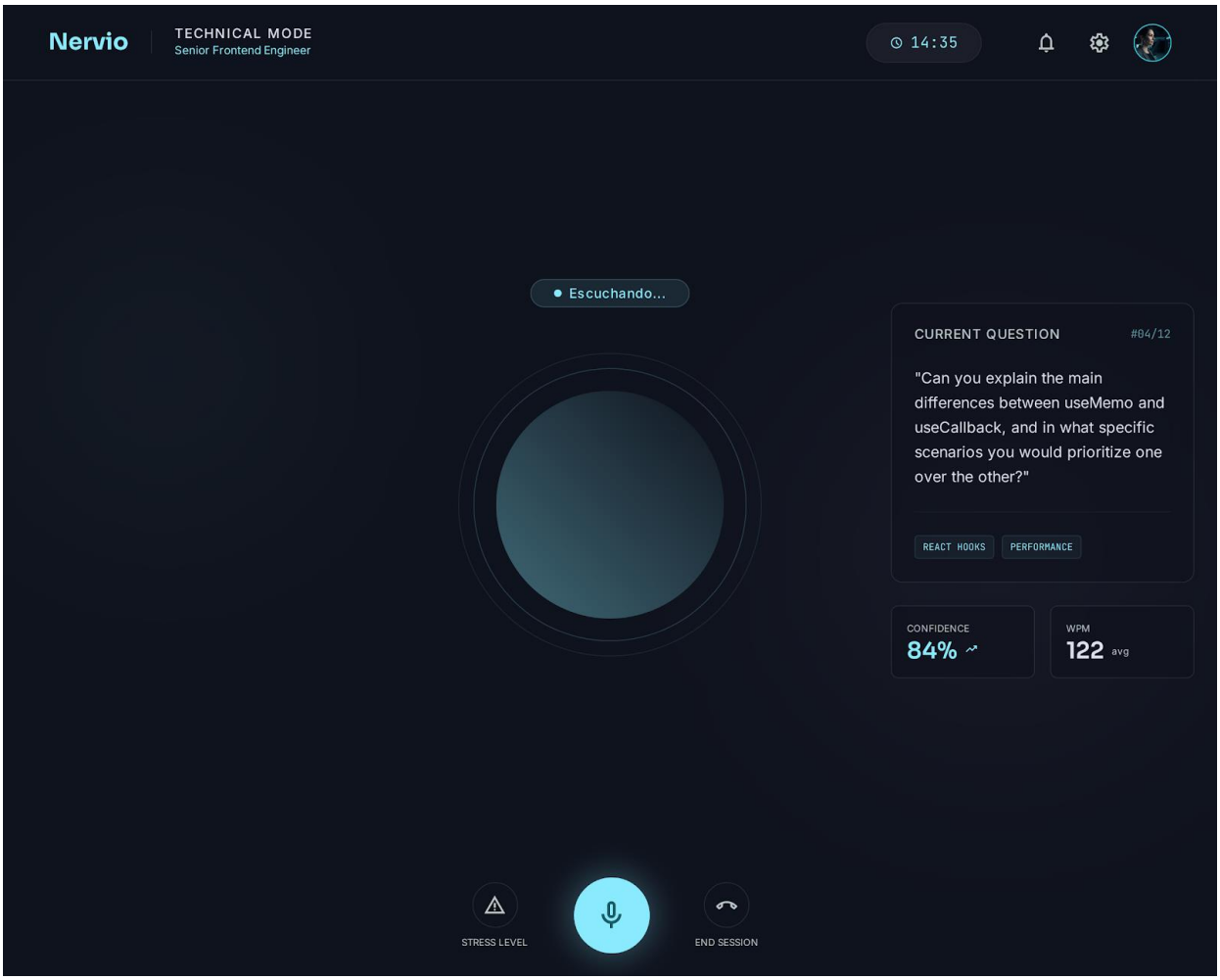
Diseño de la página principal: Será la vista mediante la cual los usuarios serán recibidos en la página.



Formulario para poder configurar la entrevista, mediante este formulario el usuario podrá configurar la entrevista a su elección, modificando el tipo, tecnología e incluso si es modo estrés.



UI de la llamada: Como se espera que sea la llamada de la aplicación, la cual buscará ser minimalista para no distraer al usuario sin olvidar el efecto futurista que buscamos resaltar.



Guia de estilos y navegación.

1. INTRODUCCIÓN Y FUNDAMENTOS DE MARCA

Nervio es una plataforma de simulación de entrevistas técnicas y de comportamiento de alto rendimiento impulsada por inteligencia artificial en tiempo real.

Filosofía Visual

- **Enfoque Cognitivo y Concentración:** Interfaz en modo oscuro profundo (#0A0E18 a #171B26) formulada para reducir la fatiga visual durante simulaciones prolongadas y eliminar el ruido perceptual.
- **Respuesta en Tiempo Real y Tensión Calibrada:** El uso de acentos cian/neón (#22D3EE / #38BDF8) actúa como el núcleo vital de la IA, representando claridad, precisión y escucha activa.
- **Sofisticación Técnica:** Geometría depurada con esquinas redondeadas calibradas (8px a 16px), tarjetas con efecto cristalino (backdrop-filter / surface containers) y jerarquía

tipográfica rigurosa orientada a la rápida asimilación de métricas de desempeño y retroalimentación inmediata.

2. GUÍA DE ESTILO Y SISTEMA DE DISEÑO (DESIGN TOKENS)

2.1. Paleta de Colores Oficial

La paleta cromática se estructura en tres capas principales: **Superficies y Fondos**, **Acentos de Interacción e IA**, y **Estados Semánticos y Métricas**.

A. Superficies y Contenedores (Dark Surface Layering)

- **Canvas / Background Primario (surface-container-lowest):** #0A0E18
  - *Uso:* Fondo base de la aplicación completa y pantallas inmersivas como la simulación en vivo.
- **Superficie de Aplicación (surface / surface-dim):** #0F131D
  - *Uso:* Fondos de dashboards, áreas de trabajo y layout general.
- **Contenedor Base (surface-container-low):** #171B26
  - *Uso:* Paneles secundarios, barras de navegación lateral y encabezados fijos.
- **Contenedor de Tarjetas (surface-container):** #1C1F2E
  - *Uso:* Módulos principales, formularios de configuración, tarjetas de calendario y bloques de preguntas.
- **Bordes y Delimitadores:** rgba(255, 255, 255, 0.08) a rgba(255, 255, 255, 0.12)
  - *Uso:* Líneas divisorias de tarjetas, tableros y separadores de navegación sin sobrecargar el contraste visual.

B. Colores Primarios y Acentos de Marca (Brand & AI Core)

- **Primario Neón (primary):** #22D3EE (Cyan / Eléctrico)
  - *Uso:* Esfera activa de la IA, botones de llamada a la acción primarios (*"Iniciar Simulación"*, *"Save Schedule"*), estados activos de pestañas, barras de progreso clave y microchips de alta prioridad.
- **Primario Secundario / Gradiente (primary-glow):** #38BDF8 / #06B6D4
  - *Uso:* Resplandores radiales (glow effects), gradientes de fondo ambiental y estados *hover*.
- **Acento de Red Neuronal / IA Secundario (accent-violet):** #818CF8 / #A855F7

- *Uso:* Resplandores difusos del asistente virtual, badges especiales y elementos de automatización.

C. Tipografía y Contraste de Texto

- **Texto Primario (on-surface):** #F8FAFC (Blanco Slate puro)
  - *Uso:* Títulos principales, preguntas activas del entrevistador, métricas grandes y valores clave.
- **Texto Secundario (on-surface-variant):** #94A3B8 (Gris Slate balanceado)
  - *Uso:* Subtítulos, descripciones secundarias, labels de formulario y navegación inactiva.
- **Texto Atenuado / Placeholder:** #64748B
  - *Uso:* Textos de apoyo, marcas de tiempo en el timeline y campos vacíos.

D. Estados Semánticos y Modos

- **Modo Estrés / Alerta (stress-accent):** #F43F5E (Rose / Carmesí)
  - *Uso:* Tono visual del Modo Estrés cuando el simulador detecta inseguridad o interrumpe al candidato.
- **Éxito / Calificación Alta (success):** #34D399 (Esmeralda)
  - *Uso:* Diagnóstico de estado "Apto / Recomendado", porcentajes de claridad superiores al 80%.
- **Precaución / Áreas de Crecimiento (warning):** #FBBF24 (Ámbar)
  - *Uso:* Indicadores de verbosidad bajo presión y llamadas de atención en el análisis de voz.

2.2. Tipografía y Jerarquía

- **Familia Tipográfica Principal:** Sora (Google Fonts)
  - *Racional de diseño:* Tipografía geométrica sans-serif con terminaciones limpias y curvas optimizadas para pantallas digitales. Proporciona una identidad moderna, técnica y con alta legibilidad en interfaces de comando de voz y dashboards analíticos.
- **Familia Tipográfica de Respaldo:** Inter, system-ui, -apple-system, sans-serif.

Escala Tipográfica (Type Scale)

1. **Display / Hero Headline:**

- Tamaño: 40px – 48px | Peso: 700 (Bold) | Interlineado: 1.15
- Aplicación: Titular de impacto en bienvenida (*"Domina tus entrevistas con IA en tiempo real."*).

2. **Title Large / Page Title:**
   - Tamaño: 28px – 32px | Peso: 700 (Bold) | Interlineado: 1.2
   - Aplicación: Títulos de sección como *"New Interview"*, *"Schedule"*, *"Report: Senior Frontend Dev"*.
3. **Headline Medium / Scores:**
   - Tamaño: 22px – 24px | Peso: 600 (SemiBold) / 700 (Bold)
   - Aplicación: Títulos de módulos (*"Session Core"*, *"Key Strengths"*, *"Session Playback"*) y marcadores de porcentaje (85/100).
4. **Body Regular / Question Text:**
   - Tamaño: 15px – 17px | Peso: 400 (Regular) / 500 (Medium) | Interlineado: 1.6
   - Aplicación: Enunciado de preguntas formuladas por la IA, citas del reporte y descripciones operativas.
5. **Labels / UI Meta:**
   - Tamaño: 12px – 13px | Peso: 600 (SemiBold) | Transformación: Uppercase con tracking suave (tracking-wider)
   - Aplicación: Badges de tags (REACT HOOKS, SYSTEM DESIGN), etiquetas de campos de formulario y cabeceras de tabla.

3. MAPA DE SITIO Y ARQUITECTURA DE INFORMACIÓN

La plataforma está diseñada bajo un modelo híbrido: **Landing pública con onboarding directo** + **Suite de Aplicación Web con barra de navegación lateral persistente** para los módulos operativos + **Modo Pantalla Completa / Enfoque (Focus Mode)** durante la simulación activa.

NERVIO (Arquitectura de Información)

│

├── 1.0 ACCESO Y LANDING PÚBLICA

│ ├── Hero & Propuesta de Valor (Voz, Métricas, Stress Mode)

│ ├── Acceso / Autenticación (Google OAuth, GitHub OAuth, Magic Link)

```
│  └── Prueba Rápida / CTA a Configuración
│
└── 2.0 CONSOLA DE APLICACIÓN (Workspace)
   │
   ├── 2.1 Configuración de Entrevista (New Interview)
   │   ├── Parámetros: Tipo (Técnico / HR / Estrés), Rol, Seniority
   │   ├── Selector de Stack Tecnológico (Tags interactivos)
   │   ├── Campo contextual de necesidades específicas
   │   ├── Toggle "Stress Mode"
   │   └── Acciones: [Iniciar Simulación Ahora] | [Reservar Sesión]
   │
   ├── 2.2 Entorno de Simulación en Vivo (Live Practice Core)
   │   ├── Esfera Reactiva de IA (Audio Visualizer con feedback visual)
   │   ├── Transcriptor dinámico y estado de escucha ("Escuchando...")
   │   └── Barra de control de llamada (Mute Mic, Stress Trigger, Finalizar)
   │
   ├── 2.3 Reporte Final y Reproductor (Post-Session Review)
   │   ├── Score Global (Índice 0-100 y veredicto "Apto / Recomendado")
   │   ├── Desglose de competencias (Claridad, Técnico, Confianza, Estructura)
   │   ├── Resumen de IA y habilidades detectadas
```

4. FLUJO DE NAVEGACIÓN PASO A PASO (USER FLOW)

Flujo 1: Acceso e Incorporación (Onboarding)

- **Entrada:** El usuario ingresa a Nervio - Bienvenida y Acceso.
- **Acción:** Puede consumir la propuesta de valor o autenticarse directamente mediante Better Auth (Google, GitHub o Magic Link sin contraseña).
- **Destino:** Al autenticarse con éxito, se redirige de manera predeterminada al configurador de entrevistas (Nervio - Configurar Entrevista).

Flujo 2: Creación e Inicio de Simulación Inmediata

1. **Configuración:** En Nervio - Configurar Entrevista, el usuario parametriza su sesión:
    - Define el tipo de entrevista (*Technical*, *Behavioral*, etc.).
    - Establece su Rol (*Frontend Engineer*) y nivel de experiencia (*Senior*).
    - Agrega etiquetas a su stack técnico (*React, TypeScript, Node.js*).
    - Ingresa requerimientos específicos en el área de texto.
    - Activa o desactiva el interruptor **Stress Mode**.
2. **Disparo:** El usuario hace clic en el CTA primario: **"Iniciar Simulación"**.
3. **Transición:** El sistema conmuta de la vista de dashboard a la interfaz inmersiva de concentración total (Nervio - Simulación en Vivo), ocultando la barra lateral para maximizar el foco.

Flujo 3: Simulación en Vivo e Interacción por Voz

1. **Inicio de llamada:** La esfera de IA cobra vida con modulación luminosa cian, reproduciendo la voz de síntesis (ElevenLabs).
2. **Interacción:** El usuario responde por voz utilizando su micrófono; Whisper convierte el audio en texto y la lógica de backend calibra la confianza y velocidad (WPM).
3. **Cierre de simulación:** Al completar las preguntas asignadas o pulsar el botón rojo de **"End Session"**, se activa el webhook hacia el flujo de consolidación en.
4. **Destino:** La interfaz navega de forma automática al reporte final de la sesión (Nervio - Reporte Final y Reproductor).

Flujo 4: Análisis Post-Entrevista y Reproducción

1. **Revisión de resultados:** En Nervio - Reporte Final y Reproductor, el aspirante evalúa su porcentaje global y las 4 dimensiones clave.
2. **Decisión de seguimiento:**
    - Si desea mejorar de inmediato: hace clic en **"Start Practice"** en la barra lateral para abrir un nuevo setup.
    - Si desea calendarizar un ensayo posterior: hace clic en la pestaña **"Schedule"** de la barra lateral.

    FONTS:

    - Primary Font: 'Sora', sans-serif

- Fallbacks: 'Inter', system-ui, sans-serif

PALETA DE COLORES (HEX):

- Dark Background (Lowest): #0A0E18
- App Surface (Dim): #0F131D
- Panel & Sidebar Container: #171B26
- Card & Widget Container: #1C1F2E
- Surface Elevated / Bright: #353944
- Brand Primary (Cyan): #22D3EE
- Brand Glow (Cyan/Sky): #38BDF8
- Secondary Accent (Purple): #818CF8
- Text Primary (High Contrast):#F8FAFC
- Text Secondary (Slate): #94A3B8
- Stress Mode / Critical: #F43F5E
- Success / High Score: #34D399
- Warning / Growth Area: #FBBF24

Mapa de pantallas

1. **Nervio - Landing Page y Bienvenida (Pública)**
    - **Ruta real:** / (src/app/page.tsx)
    - **Descripción:** Presentación institucional de la plataforma, orbe interactivo de muestra y accesos directos hacia el registro o inicio de sesión.
2. **Nervio - Inicio de Sesión / Autenticación (Pública)**
    - **Ruta real:** /login (src/app/login/page.tsx)
    - **Descripción:** Formulario de ingreso mediante correo electrónico y contraseña gestionado con Better Auth, con redirección protegida a la configuración de entrevista.
3. **Nervio - Registro de Candidato (Pública)**
    - **Ruta real:** /register (src/app/register/page.tsx)

- **Descripción:** Formulario de alta de nuevos usuarios con validación de credenciales y creación de perfil.

4. **Nervio - Configurar Entrevista (Protegida)**
    - **Ruta real:** /interview/setup (src/app/interview/setup/page.tsx)
    - **Descripción:** Formulario para parametrizar la simulación: tipo de entrevista (técnica, HR, modo agresivo), rol objetivo, nivel de seniority, tecnologías (stack), contexto adicional y activación de modo estrés.
5. **Nervio - Simulación en Vivo (Protegida)**
    - **Ruta real:** /interview/[sessionId] (src/app/interview/[sessionId]/page.tsx)
    - **Descripción:** Cabina de conversación por voz bidireccional en tiempo real con el agente IA (ElevenLabs vía WebSockets), orbe visual reactivo a la voz, cronómetro y controles de llamada (silenciar micrófono, finalizar).
6. **Nervio - Cierre y Procesamiento de Sesión (Protegida)**
    - **Ruta real:** /interview/[sessionId]/ended (src/app/interview/[sessionId]/ended/page.tsx)
    - **Descripción:** Pantalla de transición post-entrevista que confirma la culminación de la sesión y canaliza al usuario hacia su reporte.
7. **Nervio - Reporte Analítico Individual (Protegida)**
    - **Ruta real:** /interview/[sessionId]/report (src/app/interview/[sessionId]/report/page.tsx)
    - **Descripción:** Evaluación detallada de la entrevista con métricas cuantitativas (calificación global, claridad, profundidad técnica, manejo de estrés, método STAR), resumen de fortalezas, debilidades y dictamen cualitativo.
8. **Nervio - Historial General de Reportes (Protegida)**
    - **Ruta real:** /interview/reports (src/app/interview/reports/page.tsx)
    - **Descripción:** Panel histórico donde el candidato visualiza la lista cronológica de todas sus simulaciones previas, puntajes obtenidos y acceso a cada reporte individual.
9. **Nervio - Agendamiento de Sesiones (Protegida)**
    - **Ruta real:** /interview/schedules (src/app/interview/schedules/page.tsx)

- **Descripción:** Interfaz para programar fechas y horas futuras de simulaciones, sincronizada con flujos asíncronos del servidor de Next Js para el envío de recordatorios automáticos por correo electrónico o mensajería.

Mapeo de clases y responsabilidades técnicas

El propósito de esta sección es formalizar la arquitectura de software interna de la plataforma Nervio mediante la transición del modelo de dominio conceptual (Fase 1) a un modelo de diseño técnico orientado a objetos y estructurado en capas. En este apartado se definen la visibilidad de atributos, los tipos de datos en TypeScript, las firmas completas de métodos, así como la especificación detallada de los Controladores de API (Route Handlers), Servicios de Negocio y Repositorios de Acceso a Datos.

Arquitectura de Clases y Principios de Diseño

El diseño técnico de Nervio adopta el patrón arquitectónico Controller-Service-Repository (C-S-R) adaptado al ecosistema de Next.js (App Router) y TypeScript. Esta separación desacopla la recepción y validación de peticiones HTTP, la orquestación lógica del negocio y la persistencia de base de datos relacional.

Principios y Patrones Aplicados:

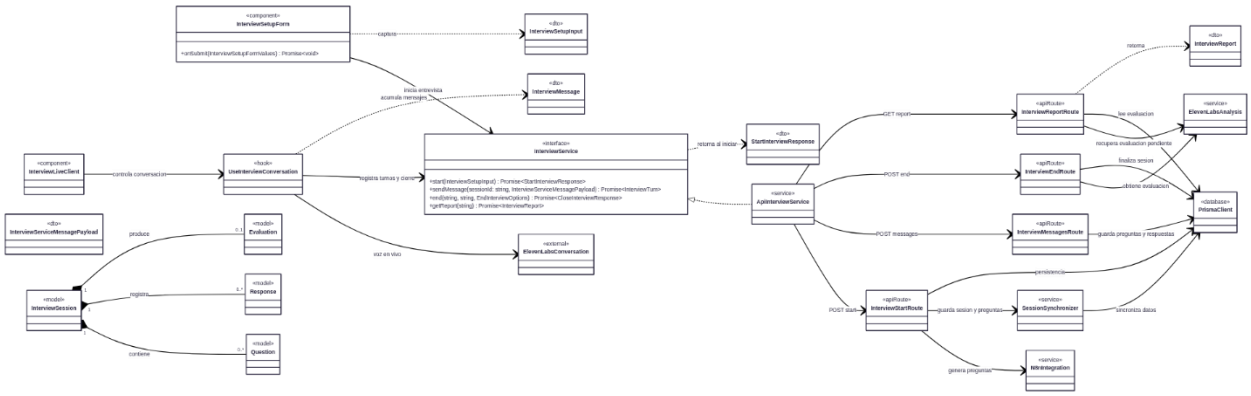
- **Principio de Responsabilidad Única (SRP):** Cada componente posee un único motivo de cambio. Los controladores validan contratos HTTP y seguridad; los servicios coordinan turnos, llamadas a IA y lógica de negocio; y los repositorios encapsulan transacciones de base de datos.
- **Inversión de Dependencias (DIP) y Patrón Service Factory:** La interfaz InterviewService define el contrato abstracto del cliente de entrevistas. Mediante service-factory se conmuta dinámicamente entre la implementación en vivo (ApiInterviewService) y la de pruebas offline (MockInterviewService) sin alterar la capa de presentación.
- **Patrón Singleton:** Implementado en PrismaClientSingleton para garantizar una única instancia activa del pool de conexiones a PostgreSQL en tiempo de ejecución, evitando la saturación de conexiones por Hot Module Replacement.
- **Validación Fuerte en Tiempo de Ejecución (DTOs con Zod):** Los objetos de transferencia de datos (Data Transfer Objects) se validan estrictamente mediante esquemas Zod

(interviewSetupSchema, scheduleInterviewSchema) antes de que el flujo alcance los servicios del dominio.

Diagrama de Clases de Diseño (DCD)

El Diagrama de Clases de Diseño representa la estructura estática del software, especificando la visibilidad (+ público, - privado, # protegido), multiplicidades, interfaces, métodos con parámetros tipados y valores de retorno basados en promesas asíncronas (Promise<T>).

Figura 32 *Diagrama de Clases de Diseño de la plataforma Nervio.*



Link del diagrama mermaid

Nota. *Representa la organización en cuatro capas: Controladores (Route Handlers), Servicios de Negocio e Integración, Repositorios de Persistencia y Modelos de Entidad / DTOs. Elaboración propia en base a la arquitectura TypeScript y Prisma ORM.*

Definición de Controladores y APIs (Capa de Control)

Los controladores corresponden a los Route Handlers de Next.js situados en src/app/api/*. Son responsables de interceptar las solicitudes entrantes, autenticar la sesión del cliente mediante Better Auth, validar las cargas útiles con Zod y transformar las excepciones de negocio en códigos de estado HTTP estandarizados (RFC 7231).

Tabla 1 *Especificación técnica de Controladores y APIs (Route Handlers).*

Controlador / Endpoint	Método HTTP	Parámetros / Payload Recibido	Validación / Seguridad	Servicios Invocados	Respuesta HTTP y Código
InterviewStartController (/api/interview/start)	POST	InterviewSetupInput: { interviewType, role, level, stack, extraContext }	auth.api.getSession() interviewSetupSchema.safeParse()	AutomationService SessionSyncService ElevenLabsVoiceService	200 OK: StartInterviewResponse 400: Datos inválidos 401: No autorizado 502: Error /Fallback
InterviewMessageController (/api/interview/[id]/messages)	POST	Path: id (UUID) Body: { role: string, text: string }	auth.api.getSession() Verificación de propiedad	ResponseRepository QuestionRepository	200 OK: InterviewTurn 400: Parámetros vacíos 401: No autorizado 404: Sesión no encontrada
InterviewEndController (/api/interview/[id]/end)	POST	Path: id (UUID) Body: { conversationId, reason }	auth.api.getSession() Verificación de propiedad	InterviewSessionRepository ElevenLabsAnalysisService ReportingEngineService	200 OK: CloseInterviewResponse 401: No autorizado 500: Fallo de cierre
InterviewReportController (/api/interview/[id]/report)	GET	Path: id (UUID)	auth.api.getSession() Verificación de propiedad	EvaluationRepository InterviewSessionRepository	200 OK: InterviewReport 401: No autorizado 404: Reporte inexistente
ScheduleController (/api/interview/schedule)	POST	ScheduleInterviewInput: { date, time, email, role, level }	auth.api.getSession() scheduleInterviewSchema.safeParse()	ScheduleRepository AutomationService	200 OK: ScheduleResponse 400: Fecha inválida 401: No autorizado

Controlador / Endpoint	Método HTTP	Parámetros / Payload Recibido	Validación / Seguridad	Servicios Invocados	Respuesta HTTP y Código
ElevenLabsTokenController (/api/elevenlabs/token)	GET	Headers de sesión del usuario	auth.api.getSession() Custodia de ELEVENLABS_API_KEY	ElevenLabsVoiceService	200 OK: { token: string } 401: No autorizado 500: Error de emisión
AuthController (/api/auth/[...all])	POST / GET	Credenciales (email, password) o solicitudes de sesión	Better Auth Internal Handler Rate Limiting & Hashing	UserRepository SessionRepository AccountRepository	200 OK: Cookie HttpOnly 400: Credenciales inválidas 500: Error de auth

Nota. *Todos los controladores aplican validación estricta de sesión y verificación de propiedad del usuario para prevenir vulnerabilidades de referencia directa insegura a objetos (IDOR).*

Definición de Servicios de Negocio (Capa de Dominio)

Los servicios de negocio encapsulan las reglas algorítmicas, la gestión del ciclo de vida de las sesiones y la coordinación de integraciones externas (ElevenLabs). Se caracterizan por su independencia de la capa de presentación y su capacidad de ejecución asíncrona no bloqueante.

Tabla 2 *Especificación técnica de Servicios de Negocio y Lógica de Dominio.*

Servicio de Negocio	Responsabilidad Principal	Métodos Principales (Firma Técnica)	Dependencias Clave	Estrategia de Resiliencia / Fallback
InterviewService (Interface)	Contrato abstracto de operaciones de	+ start(input: InterviewSetupInput): Promise<StartInterviewResponse> + sendMessage(sessionId: string, payload):	Tipos de dominio DTO	Permite desacoplar y conmutar implementaciones de

Servicio de Negocio	Responsabilidad Principal	Métodos Principales (Firma Técnica)	Dependencias Clave	Estrategia de Resiliencia / Fallback
	entrevista para clientes frontend y lógica de control.	Promise<InterviewTurn> + end(sessionId: string, convId?: string): Promise<CloseInterviewResponse> + getReport(sessionId: string): Promise<InterviewReport> + schedule(input: ScheduleInput): Promise<ScheduleResponse>		prueba (Mock) y producción (API) dinámicamente.
ApiInterviewService	Implementación real de InterviewService consumiendo Route Handlers y almacenamiento en sesión.	+ start(input): Promise<StartInterviewResponse> + getOpeningQuestion(sessionId): Promise<string> + sendMessage(sessionId, payload): Promise<InterviewTurn> + end(sessionId, convId, opts): Promise<CloseInterviewResponse> + getReport(sessionId): Promise<InterviewReport>	Fetch API SessionStorage Manager	Reintento de red ante fallos de conexión y persistencia local en caché si la red fluctúa.
SessionSyncService (sync-session.ts)	Sincroniza el estado de las entrevistas generadas en con la persistencia en Prisma.	+ syncSessionToPrisma(Resp: Response, setup: SetupInput, userId: string): Promise<void>	PrismaClient InterviewSession Model Questions Model	Transacción upsert: si la sesión existe la reactiva a in_progress; limpia preguntas previas y registra el nuevo set.
ElevenLabsVoiceService (elevenlabs.ts)	Generación de síntesis de voz inicial y mediación de tokens de WebSocket para el	+ synthesizeSpeech(text: string, voiceId?: string): Promise<ArrayBuffer> + arrayBufferToDataUrlNode(buffer: ArrayBuffer): string + getSignedWebSocketToken(): Promise<string>	ElevenLabs REST API Node.js Buffer	Si falla la generación de audio TTS del primer mensaje, el flujo continúa en modo puramente textual sin

Servicio de Negocio	Responsabilidad Principal	Métodos Principales (Firma Técnica)	Dependencias Clave	Estrategia de Resiliencia / Fallback
	agente conversacional.			interrumpir la sesión.
ElevenLabsAnalysisService (elevenlabs-analysis.ts)	Consulta asíncrona de reportes y análisis conversacionales del agente de voz externo.	+ rememberElevenLabsConversation(sessionId, convId): Promise<void> + getRememberedElevenLabsConversationId(sessionId): Promise<string | null> + fetchElevenLabsEvaluation(convId, attempts): Promise<LocalEvaluation | null>	ElevenLabs ConvAI API Webhook Logs Repository	Polling inteligente con 8 intentos espaciados en 1.5s; persistencia del conversationId en webhook_logs para reanudación.
ReportingEngineService (reporting.ts)	Construcción y ponderación de calificaciones analíticas (STAR, elocuencia, conocimientos y estrés).	+ buildElevenLabsEvaluation(analysis: ConversationAnalysis): LocalEvaluation + computeLocalFallbackEvaluation(messages: Message[]): LocalEvaluation	Métricas cuantitativas Modelos de dominio	Motor analítico fallback que genera una evaluación heurística estructurada a partir del chat transcript si la API de IA no responde.
AutomationService	Orquestación de webhooks asíncronos para generación de preguntas dinámicas y	+ callGenerateInterview(payload: Payload): Promise<Response> + callScheduleNotification(payload: SchedulePayload): Promise<void>	HTTP Webhook client Webhook Logs Repository	Fallback Engine: ante timeout o HTTP 5xx de, despacha el banco de preguntas por defecto (getMockQuestions) y

Servicio de Negocio	Responsabilidad Principal	Métodos Principales (Firma Técnica)	Dependencias Clave	Estrategia de Resiliencia / Fallback
	recordatorios.			registra el error.

Nota. *Los servicios implementan patrones de degradación elegante garantizando que el usuario jamás quede bloqueado ante caídas de proveedores de IA.*

Definición de Repositorios de Acceso a Datos (Capa de Persistencia)

La capa de persistencia se implementa mediante Prisma ORM sobre PostgreSQL. Los repositorios encapsulan las consultas relacionales, garantizan el aislamiento transaccional y aplican políticas de integridad física mediante tipos generados en tiempo de compilación.

Tabla 3 *Especificación técnica de Repositorios y Acceso a Datos (Prisma ORM).*

Repositorio / Entidad	Entidad de Base de Datos	Métodos Técnicos de Persistencia	Filtros y Restricciones	Comportamiento en Cascada
InterviewSessionRepository	interview_sessions (InterviewSession)	+ create(data: PrismaSessionCreate): Promise<Session> + findById(id: string): Promise<Session | null> + findByUserId(userId: string): Promise<Session[]> + updateSta	Filtro estricto por userId para control de acceso. Status: pending, scheduled, in_progress, ended.	Al eliminar una sesión, se purgan automáticamente en cascada sus questions, responses, evaluations y recordings.

Repositorio / Entidad	Entidad de Base de Datos	Métodos Técnicos de Persistencia	Filtros y Restricciones	Comportamiento en Cascada
		tus(id: string, status: Status): Promise<Session> + endSession(id: string, endedAt: Date): Promise<Session>		
QuestionRepository	questions (Questions)	+ createMany(questions: QuestionCreate[]): Promise<Batch> + findBySessionId(sessionId: string): Promise<Question[]> + deleteBySessionId(sessionId: string): Promise<Batch> + createFollowup(data: FollowupCreate): Promise<Question>	Ordenamiento ascendente por order_index. Relación autorreferencial con parent_question_id.	Subordinada a interview_sessions mediante onDelete: Cascade.
ResponseRepository	responses (Responses)	+ create(data: ResponseCreate):	Vinculación obligatoria a session_id	Subordinada a interview_session

Repositorio / Entidad	Entidad de Base de Datos	Métodos Técnicos de Persistencia	Filtros y Restricciones	Comportamiento en Cascada
		Promise<Response> + findBySessionId(sessionId: string): Promise<Response[]> + updateMetrics(id: string, flags: StressFlags): Promise<Response>	y question_id. Captura de long_pause y filler_words.	s mediante onDelete: Cascade.
EvaluationRepository	evaluations (Evaluations)	+ upsertBySessionId(sessionId: string, data: EvalData): Promise<Evaluation> + findBySessionId(sessionId: string): Promise<Evaluation | null>	Restricción de unicidad 1:1 con session_id (idx_evaluations_session_id). Validación de rangos 0.00-100.00.	Subordinada a interview_sessions mediante onDelete: Cascade.
ScheduleRepository	schedules (Schedules)	+ create(data: ScheduleCreate): Promise<Schedule> + findUpcoming(now: Date):	Índice sobre scheduled_at. Índice compuesto sobre banderas de recordatorio.	Subordinada a interview_sessions mediante onDelete:

Repositorio / Entidad	Entidad de Base de Datos	Métodos Técnicos de Persistencia	Filtros y Restricciones	Comportamiento en Cascada
		Promise<Schedule[]> + markReminderSent(id: string, channel: string): Promise<Schedule>		Cascade.
User Repository	user (User)	+ findById(id: string): Promise<User | null> + findByEmail(email: string): Promise<User | null> + updateProfile(id: string, data: Profile): Promise<User>	Restricción UNIQUE sobre campo email. Gestión automatizada por Better Auth.	Al eliminar un usuario se purgan sus sesiones de login y credenciales en cascade.
WebhookLogsRepository	webhook_logs (WebhookLogs)	+ logRequest(data: WebhookLogCreate): Promise<Log> + updateResponse(id: string, resp: JSON): Promise<Log> + findLatestByFlow(sessionId:	Índices sobre sessionId y flowName. Almacenamiento de payloads en JSONB.	Subordinada a interview_sessions mediante onDelete: Cascade.

Repositorio / Entidad	Entidad de Base de Datos	Métodos Técnicos de Persistencia	Filtros y Restricciones	Comportamiento en Cascada
		string, flow: string): Promise<Log | null>		

Nota. *Todos los repositorios emplean la instancia compartida de PrismaClientSingleton para maximizar la reutilización del pool de conexiones.*

Matriz de Responsabilidades y Patrones de Asignación (GRASP / SOLID)

Se sintetiza la justificación técnica de la distribución de responsabilidades en las capas de Nervio conforme a los patrones GRASP (General Responsibility Assignment Software Patterns):

Tabla 4 *Asignación de responsabilidades y patrones GRASP en el diseño de clases.*

Patrón / Principio	Componente Aplicado	Mecanismo y Justificación Arquitectónica
Controlador (Controller)	Route Handlers (/api/*)	Intercepta eventos de la UI, valida DTOs y delega el flujo sin implementar lógica de cálculo.
Bajo Acoplamiento (Low Coupling)	InterviewService (Interface)	La UI solo interactúa con la interfaz abstracta, desacoplándose de la implementación HTTP o mock.
Alta Cohesión (High Cohesion)	ElevenLabsAnalysisService	Especializado exclusivamente en consultar, esperar y parsear reportes del agente de voz.
Fabricación Pura (Pure Fabrication)	SessionSyncService	Clase de servicio creada para alinear las estructuras de datos de

Patrón / Principio	Componente Aplicado	Mecanismo y Justificación Arquitectónica
		Servicios del Backend con el modelo relacional de Prisma.
Experto en Información (Information Expert)	ReportingEngineService	Posee el conocimiento algorítmico y las ponderaciones para calcular el puntaje global y STAR.
Polimorfismo (Polymorphism)	ApiInterviewService vs MockInterviewService	Comportamiento polimórfico configurable para pruebas unitarias sin dependencias externas.

Nota. *Garantiza la extensibilidad futura para nuevos tipos de entrevistadores o proveedores de voz.*

Con esta especificación exhaustiva de clases, controladores, servicios y repositorios, el diseño técnico de Nervio satisface con rigurosidad los estándares de calidad de software

Bibliografías

American Psychological Association. *(S. f.). Figure Setup*. *APA style*. (2021, diciembre). apastyle.apa.org. Recuperado 1 de septiembre de 2026, de https://apastyle.apa.org/style-grammar-guidelines/tables-figures/figures

APA Tables and Figures - Purdue OWL® - Purdue University. (s. f.). Recuperado 1 de septiembre de 2026, de https://owl.purdue.edu/owl/research_and_citation/apa_style/apa_formatting_and_style_guide/apa_tables_and_figures.html

Building a backlog | Atlassian Support | Atlassian Documentation. (s. f.). Atlassian Documentation. Recuperado 1 de septiembre de 2026, de https://confluence.atlassian.com/jirasoftware/building-a-backlog-1688898526.html

Atlassian. (2020, 20 febrero). *Use your scrum backlog*. Atlassian Support. Recuperado 1 de septiembre de 2026, de https://support.atlassian.com/jira-software-cloud/docs/use-your-scrum-backlog/

Prisma Data, Inc. (s. f.). *Prisma schema*. Prisma. Recuperado 1 de septiembre de 2026, de https://www.prisma.io/docs/orm/v6/prisma-schema/overview

Prisma Data, Inc. (s. f.-a). *Models*. Prisma. Recuperado 1 de septiembre de 2026, de https://www.prisma.io/docs/orm/v7/prisma-schema/data-model/models

Email & Password | Better Auth. (s. f.). Better Auth. Recuperado 1 de septiembre de 2026, de https://better-auth.com/docs/authentication/email-password

Authentication - OWASP Cheat Sheet Series. (s. f.). Recuperado 1 de septiembre de 2026, de https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Forgot Password - OWASP Cheat Sheet Series. (s. f.). Recuperado 1 de septiembre de 2026, de https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html

Anexos

Urls de jira:

backlog

board